logSguarDian: Desarrollo de una Librería para la Detección de Amenazas de Seguridad en Endpoints de Aplicaciones

Diego Pablo Valenzuela Palacios



UVG UNIVERSIDAD DEL VALLE DE GUATEMALA

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



logSguarDian: Desarrollo de una Librería para la Detección de Amenazas de Seguridad en Endpoints de Aplicaciones

Trabajo de graduación presentado por Diego Pablo Valenzuela Palacios
para optar al grado académico de Licenciado en Ingeniería en Ciencia de la Computación y Tecnologías de la Información

Guatemala,

2026

Vo.Bo.:

(f) _____
Ing. Erick Francisco Marroquín Rodríguez

Tribunal Examinador:

(f) _____
Ing. Erick Francisco Marroquín Rodríguez

(f) _____
Ing. Bacilio Alexander Bolaños Lima

Fecha de aprobación: Guatemala, _____ de _____ de 2026.

Prefacio

El proyecto nació de la idea del Ing. Wilson Iván Félix Reyna, primer jefe del autor, de construir una librería que combinara inteligencia artificial y ciberseguridad. Se desarrolló entre mayo y octubre de 2026 junto con Sebastián Huertas Gómez, quien implementó el extractor inicial y el lado npm de la librería y desarrolló el componente de ciberseguridad, mientras que el autor desarrolló el componente de aprendizaje automático. Por recomendación del asesor, la evaluación se amplió a cuatro aplicaciones de terceros. Durante la redacción, la verificación de cada cifra contra el repositorio llevó a corregir afirmaciones previas, y esas correcciones y sus fuentes se documentan en los anexos. Para el autor, el proyecto reunió en un solo producto buena parte de lo aprendido durante la carrera: aprendizaje automático, ingeniería de software y seguridad.

Este documento reporta el componente de aprendizaje automático de logSguarDian, desarrollado junto con Sebastián Huertas Gómez, cuya tesis aborda el componente de ciberseguridad y la librería. El trabajo se limita a cuatro categorías de ataque intra-petición, a un entorno de prueba controlado y a aplicaciones de terceros evaluadas con muestras pequeñas. No incluye la validación con una startup guatemalteca prevista en el protocolo, que era opcional. Los modelos cuyas métricas se reportan (`rf_v11` e `if_v10`) se publicaron el 11 de octubre de 2026 (UTC) en la versión 0.1.1 del paquete; las evaluaciones de integración del capítulo de Resultados no usaron esa versión; el anexo XIII.8 recoge dos corridas del coautor que sí la usaron. Las limitaciones que acotan las cifras principales se declaran en los capítulos de Resultados y Discusión y en los anexos.

Declaración de uso de inteligencia artificial. El proyecto, su diseño y sus decisiones técnicas son del autor: definió el enfoque, seleccionó los modelos, analizó y justificó cada característica del extractor, fijó el contrato de entrada, desarrolló el entrenamiento, la serialización ONNX y el pipeline de reentrenamiento, concibió el pipeline de MLOps y el análisis de interpretabilidad con SHAP, y decidió el alcance y los resultados que se reportan. Para el código de entrenamiento y de análisis, la documentación técnica y la redacción empleó a Claude, de Anthropic, como herramienta de apoyo, mediante conversación y mediante un agente de código. El autor aportó el contenido y las instrucciones; Claude redactó y reformuló borradores, los convirtió a LaTeX y verificó la coherencia entre capítulos. El autor dirigió ese trabajo, revisó los textos, contrastó las cifras con sus fuentes, puede explicar el contenido y asume la responsabilidad del documento.

El autor agradece al Ing. Wilson Iván Félix Reyna, su primer jefe en su primer trabajo, quien propuso la idea original de una librería que combine inteligencia artificial y ciberseguridad. Agradece

al Ing. Erick Francisco Marroquín Rodríguez, asesor de este trabajo, por su guía y por la recomendación de evaluar el sistema en aplicaciones de terceros. Agradece al Ing. Bacilio Alexander Bolaños Lima, catedrático, quien lo acompañó durante el desarrollo de este proyecto, y a Sebastián Huertas Gómez, compañero de proyecto.

Agradece también a los catedráticos que lo formaron durante la carrera y a la Universidad del Valle de Guatemala, su casa de estudios. Agradece a sus compañeros de carrera y a sus amigos por el acompañamiento durante estos años. Agradece a sus padres y a su hermana por su apoyo. Agradece a las comunidades de código abierto cuyos conjuntos de datos y aplicaciones vulnerables hicieron posible la evaluación, entre ellos SecLists, PayloadsAllTheThings, OWASP Juice Shop, DVNA, dvws-node y node-api-goat.

Índice

Prefacio **iii**

Lista de figuras **ix**

Lista de cuadros **x**

Lista de abreviaturas y siglas **xiii**

Resumen **xv**

Abstract **xvi**

I. Introducción **1**

II. Antecedentes **3**
II.1. Introducción al estado del arte . . . 3
II.2. Detección de intrusiones basada en aprendizaje automático: panorama general . . 3
II.3. Algoritmos base del modelo híbrido: Random Forest e Isolation Forest . . . 5
II.4. Metodologías de construcción y etiquetado de datasets . . . 5
II.5. Restricciones de inferencia embebida y Edge AI . . . 6
II.6. Soluciones perimetrales existentes y su respaldo documental . . . 6
II.7. Síntesis comparativa . . . 7
II.8. Conclusión: vacíos identificados y aporte del proyecto . . . 7

III. Justificación **9**
III.1. Importancia del problema . . . 9
III.2. Relevancia técnica y científica . . . 10
III.3. Relevancia social, económica e institucional . . . 10
III.4. Beneficiarios de la investigación . . . 11
III.5. Síntesis . . . 11

IV. Objetivos **12**
IV.1. Objetivo general . . . 12
IV.2. Objetivos específicos . . . 12

V. Alcance **14**
V.1. Alcance funcional . . . 14
V.2. Alcance técnico . . . 15
V.3. Alcance operativo y de evaluación . . . 15
V.4. Elementos fuera del alcance . . . 16
V.5. Restricciones, condiciones y supuestos . . . 16
V.6. Coherencia y viabilidad del alcance . . . 17

VI. Marco teórico **19**
VI.1. Introducción . . . 19
VI.2. Panorama de amenazas en la capa de aplicación . . . 19
VI.3. Node.js, Express y el modelo event-driven . . . 20
VI.4. Middleware y Runtime Application Self-Protection (RASP) . . . 21
VI.5. Ecosistema npm y seguridad en la cadena de suministro . . . 21
VI.6. Panorama de ciberseguridad en Guatemala y el ecosistema tech local . . . 22
VI.7. Aprendizaje automático supervisado y no supervisado en ciberseguridad . . . 23
VI.7.1. Random Forest . . . 23
VI.7.2. Isolation Forest . . . 24
VI.8. Redes neuronales ligeras (MLP) y autoencoders . . . 24
VI.9. Ingeniería de características para datos de seguridad HTTP . . . 25
VI.10. Detección de anomalías y ataques de día cero . . . 26
VI.11. Métricas de evaluación: F1-score, AUC-ROC y matriz de confusión . . . 26
VI.12. Edge AI e inferencia en tiempo de ejecución con ONNX . . . 27
VI.13. Construcción y etiquetado de datasets de tráfico HTTP . . . 28
VI.14. Síntesis: articulación de los fundamentos en la arquitectura de logSguarDian . . . 29

VII. Metodología **30**
VII.1. Enfoque y tipo de investigación . . . 30
VII.2. Metodología de trabajo y fases . . . 31
VII.3. Técnicas e instrumentos . . . 32
VII.3.1. Pares de modelos evaluados . . . 34
VII.3.2. Validación en aplicaciones de terceros . . . 36
VII.4. Recolección, procesamiento y validación de datos . . . 36
VII.5. Métricas e indicadores . . . 38
VII.6. Matriz de trazabilidad . . . 39
VII.7. Recursos, participantes y restricciones . . . 41

VIII. Resultados **43**
VIII.1. Resultados del objetivo específico 1: pipeline de ingeniería de características . . . 43
VIII.1.1. Latencia del extractor en aislamiento . . . 43
VIII.1.2. Sobrecosto absoluto del middleware completo . . . 44
VIII.2. Resultados del objetivo específico 2: modelo híbrido Random Forest e Isolation Forest . . . 44
VIII.2.1. Random Forest . . . 45
VIII.2.2. Isolation Forest . . . 46
VIII.2.3. Paridad entre scikit-learn y ONNX . . . 46
VIII.3. Resultados del objetivo específico 3: evaluación integrada en las configuraciones 1, 2, 3a y 3b . . . 46

VIII.3.1. Config 1: aplicación sin protección . . . 47
VIII.3.2. Config 2: solo logSguarDian . . . 47
VIII.3.3. Config 3a y Config 3b: corpus pequeño . . . 49
VIII.3.4. Config 3a y Config 3b: corpus grande . . . 50
VIII.3.5. Solapamiento del corpus de 590 payloads con el entrenamiento . . . 50
VIII.3.6. Evaluación sobre un corpus sin solapamiento (Ronda 5) . . . 51
VIII.3.7. Falsos positivos en aplicaciones de terceros . . . 53
VIII.3.8. Consumo de memoria y CPU . . . 53
VIII.4. Resultados de interpretabilidad: análisis SHAP . . . 54
VIII.4.1. Importancia global por clase . . . 54
VIII.4.2. Explicaciones locales . . . 58
VIII.5. Limitaciones de los resultados . . . 58

IX. Discusión de resultados **61**
IX.1. Pipeline de ingeniería de características . . . 61
IX.2. Modelo híbrido . . . 61
IX.3. Falsos positivos y generalización . . . 62
IX.4. Cobertura en la evaluación integrada . . . 62
IX.5. Latencia, memoria y CPU . . . 63
IX.6. Comparación con trabajos previos . . . 64
IX.7. Cumplimiento de los objetivos . . . 64

X. Conclusiones **65**

XI. Recomendaciones **67**
XI.1. Infraestructura complementaria: un pipeline de MLOps para reentrenamiento continuo . . . 67
XI.2. Ampliación del corpus benigno . . . 69
XI.3. Validación de la versión publicada . . . 69
XI.4. Costo de memoria y CPU por componente . . . 69
XI.5. Corpus de evaluación fuera del entrenamiento . . . 70
XI.6. Rigor del protocolo de evaluación . . . 70

XII. Bibliografía **71**

XIII. Anexos **74**
XIII.1. Revalidación del umbral de decisión de Random Forest y aporte de Isolation Forest 74
XIII.1.1. Revalidación del umbral de Random Forest . . . 74
XIII.1.2. Aporte de Isolation Forest . . . 75
XIII.1.3. Dependencia de Isolation Forest respecto del User-Agent . . . 75
XIII.1.4. Limitaciones . . . 76
XIII.1.5. Procedencia . . . 77
XIII.2. Casi-duplicados entre particiones . . . 77
XIII.2.1. Parámetros . . . 77
XIII.2.2. Resultados . . . 78
XIII.2.3. Desempeño en validación según haya o no vecino en entrenamiento . . . 79
XIII.2.4. Cobertura de `cmdi` en la Ronda 5 sin los payloads con vecino . . . 79
XIII.2.5. Limitaciones . . . 80

XIII.2.6. Lectura . . . 80
XIII.2.7. Procedencia . . . 82
XIII.3. Procedencia de las particiones de prueba . . . 82
XIII.3.1. Procedencia . . . 83
XIII.4. Longitud del *User-Agent* por clase . . . 83
XIII.4.1. Procedencia . . . 83
XIII.5. Generalización entre fuentes (exclusión por fuente con la configuración de rf_v11) 84
XIII.5.1. Protocolo . . . 84
XIII.5.2. Resultados por fuente . . . 85
XIII.5.3. Composición de payload_full . . . 85
XIII.5.4. Lectura acotada . . . 86
XIII.5.5. Notas sobre dos fuentes . . . 86
XIII.5.6. Limitaciones . . . 87
XIII.5.7. Procedencia . . . 87
XIII.6. Reconstrucción de la query en el middleware . . . 88
XIII.6.1. Historia del cambio . . . 88
XIII.6.2. Versiones medidas . . . 88
XIII.6.3. Paquetes publicados en npm . . . 88
XIII.6.4. Réplica de la Ronda 5 . . . 89
XIII.6.5. Limitaciones . . . 90
XIII.6.6. Procedencia . . . 90
XIII.7. Métricas de rf_v10 e if_v9 . . . 90
XIII.8. Evaluaciones posteriores del coautor sobre la Ronda 5 . . . 92
XIII.8.1. Escaneo con OWASP ZAP . . . 93

Lista de figuras

1. Importancia SHAP global, clase `benign`. 55
2. Importancia SHAP global, clase `sqli`. 56
3. Importancia SHAP global, clase `xss`. 56
4. Importancia SHAP global, clase `path_traversal`. 57
5. Importancia SHAP global, clase `cmdi`. 57
6. Explicación local de `GET /profile` sin *User-Agent* (clase `xss`). 58
7. Explicación local de `GET /profile` con *User-Agent* (clase `benign`). 58
8. Explicación local de una petición de `sqli` detectada (clase `sqli`). 59

Lista de cuadros

1. Síntesis comparativa de antecedentes 7
2. Elementos fuera del alcance del proyecto 18
3. Comparación entre arquitectura WAF perimetral y RASP embebida 22
4. Grupos de características del vector de 76 dimensiones 33
5. Pares de modelos evaluados 35
6. Matriz de trazabilidad del proyecto 41
7. Latencia del extractor de características en aislamiento, por *fixture* y serie de ejecución. 44
8. Latencia p95 con y sin middleware y diferencia absoluta, por entorno medido. . . . 45
9. Métricas de Random Forest (`rf_v11`) por clase sobre el conjunto de prueba bloqueado. 45
10. Métricas de Isolation Forest (`if_v10`) sobre el conjunto de prueba bloqueado. . . . 46
11. Diferencia máxima de salida entre scikit-learn y ONNX sobre 1 000 muestras. El reporte registra `parity_passed: true`. 46
12. Vectores de ataque comprometidos en Config 1, sin protección. 47
13. Cobertura de detección por categoría en Config 2, 100 payloads por categoría. . . . 47
14. Incremento relativo del p95 de Config 2 (indicador complementario), por compilación del middleware. Las generaciones medidas son `rf_v9`/`if_v8` y `rf_v10`/`if_v9`; no corresponden a `rf_v11`/`if_v10`. 48
15. Payloads bloqueados por configuración y categoría sobre el corpus de 590 payloads de SecLists. 50
16. Payloads de 200 por categoría que coinciden con cada partición, según el criterio de solapamiento. Unión: payloads que coinciden con al menos una partición. 51
17. Cobertura de Config 2 sobre el corpus de la Ronda 5, con 200 solicitudes por categoría, por par de modelos. 52
18. Memoria y CPU del contenedor con y sin middleware, por escenario (ventana A). . 53
19. Características de mayor importancia SHAP global por clase de `rf_v11` y posición de las características de *User-Agent*. 55
20. Valores SHAP locales de las características de *User-Agent* y mayor contribución, para la clase predicha de cada caso. 58

21. Cumplimiento de los criterios de aceptación por objetivo específico y criterio del protocolo. 64

22. Barrido del umbral de decisión de Random Forest (`rf_v11`) sobre el conjunto de validación de 57 481 filas. Las columnas por clase son recall en porcentaje. 75
23. Aporte de Isolation Forest sobre el conjunto de validación con $T = 0.35$ y regla hipotética "RF o IF". 76
24. Sumas SHA-256 de los modelos evaluados, de los archivos de evidencia y de los scripts que los generaron o regeneraron. 77
25. Casi-duplicados de la partición de validación frente al entrenamiento (57 463 filas con texto). 78
26. Casi-duplicados de la partición de prueba frente al entrenamiento (57 462 filas con texto). Solo se comparó el texto; el conjunto de prueba no se puntuó. 78
27. Casi-duplicados del entrenamiento frente a sí mismo, excluida la propia fila (268 158 filas con texto). 78
28. Casi-duplicados de la Ronda 5 (200 peticiones por clase) frente al entrenamiento. . 79
29. Bloqueos y recall en validación, separados por la existencia de un vecino en entrenamiento con $J \geq 0.8$. Modelo par C (`rf.onnx` e `if.onnx`; sin F1). 79
30. Cobertura de `cmdi` en la Ronda 5, con y sin los payloads que tienen un vecino en entrenamiento. 80
31. Sumas SHA-256 del script, de los resultados de la auditoría de casi-duplicados y de sus entradas. 81
32. Cadena de locks del conjunto de prueba. Los tres commits del 23 de septiembre aplican el mismo cambio de lock. 82
33. Soporte por clase de la partición de prueba de v10 y diferencia observada en la de v11. 82
34. Sumas SHA-256 de los archivos de procedencia de las particiones de prueba. . . . 83
35. Longitud del *User-Agent* por clase, en entrenamiento y validación combinados. . . 83
36. Sumas SHA-256 de los archivos de evidencia y de las particiones de entrenamiento y validación. 84
37. Exclusión por fuente: bloqueo de ataques y falsos positivos benignos sobre todas las filas de cada fuente (train + val), con intervalo de Wilson al 95 %. 85
38. Validación de cada fuente grande: (a) `rf.onnx` par C; (b) control de mismo tamaño, media [mín, máx] de 5 semillas; (c) exclusión de la fuente. Los corchetes de (a) y (c) son intervalos de Wilson al 95 %. 85
39. `payload_full`: filas `sqli` de train + val ($n = 9\,224$) según la clase que predice el modelo de exclusión. 86
40. Sumas SHA-256 de los scripts, de las salidas y de las entradas de la auditoría por exclusión de fuente. 87
41. Versiones del middleware medidas y forma de reconstrucción de la query en cada una. 88
42. Versiones del paquete logsguardian publicadas en npm. 89
43. Réplica de la Ronda 5 (600 peticiones): diferencias en las características del vector de Random Forest, cambios de clase y de decisión con $T = 0.35$. 89
44. Cadenas de consulta en las que las derivaciones anterior y de `dc380e3` difieren, y casos de control en los que coinciden. 90
45. Sumas SHA-256 de la réplica de la Ronda 5 y de los archivos de paquete y de fuentes 91
46. Métricas de `rf_v10` sobre la partición de prueba (57 308 filas). 91
47. Recall de `if_v9` por clase y global, sobre prueba y sobre validación. 91

48. Bloqueos por categoría en las mediciones del coautor sobre la Ronda 5. 92
49. Procedencia de los archivos copiados a `_evidencia/coautor-ronda5-v10/` y de los tarballes citados. 94

Lista de abreviaturas y siglas

Sigla	Significado
API	interfaz de programación de aplicaciones (Application Programming Interface)
AUC-ROC	área bajo la curva ROC (Receiver Operating Characteristic)
CAPEC	Common Attack Pattern Enumeration and Classification
CLI	interfaz de línea de comandos
CmdI	inyección de comandos (Command Injection)
CPU	unidad central de procesamiento
CRS	OWASP Core Rule Set
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DDoS	denegación de servicio distribuida
F1	media armónica de la precisión y el recall
FN, FP, VN, VP	falsos negativos, falsos positivos, verdaderos negativos y verdaderos positivos
HTTP	Hypertext Transfer Protocol
IDS	sistema de detección de intrusiones
IF	Isolation Forest
IoT	Internet de las Cosas
JSON	JavaScript Object Notation
LFI	inclusión local de archivos (Local File Inclusion)
LOSO	Leave-One-Source-Out (exclusión por fuente)
LSH	Locality-Sensitive Hashing
ML	aprendizaje automático (Machine Learning)
MLOps	Machine Learning Operations
MLP	perceptrón multicapa (Multilayer Perceptron)
OE	objetivo específico
ONNX	Open Neural Network Exchange
OWASP	Open Worldwide Application Security Project
p95	percentil 95
PL1	nivel de paranoia 1 de ModSecurity (Paranoia Level 1)
PR	pull request (solicitud de integración de cambios)
RASP	Runtime Application Self-Protection

Sigla	Significado
REST	Representational State Transfer
RF	Random Forest
ROC	Receiver Operating Characteristic
RSS	Resident Set Size
SHA	Secure Hash Algorithm
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SQL	Structured Query Language
SQLi	inyección SQL
URL	Uniform Resource Locator
vCPU	CPU virtual
WAF	cortafuegos de aplicaciones web (Web Application Firewall)
WSL2	Windows Subsystem for Linux 2
XSS	Cross-Site Scripting
Unidades	
MB	megabyte (10^6 bytes)
MiB	mebibyte (2^{20} bytes; 150 MB equivalen a unos 143 MiB)
GB	gigabyte (10^9 bytes)
GiB	gibibyte (2^{30} bytes)

Resumen

Las startups que desarrollan servicios sobre Node.js rara vez cuentan con un equipo de seguridad dedicado. Este trabajo presenta logSguarDian, una librería npm que implementa un middleware de autoprotección en tiempo de ejecución (RASP) para Express y detecta inyección SQL, *Cross-Site Scripting*, *Path Traversal* e inyección de comandos. Un Random Forest, entrenado con 268 240 de las 383 202 filas del corpus, decide el bloqueo, y un Isolation Forest, entrenado solo con tráfico benigno, registra anomalías sin bloquear.

El extractor requiere 0.0273 ms (p95; límite 5 ms). El Random Forest alcanza un F1 macro de 0.9776 y el Isolation Forest, un recall de 0.9125 con 0.0546 de falsos positivos. La cobertura por categoría supera el 80 % en inyección SQL, *Path Traversal* e inyección de comandos. La latencia cumple el criterio de 15 ms en carga normal y de ataque, pero no en volumen, y no se cumplen los criterios de memoria y CPU.

El objetivo general se cumplió de forma parcial. Con un *User-Agent* de navegador, el modelo validado marcó para bloqueo el 97.1 % de las peticiones benignas en Juice Shop (modo *monitor*) y un modelo no documentado bloqueó el 92.5 % en dvws-node; el corpus benigno es la debilidad principal. Las métricas miden separación dentro del corpus y no generalización, porque las particiones comparten casi-duplicados y el conjunto de prueba se leyó tres veces. Se recomienda repetir las mediciones con la versión 0.1.1, ampliar el corpus benigno y medir el costo por componente.

Abstract

Technology startups building on Node.js rarely have a dedicated security team. This work presents logSguarDian, an npm library that implements a runtime application self-protection (RASP) middleware for Express and detects SQL injection, cross-site scripting, path traversal, and command injection. A Random Forest, trained on 268,240 of the 383,202 rows of the corpus, decides blocking, and an Isolation Forest, trained only on benign traffic, logs anomalies without blocking.

The extractor takes 0.0273 ms (p95; 5 ms limit). The Random Forest reaches a macro F1 of 0.9776, and the Isolation Forest a recall of 0.9125 with a false positive rate of 0.0546. Coverage exceeds 80 % per category in SQL injection, path traversal, and command injection. The 15 ms p95 latency criterion is met under normal and attack load but not under volume load, and the memory and CPU criteria are not met.

The general objective was partially met. With a browser User-Agent, the validated model flagged for blocking 97.1 % of benign requests in Juice Shop (*monitor* mode), and an undocumented model blocked 92.5 % in dvws-node; the benign corpus is the main weakness. The metrics measure separation within the corpus rather than generalization, because the partitions share near-duplicates and the test set was read three times. It is recommended to repeat the measurements with version 0.1.1, expand the benign corpus, and measure per-component cost.

CAPÍTULO 1

Introducción

Las aplicaciones web modernas exponen interfaces de programación cuya superficie de ataque incluye la inyección SQL, el *Cross-Site Scripting*, el *Path Traversal* y la inyección de comandos del sistema operativo. Estas categorías figuran entre los riesgos más críticos del estándar OWASP Top 10 [1] y comparten una propiedad: la evidencia del ataque reside en una sola petición HTTP. Las startups y empresas tecnológicas que desarrollan servicios sobre Node.js rara vez cuentan con un equipo de seguridad dedicado. Este trabajo parte de la premisa de que, para esas organizaciones, las soluciones perimetrales resultan costosas de operar y mantener. De ahí surge la necesidad de una herramienta de protección que opere dentro de la propia aplicación.

Este trabajo presenta logSguarDian, una librería npm que implementa un middleware de autoprotección de aplicaciones en tiempo de ejecución (RASP) para Express. El middleware intercepta cada petición, extrae un vector de 76 características numéricas y lo evalúa con un modelo híbrido. Un Random Forest clasifica la petición en cinco clases (tráfico benigno y las cuatro categorías de ataque) y decide el bloqueo, y un Isolation Forest asigna un puntaje de anomalía que se registra y no bloquea. Ambos modelos se serializan en formato ONNX y se ejecutan en `worker_threads`, para no bloquear el *Event Loop* del proceso anfitrión. El proyecto se desarrolló junto con Sebastián Huertas, quien colaboró en el componente de ciberseguridad y en la implementación del lado npm de la librería, y este documento reporta el componente de aprendizaje automático.

El objetivo general es diseñar e implementar esa librería para que detecte y clasifique las amenazas cubiertas, y marque anomalías estadísticas, sin comprometer el rendimiento del servidor anfitrión. Tres objetivos específicos lo desarrollan. El primero fija un límite de 5 ms por petición para el pipeline de extracción de características. El segundo establece umbrales de F1, de *recall* y de tasa de falsos positivos para el modelo híbrido sobre un conjunto de prueba bloqueado. El tercero exige una cobertura mínima de 80 % por categoría de ataque y un incremento del p95 de latencia de hasta 15 ms en un entorno de prueba controlado. Este trabajo no formula una hipótesis estadística; los criterios de aceptación de los tres objetivos específicos, definidos en el capítulo de Objetivos, cumplen esa función.

La investigación es aplicada y experimental. El corpus reúne 383 202 filas de 17 fuentes, de las cuales 268 240 se destinan al entrenamiento: nueve conjuntos o listas públicas, siete generadores

propios con tráfico sintético, creados para cubrir vacíos de cobertura, y una fuente de telemetría curada. La evaluación comprende métricas del modelo sobre un conjunto de prueba bloqueado, cuatro configuraciones de integración sobre una aplicación vulnerable (sin protección, solo logSguarDian, solo un cortafuegos de aplicaciones web y ambos en capas) y pruebas en cuatro aplicaciones de terceros. Las cifras principales se acompañan de su procedencia y de las limitaciones que las afectan.

Los resultados cumplen, con los matices que detalla el capítulo de Discusión, los criterios del extractor, del modelo híbrido y de la cobertura por categoría, aunque con el par A (tarball e09120…) el intervalo de confianza de la cobertura de `path_traversal` incluye valores inferiores al 80 %; cumplen también el de latencia en carga normal y de ataque. No cumplen los criterios de latencia en carga de volumen, de memoria adicional ni de CPU. El trabajo documenta además las limitaciones que acotan la lectura de las métricas: la lectura repetida del conjunto de prueba, la estructura de casi-duplicados del corpus y el peso del corpus benigno en los falsos positivos observados en aplicaciones de terceros.

El documento se organiza como sigue. Los capítulos de Antecedentes, Justificación, Objetivos y Alcance plantean el problema y delimitan el trabajo, y el Marco Teórico desarrolla los conceptos necesarios. La Metodología describe las fases, las técnicas y los criterios de evaluación. El capítulo de Resultados presenta las mediciones, y el de Discusión las interpreta. Los capítulos de Conclusiones y Recomendaciones cierran el trabajo, y los anexos reúnen los análisis complementarios y la procedencia de los artefactos.

CAPÍTULO II

Antecedentes

II.1. Introducción al estado del arte

La detección de amenazas mediante aprendizaje automático cuenta con más de una década de desarrollo académico consolidado, particularmente en el dominio de la detección de intrusiones a nivel de red. Sin embargo, la aplicación de estas técnicas a la capa de aplicación de servicios web modernos, particularmente a arquitecturas asíncronas orientadas a eventos como Node.js, constituye un área considerablemente menos explorada en la literatura indexada. Este capítulo revisa crítica y comparativamente los antecedentes académicos y técnicos directamente relacionados con el problema abordado por logSguarDian. Se organizan en cinco ejes: (1) los estudios de referencia sobre detección de intrusiones basada en aprendizaje automático, (2) los algoritmos base del modelo híbrido propuesto, (3) las metodologías de construcción de datasets etiquetados, (4) las restricciones de inferencia embebida documentadas en la literatura de Edge AI, y (5) las soluciones comerciales y perimetrales existentes junto con su respaldo documental. El objetivo de esta comparación directa es identificar los vacíos metodológicos que este proyecto atiende, no producir un listado cronológico de estos trabajos.

II.2. Detección de intrusiones basada en aprendizaje automático: panorama general

Cuatro revisiones de la literatura de amplio alcance delimitan el estado del arte general del uso de aprendizaje automático en ciberseguridad. Buczak y Guven [2] presentan una taxonomía comparativa de métodos de minería de datos y aprendizaje automático aplicados a la detección de intrusiones. Evalúan su desempeño reportado en la literatura previa a 2016. Su hallazgo más relevante para este proyecto es doble. Por un lado, discuten la complejidad computacional de los métodos y ofrecen criterios de comparación y recomendaciones de uso.

Por otro, evidencian que la inmensa mayoría de los estudios revisados opera sobre datasets de

flujo de red heredados (KDD Cup 99, NSL-KDD). Estos datasets se centran en características de nivel de transporte (duración de conexión, bytes transferidos, banderas del protocolo TCP, Transmission Control Protocol) y no en la semántica del contenido HTTP de la capa de aplicación.

Sarker et al. [3] amplían este panorama y proponen una taxonomía de çiencia de datos de ciberseguridad"que organiza los enfoques de ML en función del tipo de amenaza abordada. Su revisión confirma la misma limitación estructural señalada por Buczak y Guven: los conjuntos de datos de referencia utilizados en la literatura mayoritaria siguen centrados en tráfico de red genérico, con escasa atención a la semántica de peticiones HTTP individuales o al comportamiento específico de APIs REST. Verma y Ranga [4] conducen una revisión enfocada en sistemas de detección de intrusiones para entornos de Internet de las Cosas (IoT), que compara el desempeño de múltiples algoritmos, incluido Random Forest, sobre datasets como NSL-KDD, CICIDS2017 y BoT-IoT. Su aporte principal es una tabla comparativa de exactitud por algoritmo y dataset, en escenarios de recursos computacionales restringidos. Ese contexto es pertinente para la inferencia embebida de logSguarDian, aunque el enfoque permanece anclado en dispositivos IoT y no en servidores de aplicaciones Node.js.

Berman et al. [5] completan este panorama con una revisión introductoria de métodos de aprendizaje profundo aplicados a ciberseguridad, con arquitecturas como redes neuronales convolucionales y recurrentes en tareas de detección de malware, tráfico de red y correo no deseado. Esa revisión no sustenta por sí sola la elección de modelos ligeros. La comparación propia de este trabajo, sobre el mismo corpus y contrato de características, es la base de esa decisión. Random Forest (macro F1 de 0.9776) se equipara en la práctica a un perceptrón multicapa (macro F1 de 0.9719) con una latencia de inferencia menor. Un autoencoder se equipara al recall y a la tasa de falsos positivos de Isolation Forest, con un tamaño de modelo y una latencia órdenes de magnitud menores (Marco Teórico, sección de redes neuronales ligeras y autoencoders).

La comparación de los trabajos de Buczak y Guven [2], Sarker et al. [3] y Verma y Ranga [4] revela un patrón común: los métodos de ensamble basados en árboles figuran entre los algoritmos revisados, pero lo hacen casi exclusivamente sobre datasets de flujo de red heredados que no capturan la semántica de una petición HTTP individual. Este vacío metodológico, la ausencia de literatura consolidada sobre ingeniería de características a nivel de request HTTP para APIs REST modernas, es precisamente el espacio que ocupa el pipeline de extracción de características de logSguarDian, descrito en la sección VII.3 de este trabajo.

Dos revisiones más recientes confirman que esta limitación estructural persiste en la literatura publicada durante el desarrollo de este proyecto. Zhen et al. [6] revisan sistemáticamente el estado de la detección de anomalías dentro del dominio de la conciencia situacional de seguridad de redes. Proponen un modelo conceptual (AFEMAD) que integraría fusión de datos multimodal, detección adaptativa en línea y aprendizaje federado. Se trata, sin embargo, de una propuesta para investigación futura y no de un sistema implementado y evaluado empíricamente. El nivel de análisis de la revisión permanece anclado en el flujo de red.

No reporta restricciones de latencia ni de huella de memoria compatibles con un entorno de inferencia embebida como el de un middleware Node.js.

Rahman et al. [7] revisan de forma sistemática más de 120 estudios publicados entre 2018 y 2024 sobre cortafuegos de aplicación web potenciados por inteligencia artificial orientados a infraestructura crítica. Documentan avances reportados en esa literatura –tasas de detección de hasta

94–99 % con falsos positivos por debajo de 0.7 % frente a escenarios de día cero– además de una arquitectura multicapa propuesta para atender restricciones de latencia ultra baja. Los sistemas que cubre esta revisión son, no obstante, cortafuegos de arquitectura perimetral por definición. La revisión no reporta, entre los más de 120 estudios cubiertos, un artefacto de código abierto reproducible. Tampoco reporta una arquitectura embebida directamente en el proceso de la aplicación protegida.

Ambas revisiones, publicadas en 2025 y 2026, confirman que el vacío señalado por Buczak y Guven [2], Sarker et al. [3] y Verma y Ranga [4] seguía sin resolverse al momento de realizar esta revisión de literatura. Ese vacío es la ausencia de literatura sobre detección híbrida embebida en el proceso de una aplicación web, evaluada bajo restricciones de recursos comparables a las de un middleware Node.js.

II.3. Algoritmos base del modelo híbrido: Random Forest e Isolation Forest

El componente supervisado del modelo propuesto se fundamenta directamente en el trabajo seminal de Breiman [8], quien introduce Random Forest como un método de ensamble que combina bagging con selección aleatoria de características en cada nodo de división. Esto le confiere robustez frente al sobreajuste sin sacrificar velocidad de inferencia. El componente no supervisado se fundamenta en Liu et al. [9], quienes proponen Isolation Forest bajo el principio de que las observaciones anómalas requieren, en promedio, menos particiones aleatorias para ser aisladas que las observaciones normales. Ambos algoritmos constituyen antecedentes directos de la arquitectura del proyecto, no meramente referenciales: sus propiedades de complejidad computacional lineal y ausencia de requisitos de normalización estricta son la razón técnica por la que fueron seleccionados frente a alternativas de mayor capacidad computacional.

Chandola et al. [10] contextualizan estos dos algoritmos dentro de una taxonomía más amplia de técnicas de detección de anomalías. Establecen la distinción formal entre detección supervisada (que requiere ejemplos etiquetados de cada clase de anomalía) y detección no supervisada de una sola clase (que modela únicamente la normalidad). Su marco conceptual es el antecedente teórico directo de la decisión arquitectónica de combinar ambos paradigmas en un modelo híbrido, en lugar de optar por uno solo. Mientras el primero garantiza precisión sobre las categorías de ataque conocidas, el segundo provee una señal complementaria de anomalía. Esa señal se registra y no se usa para bloquear, frente a variantes no vistas durante el entrenamiento.

II.4. Metodologías de construcción y etiquetado de datasets

El corpus de entrenamiento de logSguarDian se compone predominantemente de datasets públicos curados. Entre ellos figuran SR-BH 2020/CAPEC, modsec-learn, registros de honeypot de ModSecurity, SecLists, PayloadsAllTheThings y varios conjuntos de Kaggle. Cada uno conserva su etiqueta original y tiene su licencia documentada en la auditoría de datasets del proyecto. Solo de forma minoritaria, el corpus incorpora un suplemento de tráfico sintético generado deliberadamente. Ese suplemento cubre vacíos puntuales de cobertura detectados mediante análisis de exclusión por fuente (LOSO) y pruebas de ablación sobre el corpus público.

Es esta porción sintética, y no el corpus completo, la que tiene como antecedente metodológico directo el trabajo de Shiravi et al. [11]. Ese trabajo propone un enfoque sistemático para generar datasets de referencia. El enfoque consiste en desplegar un entorno de prueba controlado, simular tráfico legítimo representativo y ejecutar herramientas de ataque automatizadas. Su etiquetado es determinista y se basa en el conocimiento del proceso de generación ("ground truth by design").

Este enfoque dio origen a los datasets de la familia CICIDS, ampliamente citados en la literatura de detección de intrusiones. Constituye la base metodológica adoptada en este proyecto para el suplemento sintético. Ese suplemento se genera para atender vacíos puntuales de cobertura y no como fuente primaria de datos de entrenamiento.

La comparación crítica con este antecedente revela, no obstante, la misma limitación estructural identificada en la sección anterior: los datasets resultantes de esta metodología, incluido el propio CICIDS, se construyen predominantemente sobre características de flujo de red (duración, bytes por segundo, número de paquetes) y no sobre la semántica del contenido de la petición HTTP. La adopción del mismo principio metodológico de generación controlada, pero aplicado, para el suplemento sintético del corpus, a la extracción de características semánticas del objeto `req` de Express en lugar de metadatos de flujo de red, representa una adaptación metodológica específica de este proyecto frente a su antecedente directo.

II.5. Restricciones de inferencia embebida y Edge AI

La ejecución de modelos de aprendizaje automático dentro de un proceso con restricciones estrictas de latencia y memoria, sin comunicación con servicios externos, está documentada de forma consolidada en la literatura de Edge AI, cuyo texto de referencia es el de Warden y Situnayake [12]. Los autores establecen los principios de diseño para desplegar modelos ligeros bajo presupuestos de memoria y tiempo de inferencia del orden de milisegundos. Esos principios incluyen técnicas de cuantización y formatos de serialización optimizados, como los adoptados en este proyecto (ONNX). Sin embargo, este cuerpo de literatura se desarrolla casi en su totalidad sobre microcontroladores y dispositivos embebidos de hardware dedicado, no sobre entornos de ejecución de un solo hilo orientados a eventos como el de Node.js. No se identificó en la revisión de literatura un antecedente académico que documente la aplicación de esta disciplina de inferencia embebida a las restricciones específicas del Event Loop de Node.js mediante `worker_threads`, lo cual constituye una adaptación metodológica original de este proyecto sobre un cuerpo de conocimiento previamente establecido en un dominio de hardware distinto.

II.6. Soluciones perimetrales existentes y su respaldo documental

Como complemento a la revisión académica, la literatura técnica y los informes de la industria documentan de forma consistente las limitaciones estructurales de las soluciones perimetrales tradicionales frente a amenazas de capa de aplicación. La taxonomía de OWASP Top 10 [1] establece las categorías de vulnerabilidad más críticas en aplicaciones web modernas, entre ellas la inyección y los fallos de control de acceso, y sirve como marco de referencia frente al cual se evalúa la cobertura de logSguarDian. El reporte anual de Verizon [13] y el informe de Snyk [14] describen la presencia

de incidentes y de vulnerabilidades en el software de la industria. A diferencia de los antecedentes académicos revisados en las secciones anteriores, estas fuentes constituyen documentación técnica de la industria y no literatura indexada por pares; se incorporan en este capítulo únicamente como evidencia contextual de la magnitud del problema, no como fundamento metodológico del diseño del sistema, distinción que se mantiene consistente con la naturaleza aplicada de este trabajo.

II.7. Síntesis comparativa

El Cuadro 1 resume los antecedentes revisados en función de cuatro dimensiones críticas para el problema de investigación: el nivel de la pila de red en el que opera cada antecedente, la naturaleza del dataset o evidencia empleada, la arquitectura de despliegue del enfoque, y la limitación principal identificada frente a la propuesta de logSguarDian.

Antecedente	**Nivel**	**Arquitectura**	**Limitación frente a logSguarDian**
Buczak y Guven [2]; Sarker et al. [3]; Verma y Ranga [4]	Red (flujo)	Offline / batch	Datasets de flujo de red (KDD/NSL-KDD/CICIDS); sin semántica de petición HTTP
Berman et al. [5]	Red / malware	Offline / batch	Revisión introductoria de aprendizaje profundo; sin medición de costo de inferencia embebida (medido en este trabajo, Marco Teórico)
Zhen et al. [6]	Red (flujo)	Revisión / propuesta conceptual	Modelo híbrido (AFEMAD) propuesto como marco conceptual, sin implementación evaluada ni restricciones de latencia embebida documentadas
Rahman et al. [7]	Aplicación	Revisión sistemática (WAF perimetral)	Revisión de más de 120 estudios de WAF de arquitectura perimetral; sin artefacto reproducible ni arquitectura embebida no perimetral entre los estudios cubiertos
Shiravi et al. [11]	Red (flujo)	Entorno de prueba controlado	Metodología de generación adoptada, pero features de flujo, no de contenido HTTP
Warden y Situnayake [12]	Hardware embebido	Microcontroladores	Disciplina de Edge AI no documentada para el Event Loop de Node.js

Cuadro 1: Síntesis comparativa de los antecedentes revisados frente a la propuesta de logSguarDian. Elaboración propia a partir de la revisión de literatura de este capítulo.

II.8. Conclusión: vacíos identificados y aporte del proyecto

La revisión crítica de los antecedentes académicos y técnicos converge en tres vacíos concretos que este proyecto atiende de forma directa. En primer lugar, la literatura de referencia en detección de intrusiones mediante aprendizaje automático, ejemplificada en las revisiones de Buczak y Guven [2], Sarker et al. [3] y Verma y Ranga [4], privilegia consistentemente los datasets de flujo de red

heredados sobre la semántica de peticiones HTTP individuales. Con ello deja sin resolver la pregunta de cómo construir un pipeline de ingeniería de características orientado específicamente al objeto de petición de un framework como Express.

En segundo lugar, ni la revisión de arquitecturas híbridas supervisadas/no supervisadas de Zhen et al. [6] ni la revisión sistemática de cortafuegos de aplicación web potenciados por inteligencia artificial de Rahman et al. [7] identifican, entre la literatura que cubren, un sistema implementado bajo una arquitectura embebida dentro del proceso de una aplicación con recursos limitados. Ambas permanecen, respectivamente, en el nivel de propuesta conceptual y en el de arquitectura perimetral centralizada. No abordan las restricciones específicas de un despliegue embebido dentro del proceso de una aplicación Node.js. Ese despliegue es precisamente el escenario documentado para las startups tecnológicas guatemaltecas en el planteamiento del problema de este trabajo.

En tercer lugar, la disciplina de inferencia embebida bajo restricciones de latencia y memoria, consolidada en la literatura de Edge AI para hardware de microcontroladores, no cuenta con un antecedente documentado de aplicación al modelo de concurrencia de un solo hilo de Node.js mediante `worker_threads`.

Estos tres vacíos no son independientes entre sí. Los tres (la ausencia de ingeniería de características semántica a nivel de HTTP en la literatura de ML-IDS, la ausencia de arquitecturas híbridas embebidas, no perimetrales, documentadas para aplicaciones web, y la ausencia de una adaptación de la disciplina de Edge AI al entorno de ejecución de Node.js) se articulan en un mismo vacío estructural. Ese vacío es la falta de una solución de detección de amenazas que combine aprendizaje automático híbrido con inferencia embebida en tiempo real dentro de un servidor Node.js, sin infraestructura perimetral ni dependencias externas. logSguarDian se posiciona como una respuesta directa a este vacío convergente.

De sus antecedentes adopta los elementos: la adopción de ensambles ligeros, cuyo costo de inferencia se contrastó con una medición propia (RF 0.9776 frente a MLP 0.9719) sobre alternativas de mayor capacidad; la complementariedad teórica entre detección supervisada y no supervisada; y la metodología de generación controlada de datasets con etiquetado determinista. Los extiende hacia un dominio de aplicación, la protección embebida de APIs Node.js, en el que dicha combinación no había sido previamente documentada.

CAPÍTULO III

Justificación

La motivación central de este trabajo de graduación surge de una brecha estructural verificable. Las startups y empresas tecnológicas que desarrollan servicios sobre Node.js rara vez cuentan con un equipo de seguridad dedicado. Por su parte, la superficie de ataque de sus APIs REST –inyección SQL, Cross-Site Scripting, Path Traversal/LFI e inyección de comandos del sistema operativo– corresponde a categorías de riesgo documentadas de forma consistente en el estándar OWASP Top 10 como las más críticas para aplicaciones web modernas [1]. En un ecosistema donde las aplicaciones web dependen en gran medida de componentes de código abierto que pueden contener vulnerabilidades [14].

El informe de Verizon de 2024 documenta que la explotación de vulnerabilidades como vía inicial de acceso casi se triplicó respecto del año anterior [13]. La ausencia de mecanismos de detección accesibles a nivel de capa de aplicación es una condición estructural del ecosistema, y no un vacío incidental. Este proyecto busca atender directamente esa condición.

III.1. Importancia del problema

El problema central que motiva esta investigación es la ausencia de mecanismos automatizados, técnicamente fundamentados y económicamente viables para detectar ataques ampliamente documentados y estudiados en organizaciones sin capacidad de inversión en seguridad dedicada. Las soluciones existentes revisadas en el capítulo de Antecedentes –cortafuegos de aplicación web (WAF), sistemas de detección de intrusiones (IDS) y plataformas de gestión de eventos de seguridad (SIEM)– comparten una limitación común: exigen mantenimiento especializado continuo, infraestructura de soporte o un costo de licenciamiento que las sitúa fuera del alcance de organizaciones pequeñas. Esta condición no es exclusiva de un contexto geográfico particular. No obstante, se agudiza en economías donde el crecimiento del ecosistema tecnológico no ha sido acompañado de una madurez institucional equivalente en ciberseguridad, como es el caso de Guatemala.

Según un informe del proveedor Mastercard, los ciberataques aumentaron un 200 % en 2024

respecto de 2023, y el país se ubica entre los diez con mayor exposición cibernética de Latinoamérica [15]. Abordar este problema es importante porque la ausencia de protección a nivel de aplicación expone directamente datos de clientes, información financiera y la continuidad operativa de organizaciones que, por su tamaño, no pueden absorber el costo de un incidente de seguridad significativo.

III.2. Relevancia técnica y científica

Desde la perspectiva técnica, este trabajo es relevante porque atiende un vacío identificado de forma explícita en el capítulo de Antecedentes. La literatura de detección de intrusiones mediante aprendizaje automático se ha desarrollado predominantemente sobre datos de flujo de red, no sobre la semántica de peticiones HTTP individuales. Ni la revisión de arquitecturas híbridas de aprendizaje supervisado y no supervisado de Zhen et al. [6] ni la revisión sistemática de cortafuegos de aplicación web potenciados por inteligencia artificial de Rahman et al. [7] identifican, entre la literatura que cubren, un sistema implementado bajo una arquitectura embebida dentro del proceso de la aplicación protegida. Investigar la viabilidad de un modelo híbrido de aprendizaje automático ejecutado en tiempo de inferencia dentro de un servidor Node.js constituye un aporte técnico concreto.

Ese modelo está sujeto a restricciones estrictas de latencia impuestas por su modelo de concurrencia de un solo hilo. Extiende una disciplina de diseño –la inferencia embebida bajo restricciones de recursos, consolidada en la literatura de Edge AI para hardware de microcontroladores– hacia un dominio de ejecución distinto. Para ese dominio no se identificó antecedente documentado. Este componente de novedad metodológica es lo que justifica, desde el punto de vista científico, el carácter aplicado-experimental de la investigación: su meta es generar evidencia empírica sobre si dicha combinación arquitectónica es técnicamente viable, en lugar de implementar una solución cuya viabilidad ya esté validada de antemano.

III.3. Relevancia social, económica e institucional

En el plano social y económico, la relevancia del proyecto se sostiene en la magnitud documentada del problema en el contexto local. El ecosistema de startups guatemaltecas opera en un entorno donde la ausencia de un marco legal específico de ciberseguridad y la escasez de profesionales especializados en la región limitan estructuralmente la capacidad de estas organizaciones para invertir en controles de seguridad tradicionales. Una herramienta distribuida sin costo a través de un registro de paquetes de uso extendido, que no requiere modificaciones a la infraestructura existente ni conocimiento especializado para su integración, reduce directamente dos barreras de adopción: el presupuesto disponible y la complejidad técnica de implementación. En el plano institucional, el proyecto es relevante para la Universidad del Valle de Guatemala en la medida en que constituye una contribución documentada y reproducible en la intersección entre ciencia de datos, ciberseguridad e ingeniería de software, un área de creciente relevancia curricular y de investigación aplicada dentro de la Facultad de Ingeniería.

III.4. Beneficiarios de la investigación

Los beneficiarios directos de este trabajo son, en primer lugar, las startups y empresas tecnológicas que operan servicios sobre Node.js sin un equipo de seguridad dedicado, para quienes la investigación busca proveer una herramienta de protección accesible a nivel de capa de aplicación. En segundo lugar, son beneficiarios directos los propios autores del proyecto integrador. Para ellos, el trabajo representa la aplicación rigurosa de las competencias desarrolladas en sus respectivas áreas de formación –inteligencia artificial y aprendizaje automático, en el caso del componente documentado en este trabajo, y ciberseguridad, en el caso del componente desarrollado en paralelo– sobre un problema real y técnicamente exigente.

Como beneficiarios indirectos se identifican, por un lado, la comunidad académica y estudiantil de la Universidad del Valle de Guatemala. Esta comunidad podrá disponer de la metodología documentada de construcción de un pipeline de detección de amenazas sobre arquitecturas orientadas a eventos como referencia para cursos o trabajos de graduación posteriores. Por otro lado, de manera más amplia, se identifican los usuarios finales de las aplicaciones que eventualmente adopten la herramienta. Su información personal y financiera se beneficia de manera transitiva de cualquier mejora en la postura de seguridad de los servicios que utilizan.

III.5. Síntesis

La justificación de este trabajo de graduación se apoya en la combinación específica de tres factores que, tomados en conjunto, delimitan un problema real y no resuelto. No se apoya en la novedad de los ataques que busca detectar, ya ampliamente catalogados en estándares como OWASP Top 10 [1]. Primero, una brecha de adopción de controles de seguridad que este trabajo asume como premisa para organizaciones pequeñas, en un contexto de vulnerabilidades descrito por la industria [13, 14]. Segundo, un vacío metodológico identificado en la literatura académica respecto a la detección de amenazas embebida en tiempo de ejecución dentro de aplicaciones Node.js. Tercero, un contexto local, el ecosistema de startups guatemaltecas, donde ambos factores convergen con particular intensidad [15].

Este capítulo no pretende anticipar en qué medida la solución propuesta logrará resolver dicho problema; ese es precisamente el objeto de la investigación desarrollada en los capítulos siguientes. Lo que este capítulo establece es que el problema es real, que su abordaje es pertinente tanto desde una perspectiva técnica como social e institucional, y que existen beneficiarios concretos, directos e indirectos, para quienes una respuesta fundamentada a este problema representa un aporte de valor verificable.

CAPÍTULO IV

Objetivos

IV.1. Objetivo general

Diseñar e implementar una librería npm que integre un pipeline de ingeniería de características y un modelo híbrido de aprendizaje automático, capaz de detectar y clasificar amenazas conocidas y anomalías estadísticas correspondientes a variantes desconocidas de los vectores cubiertos en endpoints Node.js en tiempo de ejecución, con el propósito de proveer a startups tecnológicas sin equipo de seguridad dedicado una herramienta accesible de protección a nivel de capa de aplicación que no comprometa el rendimiento del servidor anfitrión.

IV.2. Objetivos específicos

- Construir, durante la fase de desarrollo de la librería (20 de junio – 1 de agosto de 2026), un pipeline de ingeniería de características que extraiga y vectorice atributos cuantificables del tráfico HTTP de forma asíncrona mediante *worker_threads*, verificando que su ejecución no introduzca una latencia adicional superior a 5 ms por petición en el servidor anfitrión.
- Evaluar, al cierre del ciclo de entrenamiento que produce la versión del modelo destinada a la fase de pruebas del sistema (2 de agosto de 2026), un modelo híbrido compuesto por Random Forest e Isolation Forest sobre un conjunto de prueba bloqueado, demostrando que el componente supervisado alcanza un F1-score mínimo de 0.80 en al menos tres de las cuatro categorías de ataque cubiertas, y que el componente no supervisado alcanza una tasa de detección de anomalías mínima del 50 % con una tasa de falsos positivos máxima del 10 %. El refinamiento iterativo del modelo continuó durante la fase de pruebas del sistema, dentro de la ventana del tercer objetivo específico, incorporando ciclos de reentrenamiento adicionales – como el que atendió vacíos de cobertura en la categoría de inyección de comandos detectados durante esa misma fase– sin alterar los criterios de aceptación aquí establecidos.
- Evaluar, durante la fase de pruebas del sistema (2 de agosto – 30 de septiembre de 2026), el

desempeño integral de la librería integrada como middleware, verificando que la detección de payloads representativos alcance al menos el 80 % por cada una de las cuatro categorías de ataque cubiertas en un entorno de prueba controlado, sin elevar la latencia p95 del servidor más de 15 ms respecto a la línea base sin protección.[1]

[1] Se reporta el percentil 95, no la media, dado que es más representativo de la experiencia real de un usuario bajo carga y menos sensible a valores atípicos que un promedio aritmético.

CAPÍTULO V

Alcance

Este capítulo delimita con precisión qué comprende el presente trabajo de graduación. Establece sus límites funcionales, técnicos y operativos, así como las condiciones, restricciones y supuestos bajo los cuales se desarrolla. La delimitación aquí definida es consistente con los objetivos planteados y sirve como referencia directa para evaluar, al cierre del proyecto, si lo entregado corresponde a lo comprometido.

V.1. Alcance funcional

logSguarDian es una librería npm que implementa un middleware de tipo *Runtime Application Self-Protection* (RASP) para aplicaciones Node.js construidas sobre el framework Express. Su alcance funcional comprende cuatro elementos concretos. Primero, la intercepción de cada petición HTTP entrante antes de que alcance la lógica de negocio de la aplicación anfitriona. Segundo, la extracción y vectorización de un conjunto de 76 características numéricas del objeto `req` de Express, organizadas en seis grupos: semántica de ataque sobre las cuatro categorías cubiertas (37), estructura HTTP (9), URL y ruta (6), parámetros y cuerpo (9), evasión por encoding (10) y variables temporales (5).

Tercero, la inferencia sobre un modelo híbrido compuesto por un Random Forest (componente supervisado) y un Isolation Forest (componente no supervisado), ambos serializados en formato ONNX y ejecutados de forma asíncrona mediante `worker_threads` sin bloquear el *Event Loop* del proceso anfitrión. Cuarto, la clasificación de cada petición como tráfico legítimo o como una de las cuatro categorías de ataque cubiertas, con la generación correspondiente de una respuesta de bloqueo configurable o de una alerta registrada.

El alcance funcional incluye además una interfaz de línea de comandos con cuatro grupos de subcomandos (`config`, `endpoints`, `attacks`, `webhooks`) que permiten al desarrollador anfitrión obtener visibilidad sobre el tráfico clasificado, perfilar rutas con mayor densidad de ataques e integrar la librería con sistemas externos de notificación mediante webhooks, sin requerir modificacio-

nes a la arquitectura de la aplicación protegida.

V.2. Alcance técnico

El componente de Machine Learning documentado en este trabajo comprende los siguientes elementos. El primero es el análisis, la justificación y el contrato de entrada del pipeline de ingeniería de características que transforma el objeto `req` de Express en un vector numérico de dimensión fija. El segundo es la construcción de un corpus de entrenamiento compuesto predominantemente por datasets públicos curados (entre ellos SR-BH 2020/CAPEC, modsec-learn, registros de honeypot de ModSecurity, SecLists, PayloadsAllTheThings y conjuntos de Kaggle, cuyas licencias se verificaron en la auditoría de datasets, salvo dos que no declaran licencia). Ese corpus se complementa con un conjunto acotado de tráfico sintético generado en un entorno de prueba controlado (Docker Compose). Ese tráfico cubre vacíos puntuales de cobertura detectados durante la validación del modelo.

Para esa porción sintética, el etiquetado es determinista y se basa en el conocimiento del proceso de generación. El tercero es el entrenamiento, la calibración de umbrales y la evaluación del modelo híbrido Random Forest e Isolation Forest sobre dicho corpus. El cuarto es la serialización del modelo en formato ONNX mediante `sklearn2onnx`, con verificación de paridad de predicción respecto al modelo original en Python. El quinto es la integración del modelo serializado en la librería npm mediante el runtime `onnxruntime-node`, con inferencia ejecutada en un *worker pool* dedicado. La implementación inicial del extractor estuvo a cargo de Sebastián Huertas (véase la sección VII.7).

La librería está delimitada técnicamente a aplicaciones que utilicen Node.js en versión igual o superior a 20, la versión mínima declarada en el campo `engines` de `packages/core/package.json`, y Express 4.18 o posterior, o Express 5 (`peerDependencies` de `packages/core`: `^4.18.0 || ^5.0.0`), como capa de enrutamiento HTTP. El entrenamiento de los modelos se realiza íntegramente en Python sobre hardware Apple Silicon local, sin dependencia de infraestructura de cómputo en la nube.

V.3. Alcance operativo y de evaluación

La evaluación del sistema se realiza en un entorno de prueba controlado y reproducible, compuesto por una aplicación Express deliberadamente vulnerable desplegada mediante Docker Compose. Sobre este entorno se definieron tres configuraciones de evaluación progresivas, la tercera con dos variantes (3a, solo cortafuegos; 3b, en capas). Una configuración base no tiene ningún mecanismo de protección y confirma la explotabilidad de la aplicación de referencia. Otra tiene logSguarDian activo como única capa de protección. Una tercera configuración tiene dos variantes.

La 3a emplea únicamente un Web Application Firewall perimetral (Nginx con ModSecurity y el Core Rule Set de OWASP). La 3b suma logSguarDian, que opera como segunda capa de protección sobre ese mismo cortafuegos. Esta tercera configuración delimita explícitamente el alcance operativo del proyecto a un escenario de defensa en profundidad. Evalúa la capacidad de detección de logSguarDian de forma independiente sobre el tráfico que logra evadir o queda fuera de la cobertura de un WAF basado en firmas. No plantea la evaluación como una comparación de qué componente

es individualmente superior.

El alcance de esta evaluación se limita al entorno de prueba controlado descrito; no se contempla dentro del alcance del proyecto un despliegue en un entorno de producción real de terceros como criterio de aceptación del sistema. Cualquier validación adicional con organizaciones externas se documenta como evidencia contextual complementaria y no como parte de los criterios de aceptación verificables del proyecto.

V.4. Elementos fuera del alcance

El Cuadro 2 enumera las exclusiones explícitas del proyecto, cada una acompañada de su justificación técnica, con el fin de delimitar con precisión la frontera entre lo que logSguarDian cubre y lo que queda deliberadamente fuera de su diseño.

V.5. Restricciones, condiciones y supuestos

El desarrollo del proyecto está sujeto a las siguientes restricciones y supuestos, que delimitan el margen de diseño y ejecución disponible. En cuanto a restricciones de tiempo, el proyecto se ejecuta dentro del período comprendido entre mayo y octubre de 2026. Ese período corresponde al calendario del trabajo de graduación de la Facultad de Ingeniería de la Universidad del Valle de Guatemala. Cualquier línea de trabajo no completable dentro de dicho período se documenta explícitamente como trabajo futuro y no como un entregable pendiente.

En cuanto a restricciones de recursos, el entrenamiento de los modelos se realiza sobre hardware local sin presupuesto asignado a instancias de cómputo en la nube, condición verificada como viable dado el tamaño de los modelos de ensamble seleccionados (Random Forest e Isolation Forest) frente a alternativas de mayor capacidad computacional. En cuanto a restricciones de dependencias externas, la librería está diseñada para operar sin comunicación con servicios de inferencia remotos ni infraestructura de terceros durante su ejecución en producción, lo cual delimita el diseño hacia modelos de inferencia local suficientemente ligeros para residir en el mismo proceso que la aplicación anfitriona.

Se asume, además, que el conjunto de herramientas de simulación de ataques y generación de tráfico legítimo empleadas para construir el corpus de entrenamiento produce muestras suficientemente representativas de las cuatro categorías de ataque cubiertas; esta condición fue verificada mediante el análisis exploratorio del corpus resultante antes de proceder a la fase de entrenamiento. Se asume también que la arquitectura de middleware de Express constituye un punto de integración representativo del patrón dominante de desarrollo de APIs REST en el ecosistema de startups tecnológicas guatemaltecas descrito en la Justificación de este trabajo.

V.6. Coherencia y viabilidad del alcance

El alcance definido en este capítulo mantiene coherencia directa con los objetivos específicos del proyecto: el alcance funcional y técnico corresponde a los objetivos de construcción del pipeline de características y del modelo híbrido, mientras que el alcance operativo de evaluación en tres configuraciones progresivas corresponde al objetivo de evaluación del desempeño integral del sistema. Las exclusiones documentadas en el Cuadro 2 son decisiones de diseño deliberadas, tomadas con anterioridad al inicio del desarrollo y orientadas a mantener un alcance verificable dentro del tiempo y los recursos disponibles, no limitaciones que el proyecto haya dejado sin resolver. La viabilidad de este alcance está respaldada por la naturaleza acotada de las cuatro categorías de ataque seleccionadas.

Todas son de naturaleza intra-request, evaluables mediante el mismo vector de características sin requerir estado persistente entre peticiones. También está respaldada por la selección de algoritmos de complejidad computacional lineal. La factibilidad de entrenamiento e inferencia de esos algoritmos bajo las restricciones de hardware y tiempo del proyecto fue confirmada antes de comprometer dicho alcance como parte de los objetivos específicos del trabajo.

Ámbito excluido	Razón técnica
Denegación de servicio distribuida (DDoS) y fuerza bruta	Ataques inter-request: su naturaleza maliciosa reside en patrones de frecuencia y volumen acumulados entre múltiples peticiones, incompatibles con un vector de características que opera petición a petición.
Broken Object Level Authorization (BOLA), Mass Assignment, Server-Side Request Forgery (SSRF), deserialización insegura	Requieren conocimiento del modelo de datos específico de la aplicación anfitriona; no generan patrones semánticos detectables sin contexto de la lógica de negocio.
Inyecciones NoSQL (MongoDB u otras)	Sintaxis de payload estructuralmente distinta a las inyecciones SQL cubiertas; requeriría un corpus de entrenamiento y marcadores de características independientes.
Modelos de aprendizaje profundo de alta capacidad (red neuronal convolucional (CNN), red neuronal recurrente (RNN), Transformers)	Costo de inferencia incompatible con el presupuesto de latencia del middleware; tamaño de modelo serializado incompatible con las convenciones de un paquete npm de uso general.
Aprendizaje continuo y reentrenamiento automático embebido en el proceso de la aplicación anfitriona	Requeriría infraestructura de almacenamiento de muestras y validación previa al despliegue dentro del propio proceso del middleware, incompatible con el principio de operación autónoma sin dependencias externas de la librería en tiempo de ejecución.
Seguridad de red (Capas 3 y 4 del modelo OSI, Open Systems Interconnection)	logSguarDian opera exclusivamente en la Capa 7, dentro del proceso Node.js; la mitigación volumétrica de red es responsabilidad de controles perimetrales complementarios.
Funcionalidades administrativas de WAF empresarial (gestión centralizada de reglas, dashboards, integración con red de distribución de contenidos (CDN))	Requieren infraestructura de servicio central, incompatible con el principio de operación sin dependencias externas que fundamenta la propuesta de valor de la librería.
Compatibilidad con runtimes distintos de Node.js/Express	El mecanismo de integración es específico al sistema de middleware de Express; soporte para otros lenguajes o frameworks constituiría un entregable independiente.

Cuadro 2: Elementos explícitamente fuera del alcance del proyecto, con su justificación técnica. Elaboración propia.

CAPÍTULO VI

Marco teórico

VI.1. Introducción

Mientras el capítulo de Antecedentes compara logSguarDian contra trabajos previos para identificar los vacíos metodológicos que este proyecto atiende, el presente capítulo desarrolla los conceptos que un lector necesita para entender por qué esos vacíos existen y por qué la arquitectura propuesta los cierra de la forma en que lo hace. Los fundamentos se organizan en el orden en que se articulan dentro del sistema. Primero se presentan las categorías de ataque que logSguarDian debe reconocer y el entorno de ejecución –Node.js, Express y el modelo de middleware– en el que opera. Después se describe el ecosistema de dependencias de terceros y el contexto local que motivan su necesidad.

Luego se explican los algoritmos de aprendizaje automático evaluados y seleccionados para el modelo híbrido. Finalmente se tratan el proceso de ingeniería de características, las métricas con las que se evalúa el sistema, y las restricciones de serialización y concurrencia bajo las cuales debe operar en producción. Cada eje se cierra con la decisión de diseño concreta a la que se vincula el concepto. Esa decisión se documenta con mayor detalle técnico en los capítulos de Alcance y Objetivos.

VI.2. Panorama de amenazas en la capa de aplicación

El OWASP Top 10 clasifica la inyección como la categoría de riesgo más consistentemente crítica en aplicaciones web desde su primera edición. La define como cualquier mecanismo por el cual datos no confiables suministrados por un atacante alteran la interpretación de un comando o consulta por parte del intérprete que la recibe [1]. En la edición 2021, la inyección SQL, XSS y la inyección de comandos se agrupan en A03 (Inyección), y el path traversal (CWE-22) en A01 (Control de acceso defectuoso). Las cuatro categorías de ataque a nivel de aplicación cubiertas por logSguarDian se dirigen cada una a un componente distinto de la pila de una aplicación Node.js

típica.

La inyección SQL (SQLi) ocurre cuando una entrada de usuario se concatena sin sanitizar dentro de una consulta destinada a un motor de bases de datos relacional. El atacante inserta así cláusulas SQL adicionales –por ejemplo, un operador `OR 1=1` o un comentario `-` que trunca el resto de la consulta original– y altera de ese modo su lógica de autenticación o extracción de datos. El Cross-Site Scripting (XSS) sigue el mismo principio pero dirigido al intérprete del navegador: un payload que contiene marcado HTML o JavaScript –típicamente una etiqueta `<script>` o un manejador de eventos como `onerror`– se refleja o persiste sin escapar en una respuesta, y se ejecuta en el contexto de sesión de otro usuario.

El Path Traversal, cuya forma más severa es la Inclusión Local de Archivos (LFI), explota la resolución de rutas del sistema de archivos: una secuencia de ascenso de directorio (`../`) inyectada en un parámetro que la aplicación concatena directamente a una ruta base permite acceder a archivos fuera del directorio previsto. La inyección de comandos del sistema operativo (CmdI) ocurre cuando una entrada de usuario llega sin sanitizar a una llamada que invoca un shell del sistema operativo. Esto habilita el uso de operadores de encadenamiento (`;`, `&&`, `|`) o de sustitución de comandos (`$(...)`) para ejecutar comandos arbitrarios con los privilegios del proceso de la aplicación.

Las cuatro categorías comparten una propiedad estructural que fundamenta directamente el diseño del pipeline de características de logSguarDian: la evidencia del ataque reside íntegramente dentro de una única petición HTTP –su método, ruta, encabezados, parámetros de consulta y cuerpo– sin requerir estado acumulado de peticiones anteriores. Esta propiedad es la razón técnica, documentada en el capítulo de Alcance, por la cual el vector de características de logSguarDian opera petición a petición y por la cual ataques de naturaleza inter-request como la denegación de servicio o la fuerza bruta quedan fuera del alcance del proyecto.

VI.3. Node.js, Express y el modelo event-driven

Node.js ejecuta el código de la aplicación –incluido el middleware que la protege– sobre un único hilo de Event Loop, que atiende operaciones de entrada/salida de forma no bloqueante mediante *libuv*, pero que no puede ejecutar dos fragmentos de código JavaScript de forma simultánea dentro de ese hilo [16]. Este modelo, conocido como orientado a eventos (*event-driven*), sustituye el patrón clásico de un hilo del sistema operativo por conexión. En su lugar, un bucle único procesa callbacks a medida que se resuelven las operaciones de E/S asociadas. Esto permite atender miles de conexiones concurrentes con un consumo de memoria muy inferior al de un servidor multihilo tradicional. El costo es que cualquier código síncrono de larga duración bloquea la atención de todas las demás conexiones mientras se ejecuta.

Express se construye sobre este modelo como un framework de enrutamiento que organiza el procesamiento de una petición HTTP en una cadena de funciones *middleware*. Cada función tiene acceso al objeto de petición (`req`) y de respuesta (`res`), y puede delegar el control a la siguiente función de la cadena mediante una llamada explícita (`next()`) o interrumpirla mediante el envío de una respuesta. Esta arquitectura de cadena de middleware es la que permite insertar un componente de seguridad como logSguarDian delante de la lógica de negocio de la aplicación sin modificar su código: basta con registrarlo como una función más en la cadena, ejecutada antes que las rutas de la aplicación anfitriona.

La consecuencia directa del modelo de un solo hilo para el diseño de logSguarDian es que cualquier operación de cómputo intensivo ejecutada de forma síncrona dentro de ese middleware bloquearía el Event Loop. Ese bloqueo afectaría a todas las peticiones concurrentes de la aplicación anfitriona, sin importar cuán rápida sea la operación en términos absolutos. Un ejemplo de ello es la inferencia de un modelo de aprendizaje automático. Esta restricción se retoma en la sección de Edge AI de este capítulo, donde se describe el mecanismo con el que logSguarDian la resuelve.

VI.4. Middleware y Runtime Application Self-Protection (RASP)

La posición de logSguarDian dentro de la cadena de middleware de Express no es solo un detalle de integración. Define su categoría arquitectónica frente a otros mecanismos de defensa de la capa de aplicación. Yuan y Malek [17] formalizan, dentro de una taxonomía más amplia de sistemas de software autoprotector, la categoría de autoprotección de aplicaciones en tiempo de ejecución (RASP, por sus siglas en inglés). Esta categoría instrumenta directamente el proceso de la aplicación para detectar y mitigar amenazas con visibilidad de su estado de ejecución interno. Se contrapone a los mecanismos perimetrales, que observan únicamente el tráfico de red que entra y sale del sistema.

logSguarDian se posiciona formalmente dentro de esta categoría. Se ejecuta como middleware dentro del mismo proceso Node.js que la aplicación protegida. Por ello tiene acceso directo al objeto `req` de Express ya parseado por el framework. No necesita reimplementar la normalización de protocolo HTTP que un mecanismo externo debe resolver de forma genérica y, por definición, con menor información de contexto que la propia aplicación.

La arquitectura con la que históricamente se ha contrastado un RASP es el cortafuegos de aplicación web (WAF) de arquitectura perimetral, ejemplificado en este proyecto por ModSecurity, que opera con el OWASP Core Rule Set (CRS). Un WAF de este tipo se despliega como proxy inverso frente al servidor de aplicaciones. Inspecciona cada petición mediante un conjunto de reglas de coincidencia de patrones y funciones de normalización. Entre esas funciones están la decodificación de URL, la remoción de espacios redundantes y la detección de sintaxis SQL mediante análisis léxico (*libinjection*). Solo después la petición se reenvía o se bloquea [18].

Su fortaleza es la cobertura inmediata de firmas de ataque conocidas, sin requerir instrumentación de la aplicación protegida. Su limitación estructural es que opera sin visibilidad del contexto interno de la aplicación. No conoce el modelo de datos, la lógica de negocio ni el resultado real de procesar la petición. Además, su eficacia depende de que el atacante no logre evadir las reglas de normalización y coincidencia mediante técnicas de ofuscación del payload.

El Cuadro 3 contrasta ambas arquitecturas en las dimensiones relevantes para la decisión de diseño de este proyecto y para la interpretación de la Configuración 3 de la evaluación, en la cual ambas arquitecturas operan en capas sobre el mismo tráfico.

VI.5. Ecosistema npm y seguridad en la cadena de suministro

logSguarDian se distribuye como un paquete del registro npm, la misma vía por la que la aplicación anfitriona obtiene la inmensa mayoría de sus dependencias. Zimmermann et al. [19] documen-

Dimensión	WAF perimetral (ModSecurity/CRS)	RASP embebido (logSguarDian)
Ubicación	Proxy inverso, proceso independiente	Middleware dentro del proceso de la aplicación
Visibilidad de contexto	Solo tráfico de red entrante/saliente	Objeto de petición ya parseado por el framework
Mecanismo de detección	Coincidencia de patrones y firmas sobre payload normalizado	Clasificación estadística sobre un vector de características extraído de la petición
Cobertura frente a variantes no vistas	Limitada a lo cubierto por las reglas de firma	El Isolation Forest registra anomalías y no bloquea; en validación marca 19 de los 39 ataques que Random Forest no bloquea (anexo, XIII.1)
Dependencia de infraestructura externa	Requiere proxy y mantenimiento de reglas	Ninguna; opera como dependencia de la aplicación

Cuadro 3: Comparación entre arquitectura WAF perimetral y RASP embebida en las dimensiones relevantes para este proyecto. Elaboración propia a partir de Yuan y Malek [17] y OWASP Foundation [18].

tan, mediante el análisis sistemático del grafo de dependencias de npm, que un paquete promedio confía implícitamente en 79 paquetes de terceros y 39 mantenedores distintos de forma transitiva. La ausencia de un proceso de revisión centralizado antes de la publicación permite que un único paquete comprometido se propague a decenas de miles de proyectos que dependen de él directa o indirectamente. El paquete comprometido puede serlo por una cuenta de mantenedor secuestrada o por código malicioso introducido deliberadamente. Este hallazgo tiene dos implicaciones directas para el diseño de logSguarDian.

La primera es una restricción de diseño: una librería cuyo propósito es proteger una aplicación no puede, sin contradecir ese propósito, introducir ella misma una superficie de ataque de cadena de suministro significativa. Esto motiva mantener el número de dependencias de tiempo de ejecución de logSguarDian deliberadamente acotado –`onnxruntime-node` para la inferencia, `better-sqlite3` para el almacenamiento local de eventos, y las dependencias del propio framework Express– en lugar de adoptar un ecosistema extenso de utilidades de terceros para funciones que pueden resolverse con la biblioteca estándar de Node.js.

La segunda implicación es de alcance. La superficie de riesgo documentada por Zimmermann et al. [19] corresponde a paquetes que ejecutan código arbitrario durante su instalación o su uso. Es un vector estructuralmente distinto de las cuatro categorías de inyección en tráfico HTTP cubiertas por este proyecto. Por eso, la seguridad de la cadena de suministro de npm se documenta aquí como parte del contexto que motiva un diseño de dependencias mínimas. No se documenta como una categoría de ataque que el modelo de clasificación de logSguarDian esté diseñado para detectar.

VI.6. Panorama de ciberseguridad en Guatemala y el ecosistema tech local

La motivación de este proyecto, desarrollada con mayor detalle en el capítulo de Justificación, se apoya en una condición estructural del contexto en el que se ubica: un ecosistema de startups

tecnológicas que opera predominantemente sin equipo de seguridad dedicado, en una región donde la exposición a ciberataques ha crecido de forma pronunciada en los años recientes. Según un informe del proveedor Mastercard [15], los ciberataques registrados en Guatemala aumentaron un 200 % en 2024 respecto de 2023, y el país se ubica entre los diez de mayor exposición cibernética de Latinoamérica. Esta cifra no es, en sí misma, un antecedente técnico del diseño de logSguarDian –no informa decisiones de arquitectura, algoritmo o umbral–. Sí fundamenta la premisa sobre la cual se justifica la existencia del proyecto.

Esa premisa es que la brecha entre la superficie de ataque documentada por estándares como OWASP Top 10 [1] y la capacidad real de organizaciones pequeñas para mitigarla no es un escenario hipotético, sino una condición medible del entorno en el que logSguarDian está pensado para operar. Este panorama se retoma en el capítulo de Justificación para argumentar la relevancia social y económica del proyecto, y no se desarrolla aquí más allá de establecer el marco contextual en el que se inserta la motivación técnica de las secciones siguientes.

VI.7. Aprendizaje automático supervisado y no supervisado en ciberseguridad

El modelo híbrido de logSguarDian combina un clasificador supervisado y un detector de anomalías no supervisado. Esta decisión se fundamenta en la distinción formal establecida por Chandola et al. [10]. La detección supervisada requiere ejemplos etiquetados de cada clase de anomalía que se desea reconocer. Ofrece alta precisión sobre esas clases específicas.

La detección no supervisada de una sola clase modela únicamente la distribución de lo normal. Señala como anómalo cualquier punto que se aleje de ella, sin haber observado ejemplos de la anomalía durante el entrenamiento. Ambos paradigmas son casos particulares de los métodos de ensamble, que Dietterich [20] caracteriza como algoritmos que combinan las predicciones de múltiples modelos base para obtener un clasificador agregado cuyo error es, bajo condiciones razonables de diversidad entre los modelos base, menor que el de cualquiera de ellos por separado.

Esta distinción formal permite justificar, más adelante en este capítulo, por qué ninguno de los dos paradigmas por sí solo –supervisado o no supervisado– es suficiente como criterio único de bloqueo. Por eso logSguarDian los combina y les asigna roles distintos. El componente supervisado es la única autoridad de bloqueo sobre las categorías de ataque conocidas. El componente no supervisado es una capa complementaria de enriquecimiento de registro frente a variantes no vistas durante el entrenamiento.

VI.7.1. Random Forest

El componente supervisado es un Random Forest, introducido por Breiman [8] como un método que construye un ensamble de árboles de decisión. Lo hace mediante dos fuentes independientes de aleatoriedad. Cada árbol se entrena sobre una muestra bootstrap distinta del conjunto de entrenamiento. En cada nodo de división se evalúa únicamente un subconjunto aleatorio de las características disponibles, en lugar de todas ellas. La predicción agrega las de los árboles (voto mayoritario en la formulación original de Breiman; scikit-learn promedia las probabilidades de clase).

Esta doble aleatorización reduce la correlación entre los árboles individuales y, con ella, la varianza del ensamble frente a un único árbol de decisión, sin requerir normalización previa de las características ni supuestos sobre su distribución. Estas dos propiedades –tolerancia a características de escalas heterogéneas y complejidad de inferencia lineal en el número de árboles– son la justificación técnica directa. Esa justificación también se documenta en el capítulo de Antecedentes. Explica por qué Random Forest fue seleccionado frente a alternativas de mayor capacidad, como las redes neuronales profundas. El componente debe ejecutarse dentro del presupuesto de latencia de un middleware.

VI.7.2. Isolation Forest

El componente no supervisado es un Isolation Forest, propuesto por Liu et al. [9]. Su principio es distinto al de la mayoría de los detectores de anomalías de la época, que suelen modelar la densidad o la distancia entre puntos. En lugar de perfilar qué aspecto tiene un punto normal, Isolation Forest mide qué tan fácil es aislar un punto del resto mediante particiones aleatorias sucesivas del espacio de características. Una observación anómala, por estar más alejada de la masa de datos normales, requiere en promedio un número menor de particiones para quedar aislada en su propia hoja que una observación típica; esa longitud de camino promedio, normalizada, constituye el puntaje de anomalía. Esta propiedad hace que el algoritmo tenga la misma complejidad computacional lineal que Random Forest sin requerir el cálculo de distancias par a par que sí exigen otros métodos de detección de anomalías, lo cual lo hace compatible con las mismas restricciones de inferencia embebida que motivaron la selección del componente supervisado.

VI.8. Redes neuronales ligeras (MLP) y autoencoders

Antes de fijar Random Forest e Isolation Forest como los algoritmos del modelo híbrido, el diseño de logSguarDian consideró dos alternativas basadas en redes neuronales, ambas descritas formalmente por Goodfellow et al. [21]. Un perceptrón multicapa (MLP) es una red neuronal compuesta por capas densas sucesivas de neuronas con una función de activación no lineal. Se entrena mediante descenso de gradiente y retropropagación del error, para aprender fronteras de decisión que un modelo lineal no podría representar. Aplicado como componente supervisado, ofrecería una capacidad de representación mayor que un ensamble de árboles. Esa capacidad mayor tiene como costo un proceso de entrenamiento más sensible a la escala de las características.

La arquitectura de inferencia de un MLP se basa en multiplicaciones de matrices en cada capa. Su costo computacional crece con la profundidad y el ancho de la red. Un autoencoder es una arquitectura no supervisada compuesta por un codificador y un decodificador. El codificador comprime la entrada a una representación de menor dimensión. El decodificador intenta reconstruirla.

Entrenado únicamente sobre tráfico benigno, el error de reconstrucción sobre una entrada nueva sirve como puntaje de anomalía. Un error alto indica una entrada que la red no aprendió a representar bien, porque difiere de la distribución de entrenamiento. Este puntaje es conceptualmente análogo al puntaje de aislamiento de Isolation Forest.

Ambas alternativas fueron consideradas y descartadas a partir de la lógica documentada en

el capítulo de Antecedentes, y la medición empírica de este proyecto sobre el mismo corpus y contrato de características usados en producción confirma esa lógica de forma desigual entre los dos componentes. Para el componente supervisado, Random Forest (macro F1=0.9776) iguala en la práctica a un MLP equivalente (macro F1=0.9719) con una latencia de inferencia menor (0.0062 ms frente a 0.0074 ms), de modo que la ganancia de precisión que ofrecería la red neuronal es marginal frente al costo adicional, mínimo pero medible, que introduce.

Para el componente no supervisado, la misma medición matiza la justificación original. Un autoencoder entrenado sobre el contrato de características de Isolation Forest iguala su recall y su tasa de falsos positivos (0.9158/0.0517 frente a 0.9125/0.0546) con una latencia de inferencia y un tamaño de modelo serializado órdenes de magnitud menores (0.0065 ms frente a 0.7441 ms; 19.8 KB (19.3 KiB) frente a 2.16 MB (2.06 MiB)). Este costo diferencial no proviene de que Isolation Forest sea, como familia de algoritmo, más costosa que una red neuronal equivalente. Proviene de la configuración adoptada en producción: 200 árboles con `max_samples=4096`, fijados por razones de estabilidad de calibración. Proviene también de que su exportación a ONNX resulta, en la práctica, un subgrafo menos eficiente que el de Random Forest.

La decisión de mantener Isolation Forest se sostiene, entonces, por una razón arquitectónica distinta del costo bruto de inferencia. Este componente no tiene autoridad de bloqueo sobre la respuesta al cliente. Se ejecuta de forma asíncrona en el *worker pool* dedicado documentado en el capítulo de Alcance. Su latencia queda absorbida fuera del camino crítico de la petición. Por eso la comparación de latencia bruta es menos determinante para este componente que para el supervisado.

La consideración explícita de ambas alternativas, y no solo la ausencia de su implementación, es lo que permite documentar la selección de Random Forest e Isolation Forest como una decisión de diseño evaluada –y en el caso de Isolation Forest, sostenida por arquitectura y no por costo bruto– y no como la única opción contemplada.

VI.9. Ingeniería de características para datos de seguridad HTTP

Ningún algoritmo de aprendizaje automático opera directamente sobre un objeto de petición HTTP. Requiere que ese objeto se transforme primero en un vector numérico de dimensión fija. Este proceso se denomina ingeniería de características, según Zheng y Casari [22]. Esos autores lo describen como la etapa que determina en la práctica gran parte del desempeño final de un modelo, con independencia del algoritmo de clasificación elegido sobre esas características.

Entre las técnicas que documentan y que logSguarDian adopta de forma directa se encuentran tres. La primera es el conteo de ocurrencias de patrones específicos dentro de una cadena de texto (por ejemplo, el número de operadores lógicos SQL o de secuencias de ascenso de directorio dentro de un payload). La segunda es la codificación de atributos categóricos –como el método HTTP o la presencia de un encabezado particular– mediante variables indicadoras. La tercera es la derivación de estadísticos simples sobre texto libre, como la longitud de una cadena o la proporción de caracteres no alfanuméricos que contiene.

El vector de 76 características de logSguarDian, descrito en detalle en el capítulo de Alcance y en la especificación de características del proyecto, aplica estos principios generales a seis grupos funcionales: semántica de ataque (marcadores de las cuatro categorías cubiertas), estructura HTTP,

URL y ruta, parámetros y cuerpo, evasión por encoding y variables temporales. La adaptación específica de este proyecto frente al cuerpo general de técnicas documentado por Zheng y Casari [22] es la selección del objeto `req` de Express como fuente de esas características. Se elige ese objeto y no un flujo de red o un documento de texto genérico. Esta decisión responde al vacío metodológico identificado en el capítulo de Antecedentes. Ese capítulo señala la ausencia de literatura de referencia sobre ingeniería de características a nivel de petición HTTP individual.

VI.10. Detección de anomalías y ataques de día cero

Un ataque de día cero se entiende aquí en su sentido más general, y no en su acepción más estricta de vulnerabilidad de software desconocida por el fabricante. Es una variante de payload que no se parece a ningún ejemplo visto durante el entrenamiento del componente supervisado. Puede tratarse de una técnica de ofuscación nueva, de una combinación de operadores no representada en el corpus, o de una categoría de ataque que ni siquiera fue anticipada como clase durante el diseño del modelo.

Un clasificador puramente supervisado como Random Forest no tiene, por construcción, forma de reconocer una entrada que no se parezca a ninguna de las clases sobre las que fue entrenado. La asigna a la clase más cercana en el espacio de características. Lo hace con una confianza que no refleja necesariamente qué tan atípica es esa entrada frente a la distribución de entrenamiento.

Esta limitación es la que motiva la inclusión de Isolation Forest como una capa de detección complementaria, según la distinción de Chandola et al. [10] desarrollada en la sección anterior. Esa capa no depende de haber visto ejemplos previos de una técnica de ataque para señalarla como estadísticamente atípica. En logSguarDian, esta capacidad se documenta en dos escenarios distintos. El primero es el diagnóstico en tiempo real dentro del middleware, donde el puntaje de anomalía enriquece el registro de una petición sin constituir por sí solo un criterio de bloqueo. El segundo, de forma más directa, es el pipeline de reentrenamiento continuo del proyecto, donde se registró una prueba de concepto, cuyo resultado se describe en la sección XI.1 de Recomendaciones.

VI.11. Métricas de evaluación: F1-score, AUC-ROC y matriz de confusión

La evaluación de un clasificador de amenazas no puede apoyarse únicamente en la exactitud (*accuracy*), la proporción de predicciones correctas sobre el total. Esto se debe a que un corpus con clases fuertemente desbalanceadas –como es el caso de tráfico benigno frente a tráfico malicioso en un entorno realista– permite que un clasificador que etiquete todo como benigno obtenga una exactitud alta sin utilidad práctica alguna. Por esta razón, los Objetivos específicos de este proyecto se formulan en términos de precisión, recuperación (*recall*) y su combinación en el F1-score, calculadas a partir de la matriz de confusión de verdaderos positivos (VP), falsos positivos (FP), verdaderos negativos (VN) y falsos negativos (FN):

$$\text{Precisión} = \frac{VP}{VP+FP} \qquad \text{Recall} = \frac{VP}{VP+FN} \qquad \text{F1} = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \tag{1}$$

La precisión mide qué proporción de las peticiones bloqueadas por el modelo corresponden efectivamente a un ataque. Su complemento (uno menos la precisión, ponderado por la tasa de falsos positivos) es la métrica directamente relevante para el criterio de aceptación del componente no supervisado. Es relevante porque un Isolation Forest con una tasa de falsos positivos alta bloquearía tráfico legítimo con la misma frecuencia con la que detecta anomalías reales. El recall mide qué proporción de los ataques reales el modelo efectivamente detecta, y es la métrica directamente relevante para el criterio de cobertura del 80 % por categoría de ataque establecido en el tercer objetivo específico.

El F1-score es la media armónica de ambas métricas. Penaliza a un clasificador que sacrifique una métrica para maximizar la otra. Un caso es bloquear indiscriminadamente para maximizar recall a costa de la precisión. Por esta razón, esta métrica se elige como criterio de aceptación del componente supervisado en el segundo objetivo específico. Fawcett [23] formaliza además el análisis de curvas ROC (*Receiver Operating Characteristic*) y de su área bajo la curva (AUC) como herramienta complementaria para evaluar el compromiso entre tasa de verdaderos positivos y tasa de falsos positivos a través de distintos umbrales de decisión. Ese principio se aplicó en este proyecto durante la calibración del umbral de bloqueo del componente supervisado y del umbral de anomalía del componente no supervisado.

VI.12. Edge AI e inferencia en tiempo de ejecución con ONNX

La ejecución de modelos de aprendizaje automático dentro de un proceso con restricciones estrictas de latencia y memoria, sin comunicación con servicios externos, está documentada de forma consolidada en la literatura de Edge AI, cuyo texto de referencia es el de Warden y Situnayake [12]. Los autores establecen los principios de diseño para desplegar modelos ligeros bajo presupuestos de memoria y tiempo de inferencia del orden de milisegundos. Esos principios incluyen técnicas de cuantización y formatos de serialización optimizados. Esta disciplina se desarrolló casi en su totalidad sobre microcontroladores y dispositivos embebidos de hardware dedicado. Provee el principio general que logSguarDian traslada a un dominio de ejecución distinto: el de un único proceso Node.js de un servidor de aplicaciones.

El entrenamiento de los modelos de logSguarDian se realiza en Python sobre *scikit-learn*, mientras que su inferencia en producción ocurre dentro de un proceso Node.js, dos entornos de ejecución sin compatibilidad binaria directa entre sí. Open Neural Network Exchange (ONNX) resuelve esta discontinuidad como un formato abierto de representación de modelos de aprendizaje automático. Define un grafo de cómputo y un conjunto de operadores estándar, independientes del framework en el que el modelo fue entrenado [24]. Así, un modelo serializado en ONNX se ejecuta mediante un motor de inferencia distinto (en este caso, *onnxruntime-node*) al que se usó para entrenarlo. Esta propiedad es la que hace viable, en primer lugar, entrenar los modelos híbridos de logSguarDian con el ecosistema maduro de *scikit-learn* en Python y, en segundo lugar, distribuirlos embebidos dentro de un paquete npm sin requerir que el proceso de inferencia dependa de un intérprete de Python en producción.

La contrapartida de esta traducción entre representaciones es la necesidad de verificar que ambas produzcan predicciones equivalentes sobre las mismas entradas –verificación de paridad que este proyecto ejecuta de forma sistemática tras cada cambio de modelo, dado que una divergencia entre el modelo evaluado en Python y el modelo efectivamente servido en Node.js invalidaría las métricas

reportadas en este trabajo.

Resta resolver, sin embargo, la restricción de concurrencia descrita en la sección de Node.js y Express de este capítulo: un motor de inferencia ONNX ejecutado de forma síncrona en el hilo principal bloquearía el Event Loop durante cada predicción. La documentación oficial de Node.js introduce `worker_threads` precisamente como el mecanismo para delegar trabajo de cómputo intensivo a hilos del sistema operativo independientes del Event Loop. Se comunican con el hilo principal mediante paso de mensajes en lugar de memoria compartida [16]. Este mecanismo es el fundamento técnico directo del primer objetivo específico del proyecto. Ese objetivo exige que el pipeline de ingeniería de características y la inferencia del modelo híbrido se ejecuten de forma asíncrona mediante `worker_threads` sin introducir una latencia adicional superior a 5 ms por petición en el servidor anfitrión.

Este mecanismo también fundamenta la arquitectura de *worker pool* dedicado documentada en el capítulo de Alcance. La librería declara Node.js 20 o superior en el campo `engines` de su paquete. El módulo `worker_threads` es estable desde la versión 12 de Node.js, pero esa versión no es el mínimo declarado por la librería.

VI.13. Construcción y etiquetado de datasets de tráfico HTTP

Entrenar un clasificador supervisado como Random Forest requiere un corpus en el que cada petición esté etiquetada con su categoría real. logSguarDian resuelve esta condición mayoritariamente con la etiqueta ya asignada por la publicación original de cada dataset público que incorpora –SR-BH 2020/CAPEC, modsec-learn, registros de honeypot de ModSecurity, SecLists, PayloadsAllTheThings y varios conjuntos de Kaggle, entre otros–. Sus licencias se verificaron en la auditoría de datasets, salvo dos que no declaran licencia. No se requiere anotación manual adicional.

Esta estrategia deja, sin embargo, un problema distinto sin resolver. Ningún dataset público disponible cubre de antemano los vacíos de cobertura específicos que solo se hacen evidentes al validar el modelo ya entrenado –por ejemplo, un patrón de navegación benigna subrepresentado que un análisis de exclusión por fuente (LOSO) revela como punto débil, o una técnica de inyección de comandos cuya confianza de detección una prueba de ablación muestra insuficiente–. Para ese problema, distinto al de la etiquetación del corpus público, logSguarDian recurre a un suplemento acotado de tráfico sintético generado deliberadamente para cubrir cada vacío identificado.

Shiravi et al. [11] proponen el enfoque adoptado como base metodológica para ese suplemento. En lugar de etiquetar tráfico observado después de generarlo, se despliega un entorno de prueba controlado. Se simula tráfico legítimo representativo y se ejecutan herramientas de ataque automatizadas de forma deliberada. Así, la etiqueta de cada muestra se conoce de antemano por construcción –*ground truth by design*– y no por inferencia posterior.

Este principio evita la necesidad de anotación manual. Garantiza una correspondencia exacta entre la muestra y su etiqueta. Tiene un costo: la representatividad de esa porción del corpus depende enteramente de qué tan bien el entorno de prueba y las herramientas de generación aproximan el tráfico real que el sistema enfrentará en producción. El capítulo de Alcance documenta este supuesto explícitamente como una condición verificada mediante análisis exploratorio del corpus resultante.

logSguarDian aplica este mismo principio metodológico únicamente al suplemento sintético. El principio consiste en desplegar un entorno de prueba controlado mediante Docker Compose, simular tráfico benigno y ejecutar payloads de ataque de forma deliberada. Además, lo dirige a un objeto distinto del que documenta la literatura de referencia sobre generación de datasets de intrusión. Esa literatura captura metadatos de flujo de red (duración de conexión, bytes transferidos, número de paquetes), como ocurre con los datasets de la familia CICIDS derivados de Shiravi et al. [11].

En cambio, cada muestra sintética del corpus de logSguarDian es un vector de características extraído del objeto `req` de Express. Ese vector se obtiene con el mismo pipeline de ingeniería de características que se ejecuta en producción, descrito en la sección correspondiente de este capítulo. Ese mismo pipeline se aplica después sobre las muestras de origen público, para producir un vector de entrada homogéneo con independencia de la procedencia de cada fila. Esta adaptación permite que el corpus de entrenamiento sea representativo de la semántica de una petición HTTP individual y no únicamente de patrones de tráfico a nivel de conexión.

VI.14. Síntesis: articulación de los fundamentos en la arquitectura de logSguarDian

Los fundamentos desarrollados en este capítulo no son piezas de conocimiento independientes. Se articulan directamente en las decisiones de diseño de logSguarDian. La caracterización de las cuatro categorías de ataque como fenómenos intra-request delimita la unidad de análisis del sistema a una petición HTTP individual. El modelo de middleware de Express sobre el Event Loop de un solo hilo de Node.js define tanto el punto de integración de logSguarDian como su restricción de latencia más estricta. La distinción entre arquitectura perimetral y arquitectura RASP embebida fundamenta la decisión de operar dentro del proceso de la aplicación en lugar de como proxy independiente.

La seguridad de la cadena de suministro de npm delimita una restricción de diseño sobre las dependencias del proyecto. El panorama de ciberseguridad del contexto local delimita la motivación de su existencia. La complementariedad entre detección supervisada y no supervisada fundamenta la combinación de Random Forest e Isolation Forest en un modelo híbrido. Esa misma complementariedad provee el mecanismo conceptual de detección de variantes de día cero. Esa combinación fue evaluada frente a las alternativas de redes neuronales descritas en este capítulo, y se sostiene en un caso por su costo de inferencia y en el otro por su arquitectura de ejecución asíncrona.

Los principios generales de ingeniería de características se adaptan al objeto de petición de Express para producir el vector de entrada de ambos modelos. Los modelos se entrenan sobre un corpus compuesto predominantemente por datasets públicos con etiqueta propia. Ese corpus se complementa con un suplemento sintético construido bajo el principio de etiquetado determinista documentado en la literatura de generación de datasets de intrusión. Los modelos se evalúan con las métricas de precisión, recall y F1-score, que expresan formalmente los criterios de aceptación de los objetivos específicos de este trabajo. Finalmente, ONNX y `worker_threads` resuelven, respectivamente, la discontinuidad entre el entorno de entrenamiento en Python y el entorno de inferencia en Node.js, y la restricción de concurrencia de un solo hilo bajo la cual ese entorno de inferencia debe operar sin degradar el servicio que protege.

CAPÍTULO VII

Metodología

VII.1. Enfoque y tipo de investigación

Este trabajo se desarrolla bajo un enfoque de investigación aplicada y de tipo experimental. Es aplicada porque su resultado no es una contribución teórica independiente, sino un artefacto de software concreto y utilizable: la librería npm logSguarDian, con su pipeline de ingeniería de características y su modelo híbrido integrados en un middleware funcional para aplicaciones Express. Es experimental porque la validez de ese artefacto no se argumenta de forma discursiva, sino que se somete a criterios de aceptación predefinidos y cuantificados –los umbrales de F1-score, recall y tasa de falsos positivos establecidos en los objetivos específicos de este trabajo– verificados empíricamente sobre un conjunto de prueba bloqueado y sobre un entorno de evaluación controlado y reproducible.

La combinación de ambos rasgos –construcción de artefacto y validación empírica contra criterios de aceptación predefinidos– responde directamente al vacío metodológico identificado en el capítulo de Antecedentes. La literatura de referencia en detección de intrusiones mediante aprendizaje automático privilegia consistentemente los datasets de flujo de red heredados sobre la semántica de peticiones HTTP individuales. Ni las revisiones más recientes de arquitecturas híbridas supervisadas/no supervisadas ni las revisiones sistemáticas de cortafuegos de aplicación web potenciados por inteligencia artificial documentan, entre los estudios que cubren, un sistema implementado bajo una arquitectura embebida dentro del proceso de una aplicación Node.js con recursos limitados.

Al no existir un antecedente documentado que combine un modelo híbrido de aprendizaje automático con las restricciones de latencia, memoria y concurrencia de un solo hilo propias de un middleware Node.js, este trabajo no puede apoyarse en un diseño experimental replicado de la literatura. Requiere construir el artefacto y definir sus propios criterios de aceptación. Por ello, la clasificación que corresponde al problema es la de investigación aplicada-experimental, y no una

alternativa entre varias igualmente válidas.

VII.2. Metodología de trabajo y fases

El desarrollo del proyecto se organizó en seis fases secuenciales, cada una con una disciplina operativa asociada y un artefacto verificable como criterio de cierre (regla R3 del proyecto: ninguna fase se da por completa sin un número reproducible, una prueba que pasa, o un archivo committeado que la respalde).

Fase 1 — Construcción del pipeline de ingeniería de características (20 de junio – 1 de agosto de 2026, objetivo específico 1). Se implementó, en TypeScript, la extracción y vectorización de un vector de 76 características numéricas a partir del objeto `req` de Express, ejecutada de forma asíncrona mediante *worker_threads* para no bloquear el *Event Loop* del proceso anfitrión (implementación inicial de 72 características a cargo de Sebastián Huertas; véase la sección VII.7). Esta fase está gobernada por la regla R1 del proyecto: la extracción de características vive únicamente en `packages/extractor` (TypeScript); ningún componente en Python recalcula features a partir de datos crudos, sino que consume matrices ya extraídas por el CLI del extractor. Esta disciplina se verificó de forma explícita incluso en la evaluación final del modelo (`training/evaluate_test.py`), donde la selección de columnas se realiza por nombre contra la lista canónica `FEATURE_NAMES` copiada literalmente del extractor TypeScript, en lugar de inferir el vector de entrada por descarte posicional. El artefacto de cierre de esta fase es el benchmark de latencia del extractor. Su p95 mixto con *worker_threads* es de 0.0273 ms (mediana de tres corridas), muy por debajo del límite de 5 ms establecido en el objetivo específico 1 (Cuadro 7).

Fase 2 — Construcción del corpus de entrenamiento. Se ensambló un corpus a partir de nueve conjuntos auditados, ocho incluibles y siete incorporados, más dos listas públicas. Su licencia y procedencia se verificaron y se documentaron en la auditoría de datasets del proyecto. El corpus se complementó con un suplemento sintético acotado, generado mediante Docker Compose para cerrar vacíos de cobertura. Esos vacíos se detectaron mediante análisis de exclusión por fuente (*Leave-One-Source-Out*) y pruebas de ablación sobre el corpus público. El suplemento nunca es la base primaria del corpus.

La fase cierra con la deduplicación exacta del corpus, el marcado de casi-duplicados y el bloqueo criptográfico del conjunto de prueba (`training/splits/test.lock.sha256`). Este bloqueo activa la regla R2: el conjunto de prueba se lee una sola vez para la evaluación final de cada generación de modelo. Ninguna decisión de ajuste posterior a esa lectura puede modificarse en función de los resultados observados en él. La generación v11 incumplió esta regla, como se detalla más adelante en este capítulo.

Fase 3 — Entrenamiento y calibración del modelo híbrido. El entrenamiento de Random Forest e Isolation Forest se realizó íntegramente en Python sobre hardware Apple Silicon local, sin dependencia de infraestructura de cómputo en la nube. La selección de los umbrales de decisión se realizó sobre el conjunto de validación. En dos versiones de Isolation Forest esa selección se revisó después de leer el conjunto de prueba. En `if_v2`, la confirmación arrojó una tasa de falsos positivos de 0.1011 frente a un límite de 0.10, y el objetivo de calibración pasó a FP $\leq$ 0.08. En `if_v5`, la primera confirmación arrojó 0.0811 frente a un objetivo interno de 0.08, y el objetivo se endureció a FP $\leq$ 0.06.

En ambos casos el umbral final se seleccionó sobre validación, pero el conjunto de prueba influyó en el criterio de calibración. El umbral de `if_v10` se calibró sobre validación como el máximo recall entre los umbrales que cumplen recall ≥ 0.50 y FP ≤ 0.06 (`retrain_v11_contract69.py`, líneas 107–110). El de Random Forest (`RF_THRESHOLD = 0.35`) se calibró sobre validación para `rf_v3` y se mantuvo sin recalibración para `rf_v11`.

Fase 4 — Verificación de paridad ONNX. Ambos modelos se serializaron a formato ONNX mediante `sklearn2onnx`, y su paridad de predicción frente al modelo original en `scikit-learn` [25] se verificó de forma automatizada (`training/models/parity_report.json`). La verificación confirmó diferencias de probabilidad en el orden de 10^{-7} entre ambos entornos de inferencia. Esto ocurrió antes de habilitar el modelo serializado para su integración.

Fase 5 — Integración en el middleware. El modelo serializado se integró en la librería npm mediante `onnxruntime-node`, ejecutado en una arquitectura de *worker pool* dedicado (un worker para Random Forest y dos workers para Isolation Forest en hilos separados), diseño adoptado tras confirmar empíricamente que `onnxruntime-node` serializa las llamadas de inferencia concurrentes dentro de un mismo hilo nativo. Esta fase cierra con la evaluación del modelo híbrido sobre el conjunto de prueba bloqueado (objetivo específico 2, cierre 2 de agosto de 2026).

Fase 6 — Evaluación en tres configuraciones progresivas, la tercera con dos variantes (3a, solo cortafuegos; 3b, en capas) (2 de agosto – 30 de septiembre de 2026, objetivo específico 3). La librería integrada como middleware se evaluó en un entorno de prueba controlado (Docker Compose) bajo tres configuraciones sucesivas, la tercera con dos variantes (3a, solo cortafuegos; 3b, en capas): sin protección, con logSguarDian como única capa, y en defensa en profundidad junto con un Web Application Firewall perimetral. El refinamiento iterativo del modelo continuó dentro de esta fase. Incluyó el ciclo de reentrenamiento que atendió los vacíos de cobertura de la categoría de inyección de comandos detectados durante la evaluación. Ese refinamiento no alteró los criterios de aceptación establecidos en los objetivos específicos.

VII.3. Técnicas e instrumentos

La técnica central de la fase de construcción del modelo es la ingeniería de características sobre el objeto `req` de Express: un vector de 76 dimensiones organizado en grupos funcionales. Estos grupos capturan los marcadores semánticos de las cuatro categorías de ataque cubiertas (inyección SQL, *cross-site scripting*, recorrido de directorios e inyección de comandos). Además, el vector incluye características estadísticas transversales (longitud, entropía de Shannon, proporción de caracteres especiales) y metadatos de la petición (método HTTP, encabezados, indicadores de codificación).

Conviene distinguir lo que el extractor produce de lo que cada modelo consume. De esas 76 dimensiones, el componente Random Forest consume 69 y el componente Isolation Forest consume 63. Cada modelo excluye las variables no disponibles en el momento de la intercepción de la petición (código de estado de respuesta, contadores de frecuencia por ventana temporal). Isolation Forest excluye además las variables con varianza próxima a cero sobre tráfico benigno.

En ambos modelos, de forma temporal, se excluye la característica más reciente del extractor (`non_json_quote_count`, que descuenta las comillas de sintaxis JSON del conteo de comillas del

payload). Esta última se excluye no por invalidez, sino porque su incorporación al vector de entrada de los modelos en producción implica un reentrenamiento completo. Esa promoción es una decisión pendiente y deliberadamente separada de su adición al extractor.

El Cuadro 4 enumera los seis grupos funcionales y el número de variables que contiene cada uno, según la tabla de features de la especificación del extractor.

Grupo	Descripción	Variables	Posiciones en la especificación
A	Semántica de ataque (SQLi, XSS, *path traversal*, CmdI)	37	#26–#62
B	Estructura HTTP	9	#63–#71
C	URL y ruta	6	#3–#5, #8–#10
D	Parámetros y cuerpo (composición del payload)	9	#1, #2, #6, #7, #11–#14, #18
E	Evasión por encoding y caracteres	10	#15–#17, #19–#25
—	Temporales (ventanas de 1, 5 y 60 segundos)	5	#72–#76
Total		**76**	

Cuadro 4: Grupos de características del vector de 76 dimensiones, con el número de variables de cada uno. Fuente: `docs/feature-spec.md` del repositorio logSguarDian, rama `docs/feature-spec-76`. Elaboración propia.

Las variables temporales (#72–#76) no son consumidas por ninguno de los dos modelos, porque requieren estado compartido entre peticiones que el extractor no mantiene por diseño. Tampoco lo son `status_code` (#71), que no existe al interceptar la petición, ni `non_json_quote_count` (#32). Random Forest consume así 69 variables; Isolation Forest consume 63, tras excluir además `null_byte_count`, `sensitive_file_target`, `dotdot_encoded_count`, `os_path_indicator`, `authorization_length` y `unusual_headers_count`.

Sobre ese vector opera el modelo híbrido de aprendizaje automático descrito en el Marco Teórico: Random Forest [8] como componente supervisado, encargado de clasificar cada petición en una de las cuatro categorías de ataque cubiertas o como tráfico legítimo, e Isolation Forest [9] como componente no supervisado, entrenado únicamente sobre tráfico benigno para aportar una señal complementaria de detección de anomalías estadísticas frente a variantes no vistas durante el entrenamiento. La justificación teórica de esta combinación –la complementariedad entre detección supervisada y no supervisada de una sola clase [10], y la selección de métodos de ensamble ligeros frente a alternativas de redes neuronales de mayor capacidad– se desarrolla en el Marco Teórico y no se repite aquí; esta sección se limita a documentar su aplicación instrumental. En su forma ONNX publicada, `rf.onnx` pesa 8.90 MB (8.49 MiB) e `if.onnx` 2.16 MB (2.06 MiB).

El instrumental de construcción y despliegue del modelo comprende `scikit-learn` [25] para el entrenamiento de ambos algoritmos, `sklearn2onnx` para su serialización a formato ONNX [24], `onnxruntime-node` como motor de inferencia embebido en el proceso Node.js, y Docker Compose. Docker Compose levanta tanto el entorno de generación de tráfico sintético (fase 2) como el entorno de prueba vulnerable de las tres configuraciones de evaluación (fase 6). La restricción de recursos de este instrumental –modelos de ensamble de complejidad computacional acotada, entrenados y servidos sin infraestructura de cómputo en la nube– es consistente con la disciplina de inferencia embebida bajo restricciones de memoria y latencia documentada en la literatura de Edge AI [12].

Como técnica de interpretabilidad se empleó SHAP, específicamente su explicador `TreeExplainer` sobre el modelo Random Forest. Se aplicaron dos modalidades complementarias. La primera es la

importancia global por clase, calculada sobre el conjunto de validación para identificar las diez características con mayor contribución promedio a cada categoría de ataque. La segunda es la explicación local de casos individuales, empleada para diagnosticar el comportamiento del modelo frente a peticiones específicas (por ejemplo, el efecto de la presencia o ausencia de encabezado *User-Agent* sobre la puntuación de Isolation Forest). Los casos locales analizados mediante esta técnica se documentan con su evidencia numérica en el capítulo de Resultados.

El análisis se calculó sobre `rf_v11.pkl` tras la corrección de trazabilidad de artefactos descrita en la sección de Recolección, procesamiento y validación de datos, por lo que las características y casos que explica corresponden al modelo efectivamente evaluado en este capítulo. La importancia global confirma que los encabezados de *User-Agent* (`ua_present`, `ua_length`) figuran entre las características de mayor contribución. Esto ocurre no solo para la calibración de Isolation Forest ya documentada, sino también para la clasificación de Random Forest en las categorías `benign`, `sqli` y `path_traversal`, con una contribución más débil, aunque presente, en `cmdi` y `xss`. Este predominio es desigual entre clases: es más marcado en tres de las cinco categorías y no debe leerse como uniforme.

Para `cmdi`, el análisis aporta evidencia cuantitativa adicional sobre la dificultad de separabilidad ya documentada. Incluso sobre el modelo corregido, las dos características de mayor contribución para esta clase (`method_is_get`, `ua_length`) no son marcadores semánticos de inyección de comandos. Solo una característica semántica relacionada (`subshell_count`) figura entre las diez de mayor peso. `shell_command_count` y `pipe_count`, pese a formar parte del vector de entrada, no se encuentran entre las diez principales. Esto es consistente con, aunque no idéntico a, la dificultad estructural ya identificada entre `cmdi` y `path_traversal`, y se documenta aquí como un hallazgo parcialmente resuelto, no cerrado.

VII.3.1. Pares de modelos evaluados

El paquete 0.1.0 y los modelos sobre los que se midieron las métricas de este trabajo no son el mismo par. Las métricas del capítulo de Resultados corresponden al par C, mientras que el paquete `logsguardian@0.1.0` del registro contiene el par A. El Cuadro 5 identifica cada par por el SHA-256 de sus archivos ONNX y registra en qué evaluaciones se usó.

Notas: (1) El JSON de resultados de la repetición de Config 2 del 9 de septiembre y el del estudio de User-Agent no registran el modelo. Su atribución se apoya en el mensaje del commit `14c0e15` de la aplicación vulnerable del proyecto, que actualizó los modelos a `rf_v11`/`if_v10`, y en el contenido de `models/` en el commit del estudio (`5966712`). (2) De las aplicaciones de terceros DVNA, node-api-goat y dvws-node no consta el modelo exacto de cada medición; solo la medición de Juice Shop usa una copia cuyo modelo sí consta.

(3) La Ronda 5 con el par A (tarball e09120…) se atribuye al par A según `docs/config3b-results.md` de la aplicación vulnerable del proyecto (rama main); el mismo documento atribuye el tarball del commit `22550ea` al par C. Los tres archivos se denominan `logsguardian-0.1.0.tgz`, pero son distintos. El tgz de la versión 0.1.0 del registro tiene SHA-256 `c5c1a1da57cfb873350ac0c8ab3e7bad395e8cbf507b18cc3272df447880dcb0` y contiene el par A; el de la versión 0.1.1 (11 de octubre de 2026, UTC) tiene SHA-256 `abf8d5c47036d9c3a289bc36158f1caaa91b9504689aa97193e51c6808258d15` y contiene el par C. El paquete 0.1.0 contiene el par A. El tarball de la Ronda 5 original (commit `a3aada0`, SHA-256 `e09120006dd17e790838f2ae92bb68ee4c7f3a5e160c2c4c83f5a513909fe359`) también contiene el par A, pero no es el tgz del registro. El tarball de validación y la copia local de Juice Shop, no publicados, contienen el par C.

Los nombres usados en este documento son cinco. El *paquete 0.1.0 del registro* (`logsguardian@0.1.0`, par A; SHA-256 `c5c1a1da57cfb873350ac0c8ab3e7bad395e8cbf507b18cc3272df447880dcb0`). El *tarball de la Ronda 5*

Par	Modelos	SHA-256 (`rf.onnx` / `if.onnx`)	Origen	Evaluaciones en que se usó
A	`rf_v10` / `if_v9`, 67 y 61 características	`18083725...` / `5de374a7...`	Commit `dc2aaa65`, 2 de agosto de 2026	Paquete 0.1.0 (par A), `logsguardian@0.1.0` (30 de agosto de 2026, commit `c3d591a`); Configuraciones 2, 3a y 3b sobre los 590 payloads (Ronda 4, 15 de agosto de 2026; sobre el solapamiento de ese corpus con el entrenamiento, véase la sección VIII.3.5)[4]; Configuración 2 sobre 100 payloads por categoría (6 de agosto de 2026; el modelo consta en `docs/config2-latency-evaluation.md`, sin hash); Ronda 5 con el par A (tarball e09120...)[3]; estudio de User-Agent de la aplicación vulnerable del proyecto (24 de septiembre)[1]
B	`rf_v11` / `if_v10`, primera versión, 69 y 63 características	`a7c015f5...` / `3cf8e45c...`	Commit `734c24f`, 30 de agosto de 2026	Primera lectura del conjunto de prueba (24 de septiembre); repetición de Config 2 sobre los 590 payloads (9 de septiembre)[1]; experimento de latencia, memoria y CPU en Linux nativo
C	`rf_v11` / `if_v10`, reentrenamiento, 69 y 63 características	`25b407e6...` / `5be28ee8...`	Commit `03de50f`, 26 de septiembre de 2026	Lecturas del 25 y 26 de septiembre del conjunto de prueba; paridad ONNX; análisis SHAP; Ronda 5 (9 de octubre de 2026) con el tarball de validación del commit `22550ea` (SHA-256 `cf1238...`; véase la sección VIII.3.6)[3]; copia de Juice Shop[2]

Cuadro 5: Pares de modelos evaluados, con su origen y las evaluaciones en que se usó cada uno.

Fuente: commits `dc2aaa65`, `734c24f` y `03de50f` del repositorio logSguarDian; hashes de `training/models/` (`rf.onnx` e `if.onnx`); `docs/config3b-results.md` del repositorio logSguarDian-vulnerable-project. Elaboración propia.

original (commit a3aada0, SHA-256 e09120006dd17e790838f2ae92bb68ee4c7f3a5e160c2c4c83f5a513909fe359, par A). El *tarball de validación del commit 22550ea* (SHA-256 cf1238deb33383975e12058de71c4423b22d7c5bcba5cf47241c0d0f712b0416, par C). El *paquete 0.1.1 del registro* (logsguardian@0.1.1, par C; SHA-256 abf8d5c47036d9c3a289bc36158f1caaa91b9504689aa97193e51c6808258d15). La *copia local de Juice Shop* (SHA-256 c3497bddc69d68bb75eadf494d98b959b7dd4a2b2653cbe945fa2c242a5848a1, par C).

(4) La Configuración 1 no usa los modelos, porque la aplicación no tiene protección. En las Rondas 1 a 3, con corpus pequeño, el modelo no consta en los resultados.

Hashes completos. Par A: rf.onnx 180837255dffd596a38bf7c29032ccc313005cd70eca5c6238e23e35bdc0d4ec e if.onnx 5de374a7f1d19176d7c66d3354e972d37b28a7306a87efa0ce8639278f489150. Par B: rf.onnx a7c015f5a5fa3fabfd91b79ec9338dc7da6a68c392c19ded4b04cc6f0fb903b2 e if.onnx 3cf8e45c285de19bc86e6dc284e502cd2c24becb888747b9645bed4cfb224417. Par C: rf.onnx 25b407e649283606a231665a5a8c8ea1c2c7ad4fd0374a8945b3f119651b0870 e if.onnx 5be28ee8437643b2f13708eee1ac3175348a350ea6fb8f8daeb09bd5717f797e.

VII.3.2. Validación en aplicaciones de terceros

La validación con una startup guatemalteca, prevista en el protocolo, era opcional y no se realizó. Por recomendación del asesor, Ing. Erick Francisco Marroquín Rodríguez, el sistema se evaluó en cuatro aplicaciones vulnerables de terceros: DVNA, node-api-goat, dvws-node y OWASP Juice Shop. En las tres primeras se usó el middleware en modo block con la configuración por defecto (RF_THRESHOLD=0.35), y se enviaron ataques reales junto con tráfico benigno.

En OWASP Juice Shop se usó el modo monitor y solo se enviaron peticiones benignas, porque con la tasa de falsos positivos observada no se generó corpus de ataque para esa aplicación. En esa misma aplicación se comparó el efecto del encabezado *User-Agent*, con un valor de navegador y con curl. Los resultados se presentan en la sección VIII.3.7.

VII.4. Recolección, procesamiento y validación de datos

El corpus final reúne 383 202 filas de 17 fuentes: 268 240 en entrenamiento, 57 481 en validación y 57 481 en prueba. Siete provienen de conjuntos públicos auditados (CAPEC, modsec-learn, registros de ModSecurity [26], Payloads.csv, payload_full, XSS_dataset y command injection). Dos provienen de listas públicas de payloads (SecLists[27] y PayloadsAllTheThings[28], ambas con licencia MIT). Siete provienen de generadores propios y una de telemetría curada del pipeline de MLOps (119 filas de una ejecución simulada, no de tráfico real; anexo XIII.5).

La auditoría evaluó nueve conjuntos: ocho se consideraron incluibles y uno se excluyó por incompatibilidad de taxonomía de etiquetas (AIT-LDSv2, porque sus etiquetas describen roles del atacante y no tipos de payload). Uno de los ocho, omurugur, no se incorporó al corpus final. Las licencias de los conjuntos auditados se verificaron con las API de Zenodo, Harvard Dataverse y GitHub y con los metadatos de Kaggle; dos conjuntos quedan con licencia desconocida (XSS_dataset y command injection).

Ese suplemento sintético no es una fuente primaria del corpus. Constituye un conjunto de correcciones dirigidas. synthetic_nav y synthetic_nav_ecommerce se generaron para cerrar un vacío de generalización sobre tráfico benigno de navegación pura detectado mediante *Leave-One-Source-Out*. Al excluir synthetic_nav del entrenamiento, el F1 macro sobre sus filas fue de 0.0027, el menor de las nueve fuentes evaluadas según docs/limitations.md. Tras añadir synthetic_nav_ecommerce al corpus, fue de 0.2916 (loso_results.json; el archivo no registra el modelo, y el script entrena un Random Forest nuevo con los hiperparámetros de rf_v7, n_estimators=30 y max_depth=25). El valor 0.0027 consta solo como texto de una nota de ese archivo, no como métrica.

synthetic_multifield se generó para reforzar la señal de formularios multi-campo tras el hallazgo del bug de desempate en extractBestPayload. Por su parte, synthetic_cmdi_minimal_context, synthetic_cmdi_windows y synthetic_cmdi_compound se generaron específicamente para cerrar los vacíos de separabilidad de la categoría de inyección de comandos identificados mediante pruebas de ablación. Su justificación completa se describe en el Marco Teórico y en la sección de limitaciones documentadas del proyecto. En ningún caso el suplemento sintético sustituye al corpus público como base del entrenamiento: cada adición está motivada por un hallazgo específico de

cobertura, no por una decisión general de completar el corpus con datos generados.

La validación del corpus antes de su uso comprendió dos procedimientos concretos. El primero es la verificación de licencias: en los dos casos donde el propio proveedor (Kaggle) reporta la licencia como no especificada (`XSS_dataset.csv` y `command injection.csv`), esa ausencia se documenta explícitamente como tal en lugar de asumir una licencia permisiva por defecto. El segundo corresponde a la deduplicación del pipeline de unificación (`training/unify.py`) y a una auditoría aparte de casi-duplicados, separadas por su alcance.

El pipeline elimina las filas con huella exacta idéntica (path, query, body y etiqueta; función `deduplicate_exact`, línea 94) y solo marca, sin eliminar, los pares de consultas con distancia de Levenshtein menor que 3 dentro de la misma clase (líneas 179–181; campo `query` de 1 a 100 caracteres, con tope de 10 000 comparaciones por clase). La auditoría usó MinHash con *banding* de LSH ajustado explícitamente (`num_perm=128`, `params=(8,16)`), frente a la configuración por defecto derivada del umbral, que resultaba intratable en tiempo de cómputo sobre las clases de mayor volumen (`docs/limitations.md`, commit `bab5ce6` de logSguarDian).

Esa medición confirmó una saturación de duplicados considerable en las cinco clases: la clase `sqli` alcanza aproximadamente 395 veces más pares casi-duplicados que filas (dominada por generación por plantilla de `sqlmap`/CAPEC), y la clase `benign`, aproximadamente 9.5 veces, consistente con tráfico de cola larga y no con contenido generado por plantilla. La decisión tomada es el marcado sin eliminación de los casi-duplicados, porque remover filas reconfigura el balance de clases y requiere su propia estrategia de reentrenamiento. La cuantificación por partición se presenta en el anexo XIII.2.

El procesamiento del corpus hacia los conjuntos de entrenamiento, validación y prueba sigue un procedimiento secuencial y verificable, no una descripción abstracta de buenas prácticas. Primero, se unifican las fuentes dispares en un esquema común de columnas. Segundo, se extrae el vector de 76 características sobre cada registro unificado (de las cuales 69 alimentan a Random Forest y 63 a Isolation Forest, según se detalla en la sección de Técnicas e instrumentos). Esa extracción usa el mismo CLI del extractor TypeScript que se emplea en producción (regla R1: nunca se recalculan features en Python).

Tercero, se realiza la partición en conjuntos de entrenamiento, validación y prueba. Cuarto, se aplica el bloqueo criptográfico del conjunto de prueba. El hash SHA-256 del bloqueo se guarda en `training/splits/test.lock.sha256`, antes del reentrenamiento de v3, v4, v6, v8, v9, v10 y v11, según los mensajes de los commits. El lock de v11 se re-fijó el 23 de septiembre de 2026, después de su reentrenamiento (anexo XIII.3). Quinto, el entrenamiento y la calibración de ambos modelos se realizan exclusivamente contra los conjuntos de entrenamiento y validación.

La regla R2 –el conjunto de prueba se lee exactamente una vez para la evaluación final de cada generación de modelo– no es una declaración de intención. Es un procedimiento verificable en el propio repositorio. La evaluación final de `rf_v11`/`if_v10` (`training/evaluate_test.py`) recalcula el hash del conjunto de prueba restaurado y lo compara contra el valor bloqueado antes de ejecutar cualquier inferencia. Si los valores no coinciden, la evaluación aborta. El resultado de esa verificación queda registrado junto con las métricas en `training/results/v11_test_results.json` (`lock_verified: true`).

Para la generación v11, el conjunto de prueba se leyó en tres ocasiones, por lo que el criterio de lectura única de R2 no se cumplió. El 24 de septiembre de 2026 se evaluó el modelo entrenado el 30 de agosto (macro F1 de 0.9843) sobre el conjunto re-bloqueado el 23 de septiembre; el repositorio no conserva la huella de la partición con que ese modelo se entrenó, por lo que no puede descartarse solapamiento. El 25 de septiembre se evaluó un modelo reentrenado sobre la partición vigente, como parte de la comparación con MLP y autoencoder. El 26 de septiembre se evaluó el modelo vigente en el repositorio, reconstruido con el mismo procedimiento porque el artefacto serializado original había sido sobrescrito. Estas dos últimas lecturas coinciden en macro F1 (0.9776) y sus predicciones sobre validación son idénticas.

Las métricas de este capítulo corresponden a la lectura del 26 de septiembre. El historial no registra ninguna decisión de modelado atribuida al resultado de la primera lectura. El umbral de Isolation Forest de la primera lectura (0.00807) corresponde al modelo del 30 de agosto; el de las dos siguientes (0.00421), al modelo reentrenado, cuyo umbral se calculó sobre validación. El historial no atribuye ese cambio al resultado de la primera lectura, pero no puede descartarse su influencia.

La confirmación en prueba del umbral de 0.35 con `rf_v3`, afirmada en el commit `e74a8c6`, no tiene artefacto de respaldo en el repositorio. Dos limitaciones adicionales: 261 de las 1 468 filas de `cmdi` del conjunto de prueba (17.8 %) son sintéticas y su generador no está versionado, y la fuente de los datos restaurados el 23 de septiembre no está documentada.

Durante la preparación de este capítulo se detectó y corrigió una brecha de trazabilidad de artefactos en el pipeline de evaluación, documentada aquí por transparencia metodológica. El archivo `training/models/rf_v11.pkl`, al no estar bajo control de versiones, fue sobrescrito silenciosamente por una versión candidata –generada durante una investigación no relacionada sobre diversificación de tráfico benigno sintético, y explícitamente no promovida a producción– bajo el mismo nombre de archivo. Esto dejó huérfano el emparejamiento exacto entre modelo y conjunto de prueba que una evaluación previamente citada había medido. El mecanismo de bloqueo criptográfico del conjunto de prueba en sí no se violó. Ese bloqueo ancla la identidad de cada fila, no el valor de sus características, por lo que es correctamente invariante a cambios del extractor.

Sin embargo, ninguna suma de verificación vinculaba una métrica citada con el contenido exacto, byte a byte, del par modelo/conjunto de prueba que la produjo. Se reentrenaron `rf_v11`/`if_v10` contra el contrato de 69/63 features vigente en producción (`parity_report.json`) y se evaluaron sobre el conjunto de prueba correctamente re-bloqueado. Las cifras reportadas en este capítulo corresponden a la lectura del 26 de septiembre de 2026. El mismo conjunto se había leído también el 24 de septiembre (commit `3086318`) y el 25 de septiembre de 2026 (commit `fc47a5b`).

Como salvaguarda estructural se incorporaron tres mecanismos. El primero es un manifiesto de sumas de verificación sha256 (conjunto de prueba y ambos modelos ONNX) que acompaña ahora cada evaluación citada. El segundo es una extensión del pipeline de integración continua de entrenamiento que respalda y restaura `training/splits/` de la misma forma en que ya protegía `training/models/`. El tercero es el paso de `rf_v11.pkl`/`rf_v11_metadata.json` a control de versiones, para que el modelo efectivamente evaluado no pueda volver a sobrescribirse sin dejar rastro.

VII.5. Métricas e indicadores

Ninguna métrica de este trabajo se reporta como *accuracy* global. El corpus es fuertemente desbalanceado –la clase `benign` y la clase `sqli` concentran, entre las dos, más de la mitad de los registros, mientras que `cmdi` es la categoría de ataque con menor representación– y bajo ese desbalance un clasificador que prediga siempre la clase mayoritaria obtendría una *accuracy* alta sin haber aprendido a distinguir ninguna categoría de ataque. Por esa razón, cada componente del sistema se evalúa con la métrica que corresponde a su naturaleza –clasificación multiclase supervisada en un caso, detección de anomalías de una sola clase en el otro– y no con una métrica agregada que oculte ese desbalance.

Para Random Forest, la métrica de aceptación es el F1-score por clase: la media armónica de precisión y recall, que penaliza específicamente el sacrificio de una a expensas de la otra, a diferencia de reportar ambas por separado o de promediarlas aritméticamente. Esto importa de forma directa para este sistema: un modelo que maximizara solo precisión bloquearía pocas peticiones legítimas pero dejaría pasar ataques (bajo recall), y uno que maximizara solo recall bloquearía tráfico legítimo con frecuencia inaceptable para un middleware de producción (baja precisión); el F1-score es la métrica que exige un balance entre ambos costos operativos. El objetivo específico 2 fija el criterio de aceptación en F1-score mínimo de 0.80 en al menos tres de las cuatro categorías de ataque cubiertas, y no en las cuatro.

Esto reconoce explícitamente la dificultad estructural de separabilidad de `cmdi` documentada en el Marco Teórico y en las limitaciones del proyecto. La sintaxis de inyección de comandos comparte marcadores léxicos (rutas de sistema de archivos, operadores de redirección) con la categoría de recorrido de directorios. Esto reduce estructuralmente su separabilidad frente a las otras tres categorías, cuyos marcadores semánticos son más distintivos entre sí. El umbral de tres de cuatro clases no es, por tanto, una relajación arbitraria del criterio, sino el reconocimiento de una dificultad de separabilidad ya documentada antes de fijar el criterio de aceptación.

Sobre el conjunto de prueba bloqueado, la generación de modelos vigente (`rf_v11`/`if_v10`) alcanza un F1-score macro de 0.9776, con F1 por clase de 0.9984 (`benign`), 0.9239 (`cmdi`, precisión 0.9015 / recall 0.9475), 0.9826 (`path_traversal`), 0.9956 (`sqli`) y 0.9877 (`xss`). Las cinco clases superan el umbral de 0.80 del objetivo específico 2, incluida la clase estructuralmente más difícil. Como evidencia complementaria se calculó también el área bajo la curva ROC uno-contra-el-resto [23] para cada clase, con valores entre 0.99899 y 0.99998, que confirma que la separación entre clases no depende del punto de corte de probabilidad elegido para la decisión de bloqueo. Las métricas por clase se calculan sobre la clase de mayor probabilidad (argmax), sin el umbral de confianza de 0.35 con el que el sistema decide el bloqueo; no se calculó una métrica de prueba con la regla de decisión desplegada.

Para Isolation Forest, la métrica de aceptación es distinta por diseño: recall de detección de anomalías y tasa de falsos positivos sobre tráfico benigno, no F1-score. La razón es la naturaleza no supervisada del componente: Isolation Forest se entrena únicamente sobre tráfico benigno y no tiene conocimiento de las categorías de ataque como clases distintas,

por lo que no existe una matriz de confusión multiclase sobre la cual calcular precisión o F1 por categoría; su tarea es binaria (anómalo o no) y se evalúa como tal. El protocolo original fija el criterio de aceptación en recall de detección mínimo del 50 % con una tasa de falsos positivos máxima del 10 %. Ambos criterios se cumplen sobre el conjunto de prueba bloqueado (`training/results/v11_test_results.json`, lectura verificada contra el bloqueo criptográfico, correspondiente a la evaluación del 26 de septiembre de 2026 descrita en este capítulo) con la generación vigente. Se obtuvo recall agregado de 0.9125 y tasa de falsos positivos de 0.0546.

Ese umbral del 10 % de falsos positivos se endureció operativamente durante iteraciones previas del proyecto (generación `if_v5`) a un objetivo de calibración de 6 % en el conjunto de validación. No fue una preferencia estética, sino una lección concreta de margen insuficiente. Una calibración anterior de `if_v5` (umbral 0.02868, con un objetivo interno de calibración de 8 % en validación) dio en el conjunto de prueba una tasa de falsos positivos de 0.0811. Es decir, quedó 0.0011 por encima de ese objetivo interno.

El límite formal del objetivo específico 2 se mantenía en 10 %. El corrimiento observado entre validación y prueba se atribuyó al cambio de features introducido junto con `rf_v6` (el análisis por campo del cuerpo de la petición, que redujo la señal de longitud de payload de la que Isolation Forest dependía parcialmente). Ese cambio se introdujo antes de la calibración de `if_v5` dentro del mismo ciclo de reentrenamiento, y motivó la recalibración.

El modelo recalibrado de `if_v5` (umbral 0.00941) no es el de producción: el modelo vigente, `if_v10`, usa el umbral 0.00421 y también se calibró en validación contra una tasa de falsos positivos $\leq 6\,\%$. Desde entonces, la calibración de cada nueva generación de Isolation Forest elige como umbral el de máximo recall entre los que cumplen recall ≥ 0.50 y FP $\leq 6\,\%$ (`retrain_v11_contract69.py`, líneas 107–110). Así se deja margen deliberado de holgura frente al límite duro de 10 % fijado por el objetivo específico 2.

La verificación de paridad ONNX se reporta como una métrica de validación en sí misma, no como un detalle de implementación: dado que el entrenamiento ocurre en Python sobre `scikit-learn` y la inferencia en producción ocurre en Node.js sobre `onnxruntime-node`, cualquier discontinuidad entre ambos entornos invalidaría todas las métricas anteriores como descripción del comportamiento real del sistema en producción. La paridad medida entre ambos entornos es del orden de 10^{-7} en la diferencia de probabilidad de salida, confirmada de forma automatizada antes de habilitar cada modelo para su integración (`training/models/parity_report.json`, `parity_passed: true`). Esta métrica es la que permite afirmar que el F1-score, el recall y la tasa de falsos positivos reportados para el modelo en Python son también válidos como descripción del modelo efectivamente servido por el middleware.

Finalmente, la cobertura de detección por categoría en las configuraciones de evaluación integradas (Config 2 y Config 3) es una métrica distinta de las anteriores por su nivel de análisis. Mientras el F1-score y el recall/FP de Isolation Forest evalúan el modelo de forma aislada sobre un conjunto de prueba estático, la cobertura por categoría evalúa el sistema integrado –extractor, modelos e inferencia en *worker_threads*– contra tráfico HTTP real dirigido a una aplicación en ejecución. El objetivo específico 3 fija este criterio exactamente como sigue: "verificando que la detección de payloads representativos alcance al menos el 80 % por cada una de las cuatro categorías de ataque cubiertas en un entorno de prueba controlado, sin elevar la latencia p95 del servidor más de 15 ms respecto a la línea base sin protección". Este umbral del 80 % por categoría se cumple en Config 2 para las cuatro categorías, medido sobre 100 payloads por categoría (`sqli` 100 %, `xss` 100 %, `path_traversal` 93 %, `cmdi` 100 %).

Como indicador complementario –de interpretabilidad, no de desempeño– se reporta también el análisis SHAP descrito en la sección anterior. La importancia global por clase documenta qué grupos de características sostienen cada F1-score por categoría. Las explicaciones locales documentan casos específicos donde el modelo se comporta de forma distinta a lo esperado (por ejemplo, el efecto del encabezado *User-Agent* sobre la puntuación de Isolation Forest). Esta evidencia complementa las métricas agregadas anteriores sin sustituirlas. Los hallazgos de esta importancia global, incluido el carácter parcialmente resuelto de `cmdi`, se detallan en la sección de Técnicas e instrumentos.

VII.6. Matriz de trazabilidad

El Cuadro 6 articula la correspondencia entre cada objetivo del proyecto, la actividad concreta de la fase de trabajo que lo atiende, el instrumento o técnica empleada, la métrica de aceptación aplicada y el resultado real medido, con su fuente verificable en el repositorio del proyecto.

Objetivo	**Actividad**	**Instrumento/ técnica**	**Métrica**	**Resultado real**
Específico 1 (pipeline de features, 20 jun – 1 ago 2026)	Fase 1: construcción del pipeline de ingeniería de características	Extractor TypeScript + *worker_threads* (regla R1)	Latencia p95 de extracción $\leq$ 5 ms por petición	p95 mixto = 0.0273 ms con *worker_threads* (microbenchmark en aislamiento, mediana de tres corridas; Cuadro 7); Δp95 = 0 ms en benchmark Artillery de middleware completo sobre WSL2, con resolución de 1 ms
Específico 2 — RF (modelo híbrido, cierre 2 ago 2026)	Fase 3: entrenamiento y calibración; Fase 4: verificación de paridad ONNX	Random Forest (`scikit-learn` + ONNX) sobre conjunto de prueba bloqueado (R2)	F1-score $\geq$ 0.80 en al menos 3 de 4 categorías de ataque	F1 macro = 0.9776; por clase: `benign` 0.9984, `cmdi` 0.9239, `path_traversal` 0.9826, `sqli` 0.9956, `xss` 0.9877 — las 5 clases superan el umbral
Específico 2 — IF (modelo híbrido, cierre 2 ago 2026)	Fase 3: entrenamiento y calibración; Fase 4: verificación de paridad ONNX	Isolation Forest (`scikit-learn` + ONNX) sobre conjunto de prueba bloqueado (R2)	Recall de anomalías $\geq$ 50 %, tasa de falsos positivos $\leq$ 10 %	Recall = 0.9125, FP = 0.0546 — ambos criterios cumplidos con margen
Específico 2 — paridad (prerrequisito de integración)	Fase 4: verificación de paridad ONNX	Comparación de salida `scikit-learn` (Python) vs. `onnxruntime-node`	Diferencia de probabilidad entre ambos entornos	$\sim 10^{-7}$; `parity_passed: true` (`parity_report.json`)
Específico 3 — cobertura (2 ago – 30 sep 2026)	Fase 6: evaluación en Config 2 (+logSguarDian)	Middleware integrado sobre app vulnerable en Docker Compose	Detección $\geq$ 80 % de payloads representativos por categoría	Config 2, sobre 100 payloads por categoría: `sqli` 100 %, `xss` 100 %, `path_traversal` 93 %, `cmdi` 100 %; las cuatro categorías superan el umbral (véase la sección VIII.3.2 sobre su solapamiento con el entrenamiento). Ronda 5, sin solapamiento con el entrenamiento, 200 payloads por categoría, par A (tarball e09120…) y par C validado: `sqli` 100 % y 99.5 %, `path_traversal` 85.0 % y 95.0 %, `cmdi` 91.5 % y 96.0 % (secciones VIII.3.5 y VIII.3.6). `xss` no tiene corpus limpio y no se evaluó en la Ronda 5
Específico 3 — latencia (2 ago – 30 sep 2026)	Fase 6: evaluación en Config 2 (+logSguarDian)	Benchmark de latencia (Artillery; WSL2, Linux nativo y Docker+Postgres)	Δp95 $\leq$ 15 ms respecto a la línea base sin protección	Δp95 de 0 ms en WSL2 (resolución de 1 ms) y de 7–9 ms en Docker+Postgres, dentro del límite. En Linux nativo (GitHub Actions, ejecución 36276690236, 2 vCPU), Δp95 de 12.08 ms en normal y 12.85 ms en ataque, dentro del límite, y de 6 495.5 ms en volumen con 7.00 % de timeouts, que no cumple (sección VIII.3.2)

Objetivo	Actividad	Instrumento/ técnica	Métrica	Resultado real
Específico 3 — defensa en profundidad (Config 3)	Fase 6 extendida: evaluación en Config 3a/3b (WAF solo / WAF + logSguarDian) sobre un corpus de 590 payloads de SecLists (solapamiento con el entrenamiento en la sección VIII.3.5)	ModSecurity v3 + OWASP CRS (PL1) como capa perimetral, logSguarDian como segunda capa	Cobertura agregada por configuración; capacidad de logSguarDian de detectar de forma independiente los ataques que evaden el WAF	Config 3a (solo WAF): 550/590 (93.2 %), con brechas reales por categoría (`sqli` 85.7 %, `path_traversal` 87.0 %). Config 3b (capas): 587/590 (99.5 %). De los 40 ataques que evadieron el WAF, 37 fueron capturados de forma independiente por logSguarDian (92.5 % del vacío cerrado)

Cuadro 6: Correspondencia entre objetivos, actividades, instrumentos, métricas y resultados verificados. Elaboración propia.

VII.7. Recursos, participantes y restricciones

En cuanto a recursos, el entrenamiento de los modelos se realizó sobre hardware local (Apple Silicon), sin presupuesto asignado a instancias de cómputo en la nube –condición ya establecida como restricción de recursos en el capítulo de Alcance y verificada como viable dado el tamaño acotado de los modelos de ensamble seleccionados frente a alternativas de mayor capacidad computacional–. El entorno de evaluación integrado (Config 1/2/3) se ejecutó igualmente sobre infraestructura local mediante Docker Compose, sin servicios de terceros ni infraestructura de cómputo remota.

En cuanto a participantes, este documento corresponde al trabajo de graduación individual de Diego Pablo Valenzuela Palacios. Documenta específicamente el componente de aprendizaje automático de logSguarDian: el pipeline de ingeniería de características, el modelo híbrido Random Forest/Isolation Forest, su serialización y verificación de paridad ONNX, y su evaluación integrada como middleware. El alcance exacto queda delimitado en el capítulo de Alcance de este trabajo.

El desarrollo del proyecto logSguarDian en su conjunto contó además con la colaboración de Sebastián Huertas en el componente de ciberseguridad y en la implementación del lado npm de la librería (extractor TypeScript y paquete `core`). Esa colaboración se reconoce aquí por transparencia metodológica. No implica coautoría de este documento ni extiende su alcance evaluado más allá del componente de Machine Learning aquí documentado. El extractor inicial de 72 características y el paso a *worker_threads* los implementó Sebastián Huertas. Diego Valenzuela analizó y justificó cada característica, definió cuáles no existen en inferencia y fijó el contrato de entrada, añadió cuatro características y las correcciones del extractor surgidas del análisis del modelo, y desarrolló el entrenamiento, la serialización ONNX y el pipeline de reentrenamiento.

En cuanto a restricciones, el proyecto se ejecutó dentro de la ventana comprendida entre mayo y octubre de 2026, conforme al calendario del trabajo de graduación de la Facultad de Ingeniería de la Universidad del Valle de Guatemala. Cualquier línea de trabajo no completable dentro de dicho período se documenta explícitamente como trabajo futuro y no como un entregable pendiente sin reconocer. La configuración de evaluación en defensa en profundidad (Config 3) completó su ronda de mayor escala (590 payloads de SecLists, cuyo solapamiento con el entrenamiento se describe en la sección VIII.3.5) dentro de esa ventana. El resultado se reporta en el Cuadro 6. El corpus expuso brechas reales del Web Application Firewall que opera solo (93.2 % de cobertura agregada, con categorías tan bajas como 85.7 %), y logSguarDian, que opera como segunda capa, capturó 37 de los 40 ataques que evadieron esa primera capa.

Esa misma ronda dejó, no obstante, un hallazgo de calibración que motivó el estudio de falsos positivos descrito en las secciones VIII.3.2 y VIII.3.7. Una petición benigna de bajo contenido semántico puede acercarse lo suficiente al umbral de decisión de Random Forest. En ese caso, un metadato incidental de la petición (la cadena de *User-Agent* del cliente HTTP, no el contenido del payload) determina el veredicto. Por ello se realizó un estudio de tasa de falsos positivos contra tráfico legítimo diverso, independiente de esta comparación basada en corpus de ataque.

Ampliar el corpus benigno queda como trabajo futuro, según se indica en el capítulo de Recomendaciones (XI). De igual manera, y como ya se estableció en el capítulo de Alcance, no se contempla dentro del alcance del proyecto un despliegue en un entorno de producción real de terceros como criterio de aceptación del sistema; esta es una restricción

reconocida explícitamente, no un límite que el proyecto haya dejado sin identificar.

Finalmente, las cifras del componente Random Forest/Isolation Forest reportadas en el Cuadro 6 (F1 macro, F1 por clase, recall y tasa de falsos positivos de Isolation Forest) provienen de una corrección de trazabilidad de artefactos descrita en la sección de Recolección, procesamiento y validación de datos, integrada a la rama principal del repositorio del proyecto mediante el *pull request* #79. El análisis SHAP se regeneró posteriormente contra el modelo corregido (*pull request* #80), de modo que las cifras y los hallazgos de interpretabilidad de este capítulo corresponden al mismo artefacto.

CAPÍTULO VIII

Resultados

Este capítulo presenta las mediciones obtenidas al evaluar logSguarDian, organizadas según los tres objetivos específicos del trabajo y la evaluación de interpretabilidad. Cada sección reporta los valores medidos, la condición bajo la cual se midieron y su fuente en el repositorio del proyecto. Las rutas de archivo se expresan relativas a la raíz de dicho repositorio. Cada medición se contrasta con su criterio de aceptación cuando existe uno; la interpretación de los valores se reserva para el capítulo de Discusión de resultados.

VIII.1. Resultados del objetivo específico 1: pipeline de ingeniería de características

El pipeline de ingeniería de características extrae 76 características por petición, de las cuales 69 alimentan al componente Random Forest y 63 al componente Isolation Forest (`training/results/v11_test_results.json`). Esta sección reporta la latencia del extractor en aislamiento y el sobrecosto absoluto del middleware completo en los entornos donde se midió.

VIII.1.1. Latencia del extractor en aislamiento

El Cuadro 7 presenta la latencia de la función `extractFeatureVector()` para cada una de las cinco clases de tráfico y para el tráfico mixto, en dos series: la llamada directa en el hilo principal y el ciclo de ida y vuelta a un *worker_threads*, que es la ruta asíncrona del pipeline. Cada cifra es la mediana de tres corridas independientes del benchmark `benchmarks/extractor.bench.js` del repositorio `logSguarDian` (PR #97, fusionado en `develop`), del 9 de octubre de 2026. Cada corrida descartó 200 iteraciones de calentamiento, midió 2 000 iteraciones por *fixture* de forma serial y tomó el tiempo con alta resolución. El hardware fue un Apple M5 con macOS (Darwin 27.0.0, arm64) y Node.js v25.6.0.

La cifra que se compara con el criterio de 5 ms del objetivo específico 1 es el p95 mixto de la serie con *worker_threads*, de 0.0273 ms. Ambas series son microbenchmarks en aislamiento: no incluyen el resto del middleware, la red ni la carga concurrente.

Nota: una cifra anterior de 0.0082 ms, fechada el 20 de julio de 2026, no registra el hardware ni la versión de Node.js, por lo que no se usa como referencia en este trabajo.

Fixture	Serie	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)
`benign`	Directa (hilo principal)	0.0103	0.0169	0.0409	86 059
`sqli`	Directa (hilo principal)	0.0092	0.0120	0.0340	98 161
`xss`	Directa (hilo principal)	0.0102	0.0135	0.0340	92 380
`path_traversal`	Directa (hilo principal)	0.0076	0.0097	0.0302	122 212
`cmdi`	Directa (hilo principal)	0.0090	0.0102	0.0314	105 878
Mixto (*round-robin*)	Directa (hilo principal)	0.0097	0.0126	0.0350	98 383
`benign`	Ida y vuelta a *worker_threads*	0.0213	0.0304	0.0467	43 960
`sqli`	Ida y vuelta a *worker_threads*	0.0210	0.0275	0.0405	45 459
`xss`	Ida y vuelta a *worker_threads*	0.0226	0.0305	0.0430	42 974
`path_traversal`	Ida y vuelta a *worker_threads*	0.0162	0.0264	0.0377	54 884
`cmdi`	Ida y vuelta a *worker_threads*	0.0212	0.0283	0.0405	45 180
Mixto (*round-robin*)	**Ida y vuelta a *worker_threads***	**0.0206**	**0.0273**	**0.0413**	**46 635**

Cuadro 7: Latencia del extractor de características en aislamiento, por *fixture* y serie de ejecución.

Fuente: `benchmarks/results/extractor-*.json` (tres corridas del 9 de octubre de 2026, commit `7b036bb` del código medido), rama `develop` del repositorio `logSguarDian`; mediana de las tres corridas por *fixture* y serie, a partir de los campos en nanosegundos. Elaboración propia.

VIII.1.2. Sobrecosto absoluto del middleware completo

El Cuadro 8 reúne las mediciones de latencia p95 con y sin el middleware activo en cada entorno donde se midió. Las filas difieren en entorno, escala de datos y compilación del middleware, por lo que cada una declara esas condiciones. Las filas de Docker más Postgres usan la aplicación de referencia (Express y Postgres) con el flujo de navegación benigno de Artillery (`attack-sim/artillery-baseline.yml`, 60 s, 20 usuarios virtuales por segundo con cuatro solicitudes por usuario, es decir, una tasa objetivo de 80 solicitudes por segundo). La fila de Node.js sobre WSL2 usa el benchmark de `packages/core/bench` (Artillery, 50 solicitudes por segundo, 30 s, 3 000 solicitudes por corrida, Node.js v22.5.1, Windows 11 con WSL2, 9 de julio de 2026) con resolución de 1 ms. La fila de microbenchmark mide una función Express sin aplicación detrás.

El Cuadro 8 reporta la latencia p95 con y sin middleware, y su diferencia Δp95 frente al límite de 15 ms fijado en el objetivo específico 3.

Como evidencia complementaria de inferencia, dos mediciones independientes documentan la latencia a nivel de componente, en contextos distintos que no deben compararse entre sí. Durante la instrumentación de la fase de *grace-window* (250 peticiones muestreadas), el tiempo medio de `session.run()` fue de 0.026 ms para Random Forest y 0.978 ms para Isolation Forest, con un tiempo medio de espera en cola (extracción y cola) de 0.038 ms y 0.052 ms, respectivamente (`docs/results.md`, líneas 445–460). Por separado, una ejecución de 60 s bajo carga sostenida midió el tiempo de inferencia por llamada de Isolation Forest en un rango de 2.99 ms a 3.22 ms de media (aproximadamente 2.9–3.2 ms, con p95 de 5.0–6.0 ms) a lo largo de diez segmentos cronológicos consecutivos, sin mostrar un patrón de crecimiento sostenido (`docs/results.md`, líneas 550–560; `docs/config2-latency-evaluation.md`, línea 452).

VIII.2. Resultados del objetivo específico 2: modelo híbrido Random Forest e Isolation Forest

El conjunto de prueba bloqueado (57 481 filas) se leyó tres veces para la generación `rf_v11/if_v10`: el 24 de septiembre de 2026 (commit `3086318`), el 25 de septiembre de 2026 (commit `fc47a5b`) y el 26 de septiembre de 2026 (commit `03de50f`, mediante `training/evaluate_test.py`). Las cifras reportadas en esta sección corresponden a la lectura del 26 de septiembre de

Entorno	Compilación del middleware	p95 sin middleware (ms)	p95 con middleware (ms)	Δp95 absoluto (ms)
Node.js sobre WSL2, Artillery	Versión de julio de 2026 (modo *monitor*)	1	1	0
Microbenchmark sintético (función Express)	Registro asíncrono sin ventana de gracia (PR #53)	0.116	0.300	0.184†
Aplicación real con Postgres, sin Docker	*Worker pool* con ventana de gracia, sin registro asíncrono	3	6	3†
Docker y Postgres, 10 filas	*Worker pool* con ventana de gracia, sin registro asíncrono	5.0	12.8	7.8†
Docker y Postgres, 10 filas	Diseño híbrido (ventana de gracia y cola de escritura por lotes)	≈5	≈12–14	≈7–9
Docker y Postgres, 2 010 filas	Diseño híbrido (ventana de gracia y cola de escritura por lotes)	13.67	23–28	9.33–14.33†

Cuadro 8: Latencia p95 con y sin middleware y diferencia absoluta, por entorno medido.

† Diferencia calculada como p95 con middleware menos p95 sin middleware; la fuente reporta ambos valores y no la diferencia. Las compilaciones con ventana de gracia corresponden a versiones anteriores del middleware; la versión vigente resuelve la respuesta sin esperar al Isolation Forest y actualiza el registro con su puntaje tardío (`packages/core/src/middleware.ts`, rama `develop`).
Fuente: `docs/results.md`, secciones A19 y F6.5; `docs/config2-latency-evaluation.md`, secciones Final latency y Config 2 Final Status. Elaboración propia.

2026. Se evaluaron los modelos `rf_v11` e `if_v10` en su forma ONNX (`rf.onnx` e `if.onnx`, ejecutados con `onnxruntime`) del repositorio de entrenamiento, y no los archivos `.pkl` ni el paquete 0.1.0 en npm. El archivo `training/splits/test.lock.sha256` coincidió con el *hash* registrado antes de ejecutar cualquier inferencia (`7486c258...4fd9`). Todos los valores de esta sección provienen de `training/results/v11_test_results.json`.

VIII.2.1. Random Forest

El Cuadro 9 presenta la precisión, el *recall*, el F1-score y el AUC-ROC de Random Forest por clase. El AUC-ROC se calculó con el método uno contra el resto (`roc_auc_score`, `multi_class='ovr'`). El F1-score macro fue de 0.9776, la exactitud global de 0.9932 y el F1-score ponderado de 0.9933. El AUC-ROC por clase estuvo entre 0.99899 (`path_traversal`) y 0.99998 (`benign`). Las métricas por clase se calculan por argmax de la probabilidad (`training/evaluate_test.py`, salida de etiquetas del modelo ONNX), sin el umbral de confianza de 0.35 con el que el middleware decide el bloqueo (`packages/core/src/middleware.ts`); no se calculó una métrica de prueba con la regla de decisión desplegada.

Clase	Precisión	*Recall*	F1-score	Soporte	AUC-ROC
`benign`	0.9984	0.9985	0.9984	14 886	0.99998
`cmdi`	0.9015	0.9475	0.9239	1 468	0.99901
`path_traversal`	0.9853	0.9798	0.9826	2 529	0.99899
`sqli`	0.9955	0.9956	0.9956	34 107	0.99982
`xss`	0.9950	0.9804	0.9877	4 491	0.99930
Promedio macro	**0.9751**	**0.9804**	**0.9776**	**57 481**	—

Cuadro 9: Métricas de Random Forest (`rf_v11`) por clase sobre el conjunto de prueba bloqueado.
Fuente: `training/results/v11_test_results.json`. Elaboración propia.

VIII.2.2. Isolation Forest

El Cuadro 10 presenta las métricas de Isolation Forest (`if_v10`) sobre el mismo conjunto de prueba. Una petición se considera detectada como anomalía cuando su puntaje queda por debajo del umbral de 0.004205941820353609, congelado en `training/models/if_v10_metadata.json` y no recalibrado contra el conjunto de prueba. El umbral de Isolation Forest pasó de 0.00807 (lectura del 24 de septiembre de 2026, con el modelo previo a ese reentrenamiento) a 0.00421 (lecturas del 25 y 26 de septiembre de 2026) tras reentrenar y recalibrar en validación. Las clases de ataque (42 595 filas) se tratan como positivas y la clase `benign` (14 886 filas) como negativa. La matriz de confusión resultante fue: 38 868 verdaderos positivos, 3 727 falsos negativos, 813 falsos positivos y 14 073 verdaderos negativos.

Métrica	**Valor**
Recall global	0.9125
Tasa de falsos positivos	0.0546
Precisión	0.9795
Recall en `cmdi`	0.9564
Recall en `path_traversal`	0.9968
Recall en `sqli`	0.8954
Recall en `xss`	0.9802

Cuadro 10: Métricas de Isolation Forest (`if_v10`) sobre el conjunto de prueba bloqueado.

Fuente: `training/results/v11_test_results.json`. Elaboración propia.

VIII.2.3. Paridad entre scikit-learn y ONNX

El Cuadro 11 presenta la diferencia máxima observada entre la salida de los modelos en scikit-learn y la de sus versiones ONNX, calculada sobre 1 000 muestras con *opset* 17.

Modelo	**Magnitud comparada**	**Diferencia máxima**
Random Forest (`rf_v11`)	Probabilidad por clase	1.01×10^{-7}
Isolation Forest (`if_v10`)	Puntaje de anomalía	2.42×10^{-7}

Cuadro 11: Diferencia máxima de salida entre scikit-learn y ONNX sobre 1 000 muestras. El reporte registra `parity_passed: true`.

Fuente: `training/models/parity_report.json`. Elaboración propia.

VIII.3. Resultados del objetivo específico 3: evaluación integrada en las configuraciones 1, 2, 3a y 3b

La evaluación integrada se ejecutó sobre una aplicación Express y Postgres deliberadamente vulnerable (`logSguarDian-vulnerable-project`), en cuatro configuraciones: Config 1, sin protección; Config 2, solo logSguarDian; Config 3a, solo *Web Application Firewall* (ModSecurity v3 con OWASP CRS en nivel de paranoia 1 y modo de bloqueo); y Config 3b, ModSecurity más logSguarDian en capas. En las configuraciones con logSguarDian, los modelos desplegados fueron `rf_v10` e `if_v9`, con `RF_THRESHOLD` de 0.35. La evaluación de Config 1, 2, 3a y 3b (rondas de julio-agosto de 2026) se ejecutó sobre la generación de modelos vigente en ese momento, `rf_v10`/`if_v9`. Las métricas de Random Forest e Isolation Forest reportadas en las secciones anteriores de este ca-

pítulo corresponden a la generación posterior, `rf_v11/if_v10`, resultado del reentrenamiento del 30 de agosto de 2026 (PR #66), cuyas métricas de prueba se re-derivaron en el PR #79.

Las Configuraciones 2 y 3b se evaluaron con la generación vigente en ese momento, `rf_v10/if_v9`. La Configuración 1 y la Configuración 3a no usan los modelos de logSguarDian. La Configuración 2 se repitió después con `rf_v11/if_v10`, primera versión del 30 de agosto de 2026, como se reporta en las secciones VIII.3.2 y VIII.3.5, y con el par validado, como se reporta en la sección VIII.3.6; la Configuración 3b no se repitió, salvo la ejecución provisional del coautor (anexo XIII.8). Las rutas de archivo de esta sección se expresan relativas a `docs/` del repositorio del proyecto.

VIII.3.1. Config 1: aplicación sin protección

El Cuadro 12 resume los vectores de ataque ejecutados contra la aplicación sin ningún componente de seguridad, con la prueba de concepto de `curl` de cada vector y con la respuesta real que se capturó.

Categoría	**Vectores probados**	**Vectores comprometidos**
Inyección SQL	Evasión de autenticación; exfiltración con `UNION`	2/2
XSS almacenado	Campo de biografía; contenido de publicación	2/2
Path traversal	Descarga de adjuntos	1/1
Inyección de comandos	Utilidad de administración `ping`	1/1
Total		**6/6**

Cuadro 12: Vectores de ataque comprometidos en Config 1, sin protección.

Fuente: `baseline-config1.md`, sección 5 (ejecutado el 24 de julio de 2026). Elaboración propia.

En el vector de inyección de comandos, `whoami` devolvió `root`.

VIII.3.2. Config 2: solo logSguarDian

El Cuadro 13 presenta la cobertura por categoría de Config 2 sobre 100 payloads por categoría (`attack-sim/payloads/*.json`), disparados contra el *endpoint* que corresponde a cada vulnerabilidad con el middleware activo en modo `block`. Se contabilizó como bloqueada la respuesta HTTP 403 del middleware.

Los 100 payloads por categoría de esta medición no son disjuntos del entrenamiento. Con la misma normalización de la sección VIII.3.5, coinciden con alguna partición, por igualdad normalizada, 2 de 100 en `sqli`, 14 en `xss`, 4 en `path_traversal` y ninguno en `cmdi`, y por subcadena normalizada, 15, 33, 29 y 8, respectivamente. Algunas filas de las particiones no conservan el texto crudo, por lo que los conteos son cotas inferiores. El repositorio conserva solo los valores agregados de esta medición, documentados el 6 de agosto de 2026, y no registra los resultados por payload ni el hash de los modelos, por lo que la cobertura no se recalculó sin las coincidencias.

Categoría	**Bloqueados**	**Total**	**Cobertura**
`sqli`	100	100	100.0 %
`xss`	100	100	100.0 %
`path_traversal`	93	100	93.0 %
`cmdi`	100	100	100.0 %

Cuadro 13: Cobertura de detección por categoría en Config 2, 100 payloads por categoría.

Fuente: `docs/config2-latency-evaluation.md` (secciones *Final coverage* y *Config 2 Final Status*) del repositorio `logSguarDian-vulnerable-project` (commit `944feb1`); valores agregados. Elaboración propia.

El recuento de eventos de bloqueo en la base de detección de logSguarDian (393) coincide con el total de payloads bloqueados del Cuadro 13, que el criterio de la sección anterior define como la respuesta HTTP 403 del middleware. En los escenarios normal y de volumen, las peticiones benignas de Artillery no recibieron ninguna respuesta 403 en las cinco repeticiones de cada condición (corrida de latencia de Linux, run 36276690236, `docs/evidence/latency-linux-2026-09-26.tgz` del repositorio `logSguarDian-vulnerable-project`, rama `main`). Cada repetición del escenario normal envió 6 300 solicitudes, incluido el calentamiento, de las cuales 4 725 respondieron 200 y 1 575 respondieron 302. En el escenario de volumen con middleware activo, 1 824 solicitudes en total agotaron el tiempo de espera entre las cinco repeticiones; un tiempo de espera no es una respuesta 403 y se reporta por separado.

Este resultado vale para el User-Agent por defecto de Artillery, `Artillery (https://artillery.io)`, una cadena corta que no representa a un navegador. Con cadenas de User-Agent de navegadores reales, el estudio `attack-sim/ua_fp_study.py` bloqueó 13 de 15 combinaciones de agente y ruta; solo Firefox en macOS evitó el bloqueo en `/login` y `/posts`.

Ese estudio corresponde al commit del 24 de septiembre de 2026, y su archivo de resultados no registra el modelo cargado. Por el contenido de `models/` en ese commit, el modelo es el par `rf_v10/if_v9`, mientras que la corrida de latencia de Linux usó el par `rf_v11/if_v10`, primera versión del 30 de agosto de 2026 (`a7c015f5.../3cf8e45c...`). Los dos resultados corresponden a modelos distintos y no se comparan directamente. En la condición de ataque no es posible aislar la tasa de 403 de las peticiones benignas, porque el informe agrega las respuestas por *endpoint* y `/profile` recibe tanto peticiones benignas como el ataque de XSS.

El Cuadro 14 reporta el incremento relativo del p95, definido como Δp95 expresado como porcentaje de la línea base sin middleware. Es un indicador complementario. El criterio vigente del objetivo específico 3 es el Δp95 absoluto de 15 ms del Cuadro 8. Cada medición usa el flujo benigno de Artillery (`attack-sim/artillery-baseline.yml`, 60 s, 20 usuarios virtuales por segundo con cuatro solicitudes por usuario) sobre Docker y Postgres, y las filas difieren en la compilación del middleware y en la escala de datos. Esas mediciones se documentaron entre el 31 de julio y el 6 de agosto de 2026, cuando el archivo tenía esa configuración; el cambio a 25 usuarios virtuales por segundo es del 24 de septiembre de 2026.

Compilación del middleware	**Datos sembrados**	**p95 sin middleware (ms)**	**p95 con middleware (ms)**	**Δp95 relativo**
Un solo *worker* (`rf_v9/if_v8`)	10 filas	4.0	14.6	+265.8 %
Worker pool (`rf_v10/if_v9`), medición inicial	10 filas	5.0	10.6	+111.3 %
Worker pool (`rf_v10/if_v9`), umbral de IF corregido (6 corridas)	10 filas	5.0	12.8	+156.7 %
Diseño híbrido (ventana de gracia y cola de escritura por lotes)	2 010 filas	13.67	23–28	+70.2 %

Cuadro 14: Incremento relativo del p95 de Config 2 (indicador complementario), por compilación del middleware. Las generaciones medidas son `rf_v9/if_v8` y `rf_v10/if_v9`; no corresponden a `rf_v11/if_v10`.

Fuente: `docs/config2-latency-evaluation.md`, secciones *Latency Results*, *Comparison across iterations*, *Final latency*, *Definitive comparison* y *Config 2 Final Status*. Elaboración propia.

Las tres mediciones con 10 filas sembradas arrojaron un incremento relativo del p95 de +111.3 % a +265.8 %; la medición con 2 010 filas arrojó +70.2 %.

Experimento independiente de latencia en Linux nativo. El flujo manual `Latency Benchmark` del repositorio `logSguarDian-vulnerable-project` (ejecución 36276690236, commit `f15a9e8`, 26 de septiembre de 2026) se ejecutó en un runner `ubuntu-latest` con 2 vCPU y 7.8 GiB de memoria, con contenedores Docker sobre Linux y sin Docker Desktop/macOS. El Δp95 relativo fue de +485.1 % en el escenario normal, +278.1 % en el de ataque y +10 174.6 % en el de volumen

con 2 010 filas sembradas. En términos absolutos, la diferencia entre el p95 con y sin middleware fue de 12.08 ms en el escenario normal (de 2.49 a 14.57 ms), 12.85 ms en el de ataque (de 4.62 a 17.47 ms) y 6 495.46 ms en el de volumen (de 63.84 a 6 559.30 ms). Frente al límite de 15 ms del objetivo específico 3, los escenarios normal y de ataque quedan por debajo y el de volumen lo supera ampliamente.

Este experimento difiere de las mediciones anteriores en entorno, carga ofrecida, arnés de medición y procedencia de los artefactos. Se reporta por separado: no reemplaza ni refuta las cifras de +111.3 % a +265.8 % y +70.2 %, ni se promedia con ellas.

Cada corrida descarta 15 s de calentamiento y mide 60 s, con cinco repeticiones por condición. La tasa medida fue de 25 usuarios virtuales por segundo en los escenarios normal y de volumen, y de 30 en el de ataque, mediante `-overrides` en `attack-sim/run_latency_suite.sh`. El p95 de cada corrida es una aproximación calculada como promedio ponderado por conteo de los p95 de los intervalos de Artillery posteriores al calentamiento (`attack-sim/parse-latency-report.js`). Cada condición resume sus cinco repeticiones con la mediana de tres valores, tras descartar el mínimo y el máximo; no corresponde a un percentil exacto calculado sobre el conjunto agrupado de muestras.

En el escenario de volumen con middleware, 1 824 de 26 048 solicitudes (7.00 %, sumadas las cinco repeticiones) terminaron con `ERR_SOCKET_TIMEOUT`: 1 804 en `/login` y 20 en `/posts`. La línea base de volumen no registró errores. Las solicitudes con timeout no tienen tiempo de respuesta registrado y no entran en el p95.

Los modelos de esta ejecución son `rf.onnx` (SHA-256 `a7c015f5...`) e `if.onnx` (`3cf8e45c...`), que se encuentran en el directorio `models/` de la aplicación vulnerable (commit `f15a9e8`) y no forman parte del paquete 0.1.0 en npm (`logsguardian@0.1.0`, par A), que contiene `rf_v10/if_v9`. Corresponden al build `rf_v11/if_v10` del 30 de agosto de 2026 (commit `734c24f` del repositorio logSguarDian), el mismo que usaron las ejecuciones de Config 2 del 9 de septiembre de 2026. No son el par canónico posterior al PR #79 (`rf.onnx` `25b407e6...`, `if.onnx` `5be28ee8...`), que es el que declara el campo `artifact_checksums` de `training/results/v11_test_results.json` y con el que se calcularon las métricas de la sección del objetivo específico 2. La ejecución de Linux no se repitió con ese par. Fuente: `cybersecurity-objectives-compliance.md`, sección OE3.2 y sus notas metodológica y de procedencia del modelo, en el repositorio logSguarDian.

VIII.3.3. Config 3a y Config 3b: corpus pequeño

En las Rondas 1 y 2, ejecutadas con el *stack* en capas y ModSecurity en modo de bloqueo, el corpus fue de 23 payloads: los 9 casos de ataque de las Configs 1 y 2 y 14 variantes diseñadas para evadir las reglas de firmas de CRS. ModSecurity respondió 403 a los 9 casos originales y a las 14 variantes (23/23), y ninguna petición llegó a la aplicación (`docs/config3b-results.md`, secciones *Round 1* y *Round 2*).

En la Ronda 3, con `MODSEC_RULE_ENGINE` en `DetectionOnly`, de modo que CRS registra pero no bloquea y toda petición llega a la aplicación, los 23 payloads se reejecutaron contra logSguarDian. La auditoría de CRS registró que habría bloqueado los 23. El resultado inicial fue de 16/23 bloqueados por logSguarDian (la cifra original del documento fuente, 21/23, resultó aritméticamente inconsistente con el detalle de 7 casos no detectados que el propio documento reporta; se corrigió a 16/23, consistente con 16+7=23, sin re-medición porque los datos crudos de esa ronda ya no existen); los 7 no detectados eran de XSS y se habían enviado como `multipart/form-data`. Tras montar un *parser* multipart global antes del middleware, 23/23 devolvieron `verdict: block`, confirmado en `detection_events`. Dos payloads (traversal de rutas y una inyección de comandos) se bloquearon con una clase asignada distinta de la esperada (`config3b-results.md`, sección *Round*

3).

VIII.3.4. Config 3a y Config 3b: corpus grande

La Ronda 4 ejecutó las cuatro configuraciones contra 590 payloads de SecLists (rama `master`), deduplicados y muestreados aleatoriamente con semilla 42 hasta un máximo de 200 por categoría: 77 de `sqli`, 113 de `xss`, 200 de `path_traversal` y 200 de `cmdi`. El Cuadro 15 presenta los payloads bloqueados por configuración y categoría.

Categoría	Config 1 (sin protección)	Config 2 (solo logSguarDian)	Config 3a (solo WAF, PL1)	Config 3b (en capas)
`sqli` (77)	0/77 (0.0 %)	76/77 (98.7 %)	66/77 (85.7 %)	77/77 (100.0 %)
`xss` (113)	0/113 (0.0 %)	110/113 (97.3 %)	110/113 (97.3 %)	113/113 (100.0 %)
`path_traversal` (200)	0/200 (0.0 %)	197/200 (98.5 %)	174/200 (87.0 %)	197/200 (98.5 %)
`cmdi` (200)	0/200 (0.0 %)	200/200 (100.0 %)	200/200 (100.0 %)	200/200 (100.0 %)
Total (590)	**0/590 (0.0 %)**	**583/590 (98.8 %)**	**550/590 (93.2 %)**	**587/590 (99.5 %)**

Cuadro 15: Payloads bloqueados por configuración y categoría sobre el corpus de 590 payloads de SecLists. Fuente: `docs/config3b-results.md`, sección *Round 4*. Elaboración propia.

De los 40 ataques que CRS en nivel de paranoia 1 no bloqueó en Config 3a (590 menos 550), la atribución de los bloqueos de Config 3b por firma de respuesta (el encabezado `X-Powered-By: Express` aparece solo en bloqueos de la capa de aplicación) registró 37 capturados por logSguarDian tras pasar el WAF. Tres payloads pasaron ambas capas: las cadenas `~/.gtkrc`, `/proc/file systems` y `~/.atfp_history`, de la categoría `path_traversal`.

VIII.3.5. Solapamiento del corpus de 590 payloads con el entrenamiento

Los 590 payloads de la evaluación integral provienen de SecLists. Los 200 de inyección de comandos son las líneas del archivo `command-injection-commix.txt`, del que también provienen las 2 455 filas de la fuente `seclists_cmdi` (1 719 en entrenamiento, 368 en validación y 368 en prueba). Con igualdad de la cadena normalizada (dígitos sustituidos por `N` y marcadores de seis letras mayúsculas por `CANARY` en `cmdi`; decodificación URL, minúsculas y dígitos sustituidos en las demás categorías), los 200 payloads de `cmdi` coinciden con alguna partición. Por texto literal, 63 de ellos aparecen en alguna partición: 46 en entrenamiento, 8 en validación y 9 en prueba, sin que ninguno aparezca en dos particiones.

En `xss`, 7 de 113 payloads coinciden por igualdad normalizada; en `sqli`, ninguno de 77; en `path_traversal`, ninguno de 200. Al excluir las coincidencias por igualdad normalizada, la cobertura de logSguarDian en Config 2 fue de 98.7 % en `sqli`, 97.2 % en `xss` y 98.5 % en `path_traversal`, frente a 98.7 %, 97.3 % y 98.5 % con el corpus completo. La coincidencia por subcadena produce 26 payloads de `sqli`, 13 de `xss` y los 200 de `cmdi`, y constituye un límite superior, no una medida de solapamiento. En `cmdi` todo el corpus coincide, por lo que la cobertura de 100 % en esa categoría no constituye evidencia de generalización a payloads de otra procedencia.

Config 2 se ejecutó también sobre los mismos 590 payloads con dos generaciones de modelo; el cuadro anterior corresponde a la primera. Con `rf_v10/if_v9` (`attack-sim/results_config2.json`, commit `af61eba` del repositorio `logSguarDian-vulnerable-project`) se bloquearon 583 de 590 payloads. Con `rf_v11/if_v10` (build del 30 de agosto de 2026, `attack-sim/results_config2-rfv11.json`, commit `14c0e15`) también se bloquearon 583 de 590.

Al comparar por identificador, 10 payloads cambiaron de estado.

Por categoría, `sqli` pasó de 76 a 77 bloqueados (`sqli_31` pasó a bloquearse), `xss` de 110 a 111 (`xss_12`), `path_traversal` se mantuvo en 197 con tres payloads que dejaron de bloquearse

(path_traversal_13, _127 y _186) y tres que empezaron a bloquearse (_6, _36 y _69), y cmdi bajó de 200 a 198 (cmdi_88 y cmdi_118 dejaron de bloquearse).

El archivo results_config2-rfv11.json no registra el modelo utilizado; su atribución al build del 30 de agosto de 2026 (rf.onnx a7c015f5..., if.onnx 3cf8e45c...) se apoya en el mensaje del commit 14c0e15, que describe el reemplazo de models/rf.onnx e models/if.onnx por el build rf_v11/if_v10 antes de esa ejecución. El archivo results_config2.json (590 payloads) corresponde a rf_v10/if_v9: los blobs de models/rf.onnx e models/if.onnx en af61eba tienen los hashes 18083725... y 5de374a7... de ese par, y el informe de paridad training/models/parity_report.json del commit dc2aaa65 (repositorio logSguarDian) declara rf_v10/if_v9.

VIII.3.6. Evaluación sobre un corpus sin solapamiento (Ronda 5)

Para acotar el solapamiento descrito en la subsección anterior se construyó un corpus de 608 solicitudes. Contiene 200 payloads de inyección SQL generados con sqlmap, 200 de path traversal y 200 de inyección de comandos (cmdi). Estos dos últimos conjuntos se produjeron con generadores propios deterministas, con semilla 20260923, no derivados de SecLists. El corpus incluye además 8 marcadores de xss escritos a mano; no entran en la cobertura, por lo que esta categoría no se evaluó en esta ronda.

Antes de incluir cada candidato se descartó toda coincidencia exacta del campo completo con el conjunto de entrenamiento. El filtro descartó 3 de 400 candidatos de path traversal, todos por coincidencia con owasp_logs.jsonl, y ninguno de los 1 995 candidatos de sqli ni de los 400 de cmdi (attack-sim/exclusion_report.json). El solapamiento con las particiones se midió con training/leakage_check.py del repositorio logSguarDian (commit 22550ea), ejecutado en solo lectura contra train, val y test. Se usan dos criterios.

El primero, (a), es la igualdad normalizada del campo completo: un payload coincide si su forma normalizada es igual a la de algún campo path, query o body de la partición. El segundo, (b), es la regla de subcadena de round4_vs_split: un payload coincide si aparece como subcadena de algún campo, con y sin normalización. La regla (b) es permisiva, por lo que sus cifras son un límite superior y no una medida de solapamiento.

El Cuadro 16 presenta el número de payloads de cada categoría que coinciden con cada partición, por criterio.

Categoría	Criterio	Train	Val	Test	Unión
sqli	(a) igualdad normalizada	4	1	2	4
sqli	(b) subcadena normalizada	4	1	2	4
sqli	(b) subcadena literal	0	0	0	0
path_traversal	(a) igualdad normalizada	0	0	0	0
path_traversal	(b) subcadena normalizada	6	5	5	7
path_traversal	(b) subcadena literal	5	2	5	5
cmdi	(a) igualdad normalizada	0	0	0	0
cmdi	(b) subcadena normalizada	1	1	1	2
cmdi	(b) subcadena literal	0	0	0	0

Cuadro 16: Payloads de 200 por categoría que coinciden con cada partición, según el criterio de solapamiento. Unión: payloads que coinciden con al menos una partición.

Fuente: training/leakage_check.py (commit 22550ea) sobre training/splits/ del repositorio logSguarDian, y attack-sim/large_corpus_requests.json. Elaboración propia.

Con el criterio (a), 4 de los 200 payloads de sqli coinciden con alguna partición y ninguno de path traversal ni de cmdi. Con el criterio (b), path traversal alcanza 7 coincidencias en la unión y

cmdi 2; por ser un límite superior, estas cifras no deben leerse como solapamiento confirmado.

El modelo validado tiene una cadena de procedencia verificable. El tarball de validación, que no se publicó en el registro npm, se construyó desde el commit 22550ea de `develop` del repositorio `logSguarDian`, con SHA-256 `cf1238deb33383975e12058de71c4423b22d7c5bcba5cf47241c0d0f712b0416`. Sus modelos, par C, son `rf.onnx` (`25b407e6...`) e `if.onnx` (`5be28ee8...`), y las sumas de verificación se comprobaron dentro del contenedor.

El commit 4d928b5 del repositorio `logSguarDian-vulnerable-project` registra la construcción. El 9 de octubre de 2026 se repitió Config 2 con ese tarball (`attack-sim/results_config2-develop-tarball-20261009.json`), con los mismos 200 payloads por categoría. El archivo de resultados no registra el modelo; esa atribución consta en `docs/config3b-results.md` y en dicho commit.

El Cuadro 17 presenta la cobertura de Config 2 sobre este corpus con ambos pares de modelos.

Categoría	`rf_v10/if_v9`**, par A (tarball e09120...)**	`rf_v11/if_v10` **(validado)**
`sqli`	200/200 (100.0 %)	199/200 (99.5 %)
`path_traversal`	170/200 (85.0 %)	190/200 (95.0 %)
`cmdi`	183/200 (91.5 %)	192/200 (96.0 %)

Cuadro 17: Cobertura de Config 2 sobre el corpus de la Ronda 5, con 200 solicitudes por categoría, por par de modelos.

Fuente: `attack-sim/results_config2.json` en el commit a3aada0 y `attack-sim/results_config2-develop-tarball-20261009.json`, del repositorio `logSguarDian-vulnerable-project`. Elaboración propia.

Con `rf_v10/if_v9`, los intervalos de Wilson al 95 % son 98.1–100.0 % en sqli, 79.4–89.3 % en path traversal y 86.8–94.6 % en cmdi. Con `rf_v11/if_v10` son 97.2–99.9 %, 91.0–97.3 % y 92.3–98.0 %, en el mismo orden. Las tres categorías superan el 80 % del objetivo específico 3 con ambos pares de modelos, aunque el límite inferior del intervalo de path traversal con `rf_v10/if_v9` queda por debajo de ese umbral.

Al comparar por identificador, 26 payloads de path traversal pasaron de no bloqueados a bloqueados y 6 en sentido inverso; en cmdi, 16 pasaron a bloqueados y 7 en sentido inverso; en sqli, ninguno pasó a bloqueado y 1 en sentido inverso. En la corrida del par A (tarball e09120...), `cmdi_94` terminó por *timeout*; en la corrida validada, `sqli_75` devolvió HTTP 500. Ambas entradas se contaron como no bloqueadas.

Esta evaluación tiene limitaciones que acotan su alcance. Se realizó una sola corrida, con 200 solicitudes por categoría. Los corpus de path traversal y cmdi son sintéticos, y la categoría `xss` no tiene un corpus limpio, por lo que su cobertura permanece en el corpus de 590 payloads. A la fecha de redacción de este capítulo (10 de octubre de 2026), las configuraciones 3a y 3b no se repitieron en este trabajo; el coautor las ejecutó de forma provisional (anexo XIII.8), de modo que la comparación con el WAF de este capítulo permanece en el corpus de 590 payloads. El arnés de esta ronda codifica cada payload con `quote_plus`, por lo que los que contienen secuencias `%xx` llegan doble codificados; los 30 fallos de `path_traversal` del par A son respuestas 404 de la aplicación (anexo XIII.8).

Los resultados de la Ronda 5 con `rf_v10/if_v9` se conocieron el 24 de septiembre de 2026, y el modelo `rf_v11/if_v10` se reconstruyó el 26 de septiembre de 2026. El historial no vincula ambos hechos, por lo que esta no es una evaluación estrictamente ciega del modelo reconstruido. Por último, este capítulo no evalúa la causa de la diferencia de path traversal entre ambas corridas (de 85.0 % a 95.0 %).

VIII.3.7. Falsos positivos en aplicaciones de terceros

En la aplicación vulnerable del proyecto, con el User-Agent por defecto de Artillery, ninguna petición benigna de los escenarios normal y de volumen recibió una respuesta 403 (cifras en la sección de resultados de Config 2, obtenidas con el par `rf_v11`/`if_v10`, primera versión del 30 de agosto de 2026, `a7c015f5...`/`3cf8e45c...`); el resultado cambia con User-Agent de navegador, como se describe a continuación. En aplicaciones de terceros la proporción de peticiones benignas bloqueadas fue alta. En OWASP Juice Shop (modo `monitor`, solo peticiones benignas), 303 de 312 peticiones con User-Agent de navegador (97.1 %) y 123 de 312 con `curl` (39.4 %) recibieron el veredicto de bloqueo.

Con un User-Agent de navegador se bloquearon 37 de 40 peticiones benignas en dvws-node (92.5 %) y 5 de 5 en DVNA, medición preliminar.

En node-api-goat se bloquearon 50 de 50 valores enviados a una sola ruta, todos con el User-Agent de Chrome 128 (`evidence/node-api-goat/run-attacks.js`); el chequeo de cinco peticiones con ese mismo User-Agent dio 4 de 5. En dvws-node, el User-Agent por defecto de Node bloqueó 30 de 40 (75 %).

La medición en Juice Shop usó la copia local de Juice Shop (par C), no el paquete 0.1.0 en npm. Esa copia contiene `rf_v11` e `if_v10` (SHA-256 `25b407e6...` y `5be28ee8...`), los mismos que el repositorio de entrenamiento conserva en `training/models/`. Para dvna, node-api-goat y dvws-node, el paquete y el modelo exactos de cada medición no constan en el repositorio.

VIII.3.8. Consumo de memoria y CPU

La memoria y la CPU se midieron con `docker stats` sobre el contenedor de la aplicación, no sobre un proceso. La CPU se expresa como porcentaje del contenedor, de modo que 100 % equivale a un núcleo. La medición corresponde a la misma ejecución del experimento de latencia en Linux nativo (ejecución 36276690236), en un runner `ubuntu-latest` con 2 vCPU y 7.8 GiB de memoria, con cinco repeticiones por condición. La ventana A, primaria, excluye los primeros 15 s de cada repetición, que corresponden al calentamiento a baja carga; la ventana B (repetición completa) sí lo incluye.

Escenario	Memoria baseline / active (MiB)	ΔMemoria (MiB)	CPU baseline / active (% de un núcleo)	ΔCPU (pp / relativo)
Ataque	63.2 / 394.8	331.9	16.1 / 101.3	85.1 pp / 529.6 %
Normal	65.2 / 397.5	332.0	14.5 / 103.3	88.9 pp / 614.4 %
Volumen	83.3 / 459.1	375.9	82.3 / 115.3	33.0 pp / 40.1 %

Cuadro 18: Memoria y CPU del contenedor con y sin middleware, por escenario (ventana A).

Fuente: `docs/evidence/resources_summary.json`, rama `docs/latency-linux-evidence`, ventana A. Medias por condición; ΔMemoria y ΔCPU son la media de las diferencias pareadas por repetición. Elaboración propia.

El Cuadro 18 usa la ventana A, que descarta los primeros 15 s de cada repetición. La ventana B, que conserva la repetición completa con el arranque, da una ΔCPU relativa mayor en los tres escenarios (541.3 %, 668.9 % y 57.5 %). Ambas ventanas llevan a la misma conclusión: ningún escenario cumple el criterio de CPU.

El criterio de memoria del protocolo es de 150 MB de memoria adicional, equivalentes a unos 143 MiB. Se aplica al incremento ΔMemoria que añade el middleware, no a la memoria absoluta del contenedor. Los incrementos medios fueron de 331.9 MiB en ataque, 332.0 MiB en normal y 375.9 MiB en volumen.

Los tres superan el límite, y cada uno es más del doble de los 143 MiB, por lo que el criterio no se cumple en este experimento. El repositorio del proyecto revisó internamente este límite a 300

MB; esta tesis mantiene el criterio del protocolo. Los tres incrementos (331.9, 332.0 y 375.9 MiB, es decir, 348.0, 348.1 y 394.2 MB) superan también ese valor revisado. El incremento es casi el mismo en los escenarios normal y de ataque (331.9 y 332.0 MiB).

El límite se fijó antes de medir la memoria nativa de `onnxruntime-node` y se pensó en instancias de 1 GB (Droplet). El contenedor completo con el middleware activo ocupó entre 394.8 y 459.1 MiB (ataque, normal y volumen: 394.8, 397.5 y 459.1 MiB), y no se probó en una instancia de 1 GB, por lo que el cumplimiento en ese entorno no se verificó. El criterio de CPU aprobado es un ΔCPU relativo de hasta 15 % bajo carga normal y de ataque. Ambos escenarios lo superan (529.6 % y 614.4 %), y el escenario de volumen también (40.1 %).

Estas cifras corresponden a una sola corrida (ejecución 36276690236, commit medido f15a9e8) en un runner de 2 vCPU, con cinco repeticiones por condición. Se midió el contenedor completo con `docker stats`, sin separar sus procesos, por lo que el costo no se atribuye a ningún componente. El archivo de datos tiene SHA-256 `b96d9014...2290187cec`.

La memoria con el middleware activo es prácticamente la misma en los escenarios normal y de ataque, y se mantiene en un rango similar entre repeticiones. La corrida de monitoreo continuo de 41.9 minutos (374 muestras) registró un pico de 494.8 MiB, medido sobre el contenedor.

No se midió el costo por componente. La configuración crea tres `worker_threads`: dos workers de Isolation Forest (`IF_POOL_SIZE` = 2) y uno de Random Forest. Las cifras de este experimento corresponden al conjunto del middleware y no pueden atribuirse a ninguno de sus componentes.

Estas cifras no son comparables con el RSS de 245.11 MB del benchmark aislado de `logSguarDian` (`benchmarks/onnx-memory-pool.bench.js`). Ese benchmark no incluye carga HTTP ni contenedor y no tiene línea base sin middleware.

VIII.4. Resultados de interpretabilidad: análisis SHAP

El análisis de interpretabilidad se calculó con `shap.TreeExplainer` sobre el modelo `rf_v11` (`training/models/rf_v11.pkl`) y la partición de validación (`training/splits/val.parquet`), con las características de entrada del modelo seleccionadas por nombre. El *script* es `training/results/shap_analysis.py`, y los reportes y figuras de esta sección están en `training/results/`. La importancia global de una característica para una clase es la media del valor absoluto de su valor SHAP sobre la partición de validación, para la salida de esa clase.

VIII.4.1. Importancia global por clase

El Cuadro 19 presenta, para cada una de las cinco clases, las tres características de mayor importancia global y la posición (entre las diez primeras) de `ua_length` y `ua_present`. Las Figuras 1 a 5 muestran las diez características de mayor importancia de cada clase. La Figura 2 muestra la importancia SHAP global de la clase `sqli`. La Figura 3 muestra la de la clase `xss`. La Figura 4 muestra la de la clase `path_traversal`.

En la lista de diez características de `cmdi` no figuran `shell_command_count` ni `pipe_count`.

Las características del User-Agent figuran entre las de mayor contribución. `ua_length` ocupa el primer lugar en `benign` (0.0866) y en `sqli` (0.0734), el segundo en `cmdi` y el tercero en `path_traversal`; `ua_present` ocupa el segundo lugar en `benign` y el tercero en `sqli`. En el caso local de `sqli` del Cuadro 20, `ua_length` aporta +0.1136, la mayor contribución individual. Solo el 0.4 % de las filas benignas tiene un User-Agent largo (más de 40 caracteres), frente a entre 33.3 % (`xss`) y 97.7 % (`path_traversal`) en las clases de ataque (anexo XIII.4).

Clase	Tres características de mayor importancia (media |SHAP|)	Posición de `ua_length`	Posición de `ua_present`
`benign`	`ua_length` (0.08659); `ua_present` (0.05109); `path_depth` (0.03777)	1	2
`sqli`	`ua_length` (0.07342); `sqli_keyword_density` (0.05159); `ua_present` (0.03683)	1	3
`xss`	`alert_function_present` (0.03458); `xss_marker_density` (0.0274); `html_tag_count` (0.02689)	5	8
`path_traversal`	`traversal_sequence_count` (0.05172); `special_char_ratio` (0.03731); `ua_length` (0.03207)	3	4
`cmdi`	`method_is_get` (0.02554); `ua_length` (0.02122); `subshell_count` (0.01584)	2	9

Cuadro 19: Características de mayor importancia SHAP global por clase de `rf_v11` y posición de las características de *User-Agent*.

Fuente: `training/results/shap_global_report.json`. Elaboración propia.

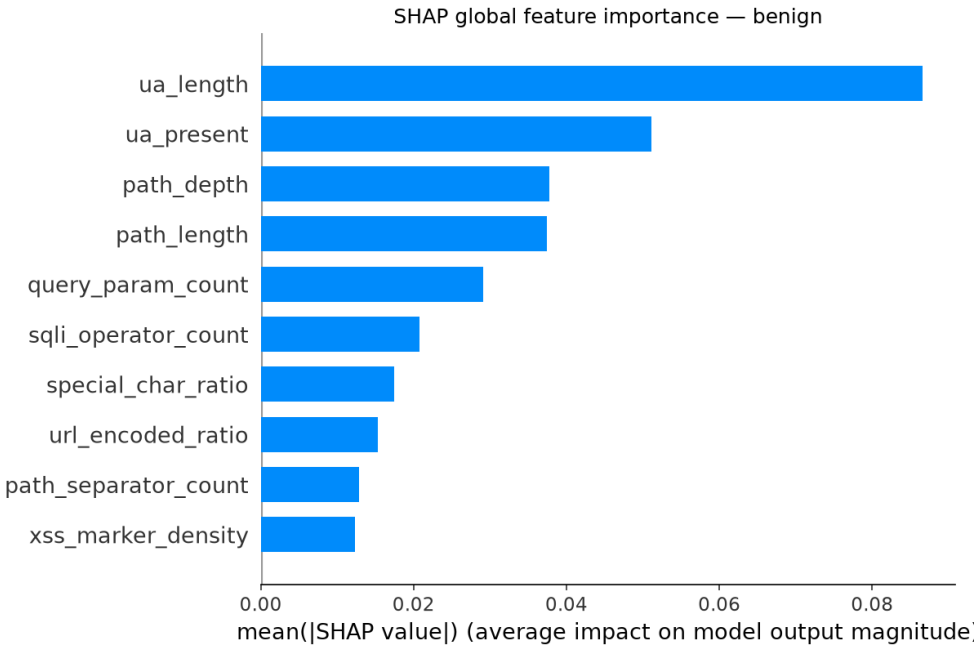


Figura 1: Importancia SHAP global, clase `benign`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

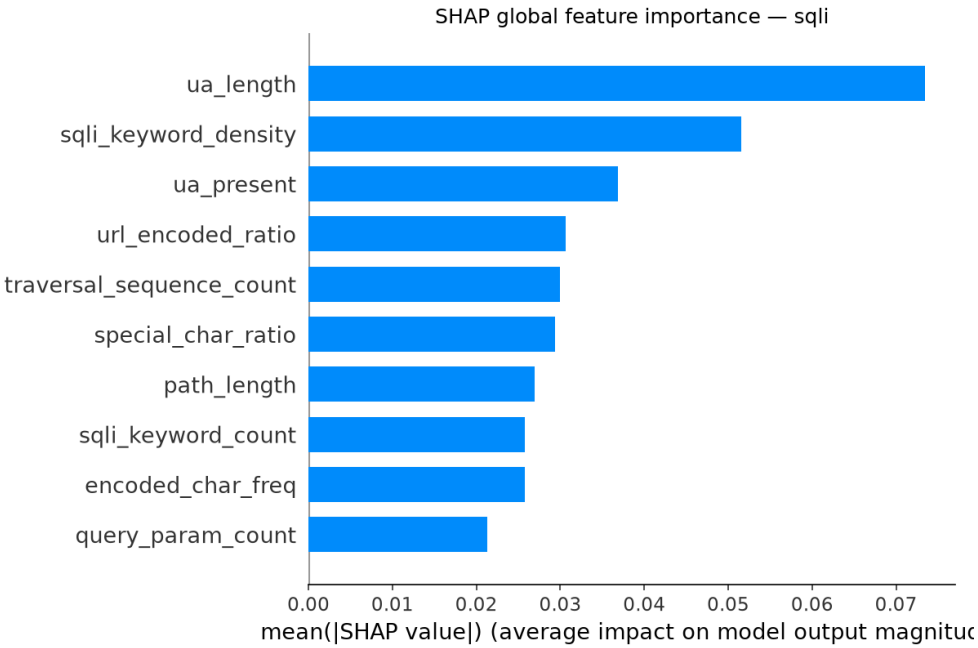


Figura 2: Importancia SHAP global, clase `sqli`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

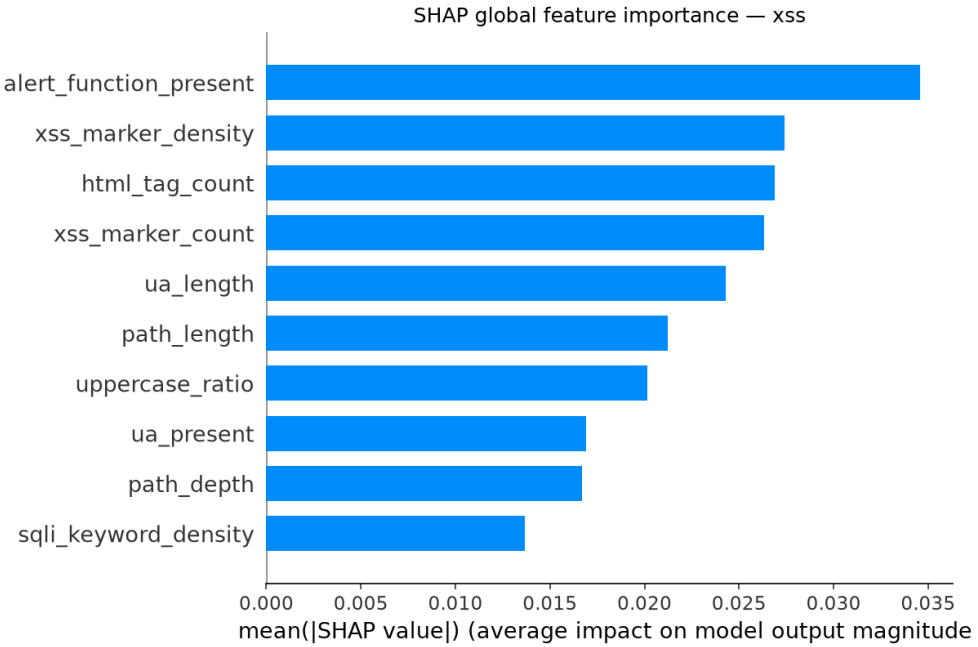


Figura 3: Importancia SHAP global, clase `xss`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

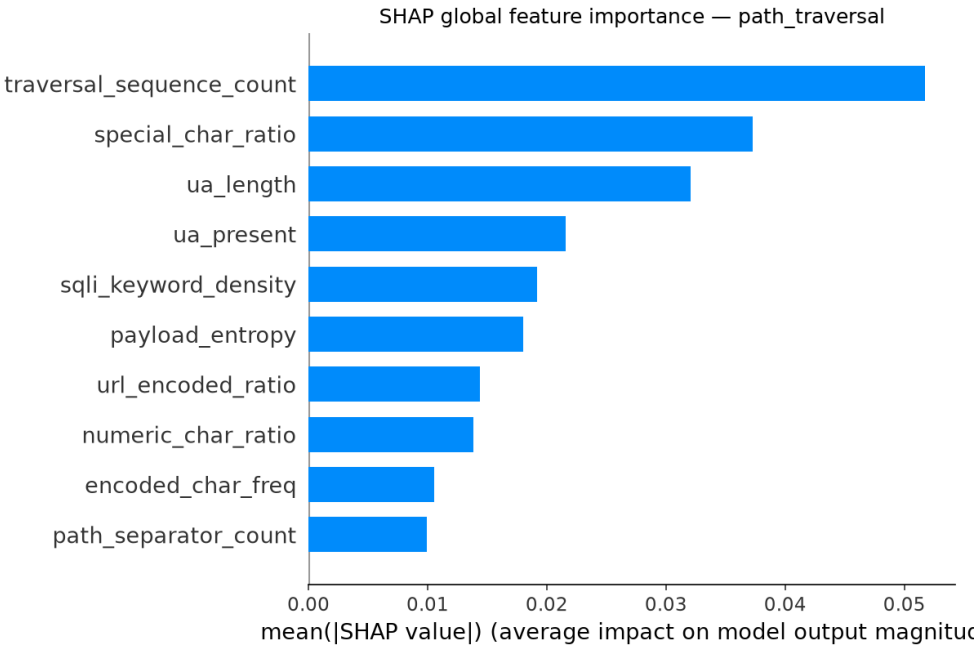


Figura 4: Importancia SHAP global, clase `path_traversal`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

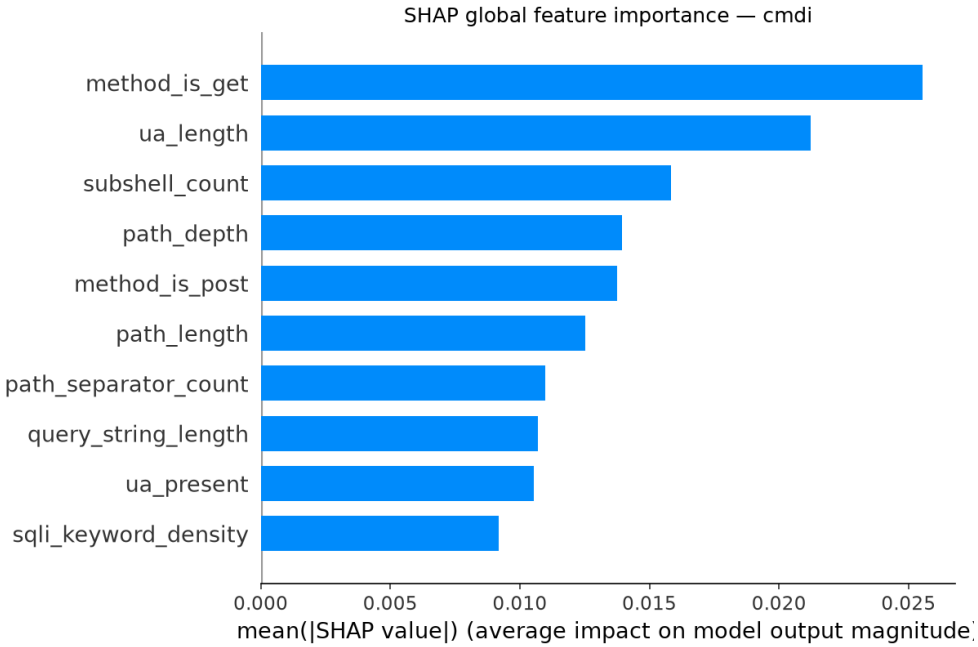


Figura 5: Importancia SHAP global, clase `cmdi`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

VIII.4.2. Explicaciones locales

Las Figuras 6 y 7 presentan la explicación local de dos peticiones `GET /profile`, una sin *User-Agent* y otra con él, y la Figura 8 la de una petición de `sqli` del conjunto de prueba de extremo a extremo (`e2e/fixtures/test_payloads.jsonl`). En cada caso el valor SHAP corresponde a la clase de mayor probabilidad predicha, y el valor base es 0.2000. El Cuadro 20 reporta los valores SHAP de `ua_present` y `ua_length` en los tres casos.

Caso	Clase predicha (probabilidad)	Valor de `ua_present`; SHAP	Valor de `ua_length`; SHAP	Mayor contribución (SHAP)
`GET /profile` sin *User-Agent*	`xss` (0.650)	0; +0.0596	0; +0.0678	`path_length` (+0.1008)
`GET /profile` con *User-Agent*	`benign` (0.533)	1; −0.0498	75; −0.0327	`special_char_ratio` (+0.0566)
Petición de `sqli` del conjunto de extremo a extremo	`sqli` (0.999)	1; +0.0485	68; +0.1136	`ua_length` (+0.1136)

Cuadro 20: Valores SHAP locales de las características de *User-Agent* y mayor contribución, para la clase predicha de cada caso.

Fuente: valores SHAP recalculados con `training/results/shap_analysis.py` sobre `rf_v11`; figuras en `training/results/shap_local_*.png`. Elaboración propia.

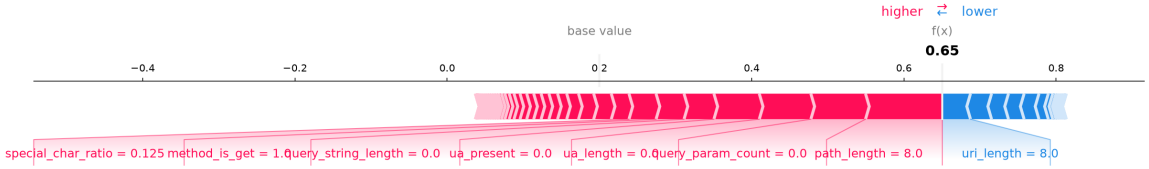


Figura 6: Explicación local de `GET /profile` sin *User-Agent* (clase `xss`).

Fuente: elaboración propia, `training/results/shap_analysis.py`.

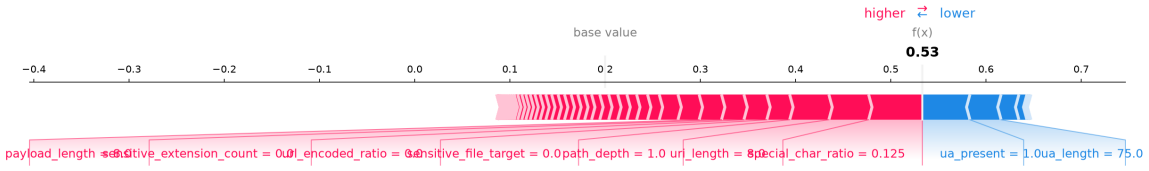


Figura 7: Explicación local de `GET /profile` con *User-Agent* (clase `benign`).

Fuente: elaboración propia, `training/results/shap_analysis.py`.

VIII.5. Limitaciones de los resultados

La regla R2 no se cumplió para la generación vigente. El conjunto de prueba se leyó tres veces para `rf_v11`/`if_v10`, el 24, el 25 y el 26 de septiembre de 2026, y las cifras de este capítulo corresponden a la lectura del 26 de septiembre. La sección de Recolección, procesamiento y validación de datos describe las tres lecturas.

Las particiones de validación y prueba comparten con la de entrenamiento la estructura de casi-duplicados, y una auditoría con exclusión por fuente muestra que, para varias fuentes, la tasa de

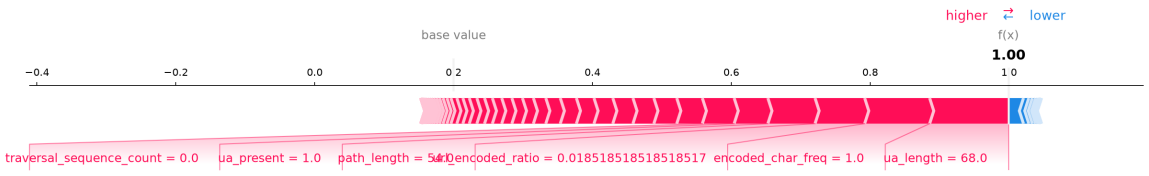


Figura 8: Explicación local de una petición de `sqli` detectada (clase `sqli`).
Fuente: elaboración propia, `training/results/shap_analysis.py`.

bloqueo de ataques o de falsos positivos cambia de forma marcada al excluirlas del entrenamiento (anexos, secciones XIII.2 y XIII.5).

Los objetivos de calibración de Isolation Forest se endurecieron después de fallos observados en el conjunto de prueba. En `if_v2`, la tasa de falsos positivos fue de 0.1011 frente a un límite de 0.10; en `if_v5`, de 0.0811 frente a un objetivo interno de 0.08. El límite formal del objetivo específico 2 se mantiene en una tasa de falsos positivos de 10 %.

El umbral de 0.35 de Random Forest se calibró sobre validación para `rf_v3` y no se recalibró para `rf_v11`. Como se indicó en la sección de resultados del objetivo específico 2, las métricas de prueba de Random Forest se calculan por argmax y no con este umbral.

De las 1 468 filas de `cmdi` del conjunto de prueba, 261 (17.8 %) son sintéticas. Su generador no está versionado en el repositorio, por lo que esas filas no pueden regenerarse a partir del código del proyecto.

En el conjunto de entrenamiento, CAPEC aporta el 81.3 % de las filas de `sqli` y el 85.8 % de las de `path_traversal`, y modsec-learn el 69.6 % de las filas `benign`. Además, 1 221 de las 6 853 filas de `cmdi` del entrenamiento (17.8 %) son sintéticas.

La fuente de los datos restaurados el 23 de septiembre de 2026 no está documentada. Además, `unified.jsonl` contiene 383 083 filas y `features.parquet` contiene 383 202; la diferencia de 119 filas corresponde a las filas de telemetría curada del pipeline de MLOps, que `unify.py` excluye y `ct_pipeline.py` añade después.

Las pruebas en aplicaciones de terceros muestran una tasa alta de bloqueo de tráfico benigno, y la contribución del User-Agent al modelo, descrita en la sección SHAP. Este trabajo no evaluó el modelo con un corpus benigno alternativo. El conjunto de tráfico benigno no cubre la diversidad de clientes HTTP que cabría esperar en un entorno real.

La contribución de Isolation Forest a la detección se midió solo sobre el conjunto de validación (anexo, sección XIII.1). Marca 19 de los 39 ataques que Random Forest no bloquea, y bloquear sus marcas agregaría 866 falsos positivos. No se evaluó sobre el conjunto de prueba. Por diseño, este componente no bloquea peticiones; su salida solo enriquece los registros.

Los datos crudos de latencia de Config 2 no constan en el repositorio: los reportes de Artillery de los experimentos en Docker no están versionados, porque `attack-sim/results/` está excluido en `.gitignore`. Las cifras de Config 2 se documentan en `docs/config2-latency-evaluation.md` del repositorio `logSguarDian-vulnerable-project`. Los reportes del experimento de latencia en Linux nativo se conservan en el artefacto de GitHub Actions `latency-benchmark-results` (id 10917957439, ejecución 36276690236), copiado a `docs/evidence/latency-linux-2026-09-26.tgz` del repositorio `logSguarDian-vulnerable-project` (rama `main`).

En cuanto a memoria y CPU, las cifras y el alcance de la medición se describen en la subsección VIII.3.8. El indicador heurístico `possibleLeak` de la corrida de monitoreo de 41.9 minutos

compara las mitades de una serie que mezcla condiciones distintas, por lo que no es concluyente. Además, 60 s por repetición no bastan para descartar una fuga de memoria.

La confusión entre `cmdi` y `sqli` en técnicas de tiempo ciego es la limitación de etiquetado más marcada. En la re-medición del 9 de septiembre de 2026 con `rf_v11/if_v10`, los 200 payloads de inyección de comandos del corpus se etiquetaron todos como `sqli` y ninguno como `cmdi`; 198 se bloquearon y 2 se registraron como `pass_anomaly`. El corpus usa oráculos de tiempo (`sleep`) y comparaciones aritméticas, y el hallazgo afecta la clase reportada en el triaje, no la decisión de bloqueo.

Como los 200 payloads coinciden con filas de las particiones de entrenamiento, validación o prueba (sección VIII.3.5), la medición no constituye evidencia de generalización.

Una evaluación sobre un corpus sin solapamiento, con las cautelas que allí se describen, se presenta en la subsección VIII.3.6. Fuentes: `docs/config2-results-v1.md`, §5.1, del repositorio `logSguarDian`, y `attack-sim/cmdi_sqli_attribution_rfv11_2026-09-09.json` del repositorio `logSguarDian-vulnerable-project` (commit 14c0e15).

Los 100 payloads por categoría de la cobertura de Config 2 tampoco son disjuntos del entrenamiento, y la cobertura no se recalculó sin esas coincidencias; véase la subsección VIII.3.2.

El conjunto de prueba se vectoriza a partir de las cadenas almacenadas, mientras que el middleware reconstruye la query con `URLSearchParams` antes de extraer las características, lo que modifica algunas de ellas para la misma petición (por ejemplo, `quote_count` pasa de 1 a 0 con el payload `' OR 1=1--`). Las métricas de prueba de este capítulo no incluyen ese efecto, que no se midió (el commit `25c7e64` del 10 de octubre de 2026, posterior a todas las evaluaciones de este capítulo, deja de re-codificar la query con `URLSearchParams`; véase la sección XIII.6).

A la fecha de redacción de este capítulo (10 de octubre de 2026), el paquete 0.1.0 (par A) contiene `rf_v10/if_v9`, y esos modelos son los que se evaluaron de extremo a extremo en las Configuraciones 1 a 3b. Las métricas de modelo de este capítulo corresponden a `rf_v11/if_v10`. La versión 0.1.1, publicada el 11 de octubre de 2026 (UTC), contiene esos modelos y el cambio de la query (anexo XIII.6).

CAPÍTULO IX

Discusión de resultados

Este capítulo interpreta los resultados del capítulo de Resultados frente a los criterios de aceptación de los tres objetivos específicos. No introduce mediciones nuevas.

IX.1. Pipeline de ingeniería de características

El extractor cumple el criterio del objetivo específico 1. El p95 mixto del ciclo de ida y vuelta a un `worker_threads` es de 0.0273 ms (Cuadro 7), frente al límite de 5 ms. La medición aísla la extracción y excluye la inferencia, el registro de eventos y la respuesta HTTP.

El incremento de latencia de extremo a extremo en Linux es de 12.08 ms en carga normal y de 12.85 ms en carga de ataque, más de 400 veces el costo del extractor (subsección VIII.3.2). La extracción no explica el sobrecosto del middleware. El middleware crea tres `worker_threads`, dos de Isolation Forest y uno de Random Forest (sección VIII.3.8), comunica los hilos por mensajes y persiste los eventos en SQLite. Este trabajo no midió el costo de cada componente por separado, y su atribución queda como trabajo futuro.

IX.2. Modelo híbrido

El Random Forest supera el F1 de 0.80 en las cuatro clases de ataque. El macro es de 0.9776, con un mínimo de 0.9239 en `cmdi` (Cuadro 9). El Isolation Forest alcanza un *recall* de 91.25 % con una tasa de falsos positivos de 5.46 %, frente a un mínimo de 50 % y un máximo de 10 % (Cuadro 10). Ambos criterios del objetivo específico 2 se cumplen con margen.

Estas cifras miden la separación entre clases dentro del corpus. Cuatro hechos acotan su lectura como estimación del desempeño en producción. El conjunto de prueba de `rf_v11` se leyó tres veces, el 24, el 25 y el 26 de septiembre de 2026 (cadena de locks en el anexo XIII.3), y se reporta la última lectura (0.9776), inferior a la de la primera (0.9843). El umbral de Isolation Forest también cambió entre esas lecturas, de 0.00807 a 0.00421; el historial de ese cambio se describe en la sección XIII.1.

El F1 se calcula por argmax y no incluye el umbral de decisión de 0.35, calibrado con `rf_v3` y sin recalibrar para `rf_v11` (sección VIII.5). Un barrido sobre validación confirma que el valor es estable entre 0.30 y 0.40, aunque no demuestra que sea óptimo (sección XIII.1). Además, 261 de las 1 468 filas de `cmdi` del conjunto de prueba (17.8 %) son sintéticas, y `cmdi` es la clase de menor

F1. Un cuarto hecho es que las particiones de validación y de prueba comparten la estructura de casi-duplicados del entrenamiento, mientras que la Ronda 5 es casi independiente de él (21 de 200 peticiones de `cmdi` tienen un vecino con $J \geq 0.8$; véase el anexo XIII.2).

El Isolation Forest no bloquea peticiones, porque el diseño reserva esa decisión al Random Forest. Su criterio se cumple como detector individual. Sobre el conjunto de validación, marca 19 de los 39 ataques que el Random Forest deja pasar, pero bloquear sus marcas agregaría 866 benignos bloqueados de más (sección XIII.1). Esas cifras son optimistas, porque el umbral de Isolation Forest se eligió sobre el mismo conjunto de validación.

IX.3. Falsos positivos y generalización

En la aplicación de referencia, ninguna petición benigna de Artillery recibió un 403. El escenario normal envió 6 300 solicitudes por repetición, incluido el calentamiento (subsección VIII.3.2). Con cadenas de User-Agent de navegadores reales se bloquearon 13 de 15 combinaciones (subsección VIII.3.2). Las cifras de Artillery (0 de 6 300, par B) y de navegadores (13 de 15, par A) corresponden a modelos distintos (sección VII.3.1).

En cuatro aplicaciones de terceros, el modelo bloqueó tráfico benigno con un User-Agent de navegador en las proporciones siguientes (subsección VIII.3.7). En dvws-node, el bloqueo fue de 92.5 % (37 de 40). En OWASP Juice Shop, el bloqueo fue de 97.1 % de peticiones benignas a las que el modelo asignó veredicto de bloqueo (312 peticiones, modo *monitor*). En DVNA, fue de 5 de 5 en una prueba preliminar.

En node-api-goat, fue de 50 de 50 en una ruta y de 4 de 5 en el chequeo de cinco peticiones. Las muestras son pequeñas. Con `curl`, Juice Shop bajó a 39.4 %, y con el User-Agent por defecto de Node, dvws-node bajó a 75 %. En tres de las cuatro aplicaciones no consta el modelo exacto de cada medición (sección VII.3.1).

El User-Agent amplifica el bloqueo, pero no lo explica por completo: con `curl`, en Juice Shop, el veredicto de bloqueo se mantiene en el 39.4 %. El análisis SHAP (Cuadro 19) coincide: `ua_length` es la característica de mayor contribución en las clases `benign` y `sqli`. Solo el 0.4 % de las filas benignas del corpus tiene un User-Agent largo, frente a entre 33.3 % y 97.7 % en las clases de ataque (anexo XIII.4). El modelo aprendió esa diferencia del corpus.

Una explicación consistente con los datos es que el corpus benigno no representa el tráfico de aplicaciones reales. Este trabajo no la verificó con un corpus benigno alternativo. La recomendación de ampliar el corpus benigno y volver a medir estos falsos positivos se formula en el capítulo XI.

La exclusión por fuente, descrita en el anexo XIII.5, muestra un patrón parecido desde otro ángulo. Al excluir `modsec_learn`, `payloads_csv` o `xss_dataset`, la tasa de falsos positivos benignos sobre sus filas de validación sube de 0.1 %, 0.4 % y 0.2 % a 76.0 %, 27.0 % y 94.5 %. Al excluir `payload_full`, `seclists_cmdi` o `command_injection`, el bloqueo de ataques cae más de 25 puntos porcentuales, de 99.8 %, 100 % y 95.8 % a 73.7 %, 29.9 % y 56.3 %.

El control de mismo tamaño no reproduce estas caídas. El aumento de falsos positivos tiene el mismo sentido que el observado en las aplicaciones de terceros (sección VIII.3.7), sin establecer una relación causal. Estas cifras provienen de modelos de exclusión distintos del evaluado en este capítulo.

IX.4. Cobertura en la evaluación integrada

Con el corpus sin solapamiento (Cuadro 17; réplica de la query en el anexo XIII.6), la cobertura supera el 80 % en `sqli`, `path_traversal` y `cmdi` con ambos pares de modelos. Con el par A

(tarball e09120…; `rf_v10/if_v9`, métricas en el anexo XIII.7) es de 100 %, 85.0 % y 91.5 %, y con `rf_v11/if_v10` es de 99.5 %, 95.0 % y 96.0 %. El corpus es el mismo en ambas corridas, así que la diferencia de 10 puntos en `path_traversal` corresponde al cambio de tarball, que difiere también en versión del código (commits `c3d591a` y `22550ea`) y en número de características (67 frente a 69); no se aisló el efecto del modelo. Con el par A (tarball e09120…), el límite inferior del intervalo de Wilson (79.4 %) queda bajo el criterio, por lo que el cumplimiento con ese par es frágil (subsección VIII.3.6). La Ronda 5 no es una evaluación estrictamente ciega del modelo reconstruido: los resultados con el par A (tarball e09120…) se conocieron el 24 de septiembre y el modelo vigente se reconstruyó el 26, y el historial no vincula ambos hechos.

La Ronda 5 no incluye XSS. La cobertura de esa categoría proviene del corpus de 590 payloads de SecLists (Cuadro 15): 97.3 %, y 97.2 % al excluir las coincidencias con el entrenamiento. En ese corpus, los 200 payloads de `cmdi` coinciden con datos de entrenamiento, y su cobertura de 100 % no es evidencia de generalización. Con el criterio de igualdad normalizada, la cobertura de `sqli`, `xss` y `path_traversal` cambia como máximo 0.1 puntos al excluir las coincidencias; `sqli` y `path_traversal` no tienen coincidencias exactas (subsección VIII.3.5). Un caso hallado con OWASP ZAP se bloquea en las configuraciones 2, 3a y 3b (1 de 1); un solo caso no permite afirmar el cumplimiento del criterio para XSS (anexo XIII.8).

Sobre esos 590 payloads, ModSecurity (CRS, nivel de paranoia 1) bloqueó 93.2 %, logSguarDian 98.8 % y ambos en capas 99.5 %. De los 40 ataques que el WAF no bloqueó (11 de `sqli`, 3 de `xss` y 26 de `path_traversal`), logSguarDian bloqueó 37. Los tres restantes pasaron ambas capas, y ninguno de los 40 es de `cmdi`. El coautor ejecutó la comparación sobre el corpus de la Ronda 5 de forma provisional y sin que conste el paquete (cortafuegos solo, 532 de 600; en capas, 597 de 600; anexo XIII.8); esas cifras no se interpretan aquí.

IX.5. Latencia, memoria y CPU

El criterio del objetivo específico 3 (Δp95 $\leq$ 15 ms) se cumple en carga normal (12.08 ms) y de ataque (12.85 ms), con márgenes de 2.92 y 2.15 ms. No se cumple en volumen: el Δp95 es de 6 495.46 ms y el 7.00 % de las peticiones termina por tiempo de espera. Esas peticiones no entran en el p95, por lo que la cifra subestima la cola.

El protocolo fijaba un incremento de hasta 10 % de la latencia media sobre la línea base. Esta tesis reporta el incremento relativo del p95, que fue de +485.1 % y +278.1 % en carga normal y de ataque, y no reporta la media. El 10 % que se menciona aquí se calcula sobre la línea base de p95 (2.49 y 4.62 ms, que equivalen a 0.25 y 0.46 ms), no sobre la latencia media que fijaba el protocolo, que no se midió. El incremento relativo del p95 se presenta como indicador complementario y no como el criterio del protocolo. Los datos son de una sola corrida en un ejecutor de 2 vCPU, con cinco repeticiones por condición (subsección VIII.3.2).

La memoria aumenta 331.9, 332.0 y 375.9 MiB en ataque, normal y volumen, más del doble del límite de 150 MB (unos 143 MiB) (Cuadro 18). El incremento es casi igual en normal y ataque, lo que sugiere un costo fijo asociado a la carga de los modelos. Esa interpretación no se verificó. La CPU aumenta 85.1 y 88.9 puntos porcentuales (+529.6 % y +614.4 %) en ataque y normal, frente a un límite de 15 %. El contenedor completo usa entre 394.8 y 459.1 MiB y no se probó en una instancia de 1 GB, para la que se fijó el límite (sección VIII.3.8).

IX.6. Comparación con trabajos previos

IntelliWAF [29] combina un Random Forest y un Isolation Forest con reglas y limitación de tasa, y opera como proxy inverso externo a la aplicación. Reporta una exactitud de 91.5 % del Random Forest sobre CSIC 2010 (61 065 muestras) y no reporta latencia de detección. Este trabajo difiere en la arquitectura, embebida en el proceso de la aplicación, y en el papel del Isolation Forest, que no participa en el bloqueo. Las diferencias de conjunto de datos y de métrica impiden comparar cifras de detección.

IX.7. Cumplimiento de los objetivos

El Cuadro 21 resume el cumplimiento de los criterios de aceptación de cada objetivo específico y de los criterios del protocolo.

Objetivo	**Criterio**	**Resultado**	**Estado**
Objetivo general	Detección de amenazas conocidas y de variantes desconocidas, sin comprometer el rendimiento del servidor	Variantes desconocidas: solo evidencia indirecta (Isolation Forest marca 19 de 39 ataques que Random Forest deja pasar, en validación); memoria (+332 MiB) y CPU (+530 % a +614 %) incumplidas	Parcial
OE1	Extractor $\leq$ 5 ms por petición	0.0273 ms (ida y vuelta al *worker*; extractor en aislamiento, el sobrecosto del middleware completo es de 12 ms)	Cumple
OE2	F1 $\geq$ 0.80 en al menos 3 de 4 clases	4 de 4 (mínimo 0.9239)	Cumple
OE2	*Isolation Forest*: *recall* $\geq$ 50 %, falsos positivos $\leq$ 10 %	91.25 % y 5.46 %	Cumple
OE3	Cobertura $\geq$ 80 % por categoría	99.5, 95.0 y 96.0 % en `sqli`, `path_traversal` y `cmdi`; XSS: 97.3 % sobre el corpus de 590	Cumple; XSS sin corpus limpio
OE3	Δp95 $\leq$ 15 ms	12.08 ms (normal) y 12.85 ms (ataque)	Cumple; volumen no cumple
Falsos positivos en aplicaciones de terceros	Sin criterio formal	92.5 % de bloqueo en dvws-node y 97.1 % de veredictos de bloqueo en Juice Shop (modo *monitor*) de tráfico benigno con User-Agent de navegador	Limitación
Protocolo	Memoria adicional $\leq$ 150 MB	+332 MiB	No cumple
Protocolo	ΔCPU $\leq$ 15 %	+530 % a +614 %	No cumple

Cuadro 21: Cumplimiento de los criterios de aceptación por objetivo específico y criterio del protocolo.

Fuente: capítulo de Resultados (Cuadros 7, 9, 10, 17, 15 y 18, y subsección VIII.3.2). Elaboración propia. OE: objetivo específico.

CAPÍTULO X

Conclusiones

Las conclusiones siguen el orden de los objetivos y se limitan a lo que los capítulos de Resultados y Discusión sustentan. Cada una indica su alcance.

Objetivo general. El objetivo general se cumplió de forma parcial. La librería existe, intercepta las peticiones de Express, clasifica las cuatro categorías de ataque y bloquea con Random Forest. La detección de variantes desconocidas solo tiene evidencia indirecta: en validación, Isolation Forest marca 19 de los 39 ataques que Random Forest deja pasar, y bloquear sus marcas agregaría 866 benignos bloqueados. Por ese costo, Isolation Forest solo registra. El requisito de no comprometer el rendimiento se cumple para el criterio de latencia en carga normal y de ataque, y no se cumple para memoria, CPU ni carga de volumen.

Objetivo específico 1. El extractor cumple el criterio en aislamiento: el p95 con `worker_threads` es de 0.0273 ms, frente al límite de 5 ms. El sobrecosto del middleware completo es de 12.08 ms en carga normal y de 12.85 ms en carga de ataque, y la extracción no lo explica. Sebastián Huertas implementó el extractor inicial de 72 características. El autor analizó y justificó cada característica, definió cuáles no existen en inferencia, fijó el contrato de entrada y añadió cuatro características y las correcciones derivadas del análisis del modelo.

Objetivo específico 2. Ambos criterios se cumplen con margen sobre el conjunto de prueba: el F1 macro de Random Forest es de 0.9776, con un mínimo de 0.9239 en `cmdi`, y Isolation Forest alcanza un *recall* de 0.9125 con una tasa de falsos positivos de 0.0546. La paridad entre scikit-learn y ONNX es del orden de 10^{-7}. Estas cifras tienen tres límites: el conjunto de prueba se leyó tres veces para la generación vigente, las métricas se calculan por argmax sin el umbral de 0.35, y las particiones comparten la estructura de casi-duplicados del entrenamiento. Miden, por tanto, la separación entre clases dentro del corpus y no la generalización a tráfico externo. Con `rf_v10` e `if_v9`, sobre otra partición de prueba, los criterios también se cumplen (F1 macro de 0.9787; anexo XIII.7).

Objetivo específico 3: cobertura. En el corpus sin solapamiento relevante, la cobertura supera el 80 % con ambos pares de modelos: con `rf_v11`/`if_v10` es de 99.5 % en `sqli`, 95.0 % en `path_traversal` y 96.0 % en `cmdi`, y con el par A (tarball e09120…), de 100 %, 85.0 % y 91.5 %. Con

el par A (tarball e09120…), el límite inferior del intervalo de `path_traversal` (79.4 %) queda bajo el criterio. La categoría `xss` carece de corpus limpio, y su cobertura de 97.3 % proviene del corpus de 590 payloads. Un caso hallado con OWASP ZAP se bloquea en las configuraciones 2, 3a y 3b (1 de 1); un solo caso no permite afirmar el cumplimiento del criterio para XSS (anexo XIII.8). En defensa en capas, logSguarDian capturó 37 de los 40 ataques que ModSecurity dejó pasar, y la comparación sobre el corpus sin solapamiento solo la ejecutó el coautor de forma provisional (anexo XIII.8).

Objetivo específico 3: latencia y recursos. El criterio de $\Delta p95 \leq 15$ ms se cumple en carga normal (12.08 ms) y de ataque (12.85 ms), y no en volumen, donde el $\Delta p95$ es de 6 495.46 ms y, en ese escenario con el middleware activo, el 7.00 % de las peticiones agota el tiempo de espera. Los datos provienen de una corrida con cinco repeticiones en un ejecutor de 2 vCPU. Los criterios del protocolo de memoria adicional (150 MB) y de CPU (15 %) no se cumplen: el incremento de memoria va de 331.9 a 375.9 MiB según el escenario, y el de CPU es de +529.6 % a +614.4 % en carga normal y de ataque. El costo por componente no se midió.

Falsos positivos y generalización. El corpus benigno es la debilidad principal. Con un User-Agent de navegador, el modelo asignó veredicto de bloqueo al 97.1 % de las peticiones benignas en OWASP Juice Shop (303 de 312, en modo *monitor*) y el 92.5 % en `dvws-node` (37 de 40; el modelo exacto de esa medición no consta). En la aplicación de referencia, con otro modelo, ninguna petición benigna con el User-Agent de Artillery recibió un 403 en los escenarios normal y de volumen. La exclusión por fuente con la configuración de `rf_v11` sigue el mismo sentido: los falsos positivos benignos suben a 76.0 %, 27.0 % y 94.5 % al excluir `modsec_learn`, `payloads_csv` o `xss_dataset` del entrenamiento (anexo XIII.5). El User-Agent es la característica de mayor peso en las clases `benign` y `sqli`, y solo el 0.4 % de las filas benignas del corpus tiene un User-Agent largo.

Contribución al problema. El problema de partida es la falta de protección accesible a nivel de aplicación en organizaciones sin equipo de seguridad. El trabajo aporta una librería distribuida en el registro npm, un pipeline de extracción de 76 características y un modelo híbrido con procedencia documentada, y establece con evidencia dónde la solución funciona y dónde no. No demuestra que la herramienta sea adecuada para producción: los criterios de memoria y CPU no se cumplen y los falsos positivos en aplicaciones de terceros son altos.

Alcance de lo entregado. La versión entregada del paquete es la 0.1.1, publicada el 11 de octubre de 2026 (UTC), que contiene `rf_v11`/`if_v10`, el cambio de la query y el extractor de 76 características. Las métricas de modelo de este trabajo corresponden a esos modelos. Las evaluaciones de integración del capítulo de Resultados usaron modelos de los pares A, B y C mediante tarballs o copias locales de la aplicación de prueba, y ninguna usó el paquete del registro de 0.1.1; las únicas dos mediciones con ese paquete son corridas de la Configuración 2 que ejecutó el coautor (anexo XIII.8). El trabajo documenta la procedencia de las cifras principales mediante hashes, locks de las particiones y anexos, y declara las limitaciones y los datos que no constan.

CAPÍTULO XI

Recomendaciones

XI.1. Infraestructura complementaria: un pipeline de MLOps para reentrenamiento continuo

El capítulo de Alcance excluye explícitamente el aprendizaje continuo y el reentrenamiento automático embebido en el proceso de la aplicación anfitriona, por ser incompatible con el principio de operación autónoma sin dependencias externas que rige el diseño de logSguarDian en tiempo de ejecución. Esa exclusión sigue vigente: la librería npm entregada no reentrena modelos dentro del proceso de la aplicación que protege. Durante el desarrollo del proyecto, sin embargo, el equipo construyó y verificó, como infraestructura separada y exploratoria –no como parte del paquete npm ni de sus dependencias en tiempo de ejecución–, un pipeline de ocho fases para el ciclo completo de recolección de telemetría, reentrenamiento y despliegue controlado de nuevas versiones del modelo híbrido. Se documenta aquí como recomendación de arquitectura de producción para quien adopte logSguarDian a escala, y no como parte del alcance evaluado en los objetivos específicos de este trabajo.

La motivación de esta exploración es directa: cada cambio al pipeline de extracción de características exige un reentrenamiento completo del modelo, condición que el proyecto siguió de forma estricta a lo largo de sus más de diez ciclos de reentrenamiento documentados. Un despliegue real de logSguarDian a múltiples aplicaciones cliente enfrentaría el mismo problema de forma recurrente, y sin un mecanismo que sistematice la recolección de evidencia de producción, la curación de esa evidencia y la validación de un modelo candidato antes de promoverlo, cada ciclo de mejora tendría que repetirse manualmente. El pipeline construido –bajo el directorio `packages/mlops` del repositorio del proyecto– explora si ese proceso puede automatizarse sin comprometer la garantía de bloqueo del modelo en producción.

Las ocho fases del pipeline son las siguientes:

1. **Recolección de telemetría.** Un endpoint dedicado recibe únicamente el vector de 76 características de cada petición clasificada, nunca el payload crudo, con el mismo patrón de

notificación asíncrona sin bloqueo (*fire-and-forget*) que ya emplea el sistema de webhooks de la librería.

2. **Simulación de flota.** Múltiples fuentes simuladas envían telemetría de forma concurrente para poblar un corpus de validación representativo de un despliegue con varios clientes.

3. **Agrupamiento de anomalías.** El componente no supervisado (Isolation Forest) y un algoritmo de agrupamiento por densidad (DBSCAN) se aplican sobre el espacio de 63 características que consume el modelo no supervisado, para aislar patrones estadísticamente atípicos sin necesidad de una etiqueta previa.

4. **Curación humana.** Una interfaz de línea de comandos presenta los grupos resultantes para revisión manual y genera un archivo con el mismo esquema que consume el script de unificación del corpus de entrenamiento del proyecto.

5. **Reentrenamiento orquestado.** Un script coordina de punta a punta la unificación del corpus curado, el reentrenamiento del modelo híbrido y la generación de los artefactos ONNX correspondientes, y reutiliza el mismo pipeline de entrenamiento documentado en el capítulo de Alcance.

6. **Validación automática mediante cinco criterios de aceptación (*gates*).** Se verifican, en orden creciente de costo computacional y con interrupción inmediata ante el primer fallo (*fail-fast*): el contrato de features del modelo candidato, sus métricas de precisión y recall frente al conjunto de prueba bloqueado, la paridad de predicción entre el modelo en Python y su versión serializada en ONNX, su desempeño sobre la suite de pruebas end-to-end, y su huella de memoria bajo la arquitectura real de *worker pool* de producción.

7. **Despliegue canario.** Un modelo candidato que aprueba los cinco criterios anteriores se evalúa mediante dos mecanismos: reemplazo del corpus de prueba etiquetado como mecanismo primario de decisión, y una ventana acotada de sombra sobre tráfico en vivo como mecanismo secundario, nunca como proceso continuo indefinido.

8. **Promoción o reversión.** La promoción de un modelo candidato a producción exige el cumplimiento simultáneo de un criterio estricto –cero casos en los que el candidato deje pasar un ataque que el modelo en producción bloquea correctamente– y un criterio de concordancia general de al menos 99.5 % entre los veredictos de bloqueo del candidato y los del modelo vigente sobre el corpus de reemplazo.

Como prueba de concepto, el pipeline de reentrenamiento se ejecutó el 29 de agosto de 2026 sobre telemetría simulada: 244 eventos de cinco fuentes simuladas (sim-host-1 a sim-host-5, 238 eventos, que reparten el corpus de payloads de prueba) y de una fuente dedicada (sim-host-novel, 6 eventos), que envió seis réplicas de un payload poliglota diseñado para combinar, en una sola petición, sintaxis de las cuatro categorías de ataque cubiertas. El reporte de agrupamiento (DBSCAN con `eps` 2.5 y `min_samples` 3) registra 183 eventos anómalos y 16 grupos, además de un grupo de ruido de 53 eventos. Las seis réplicas formaron un grupo propio, con puntaje de anomalía promedio de -0.153, inferior al umbral de 0.002486 que usa el script de agrupamiento (el de `if_v9`), y el componente supervisado las clasificó como XSS. Según un comentario del script generador, una versión anterior del script, con un payload ofuscado de una sola clase, no produjo esa separación; esa versión no se conserva en el repositorio. Dos cautelas acotan el resultado: las seis peticiones son réplicas idénticas, por lo que su agrupación no mide la capacidad de generalizar a variantes

distintas, y el reporte proviene de una sola ejecución local, conservada fuera del repositorio (SHA-256 `7fb882d5...e7e6`).

Se recomienda, para quien adopte logSguarDian en un despliegue con múltiples aplicaciones cliente, evaluar la maduración de esta infraestructura complementaria –hoy una prueba de concepto verificada, no un servicio productivo– como mecanismo de mantenimiento a largo plazo del modelo, y mantener en todo momento la separación arquitectónica ya adoptada en este proyecto entre la librería embebida que protege la aplicación en tiempo real y la infraestructura externa que sostiene su ciclo de mejora continua.

XI.2. Ampliación del corpus benigno

Se recomienda ampliar el corpus benigno con tráfico representativo de cada aplicación objetivo y volver a medir los falsos positivos en aplicaciones de terceros con el mismo protocolo descrito en la subsección VIII.3.7.

Esa ampliación debe incluir tráfico de clientes HTTP diversos y cadenas de *User-Agent* variadas, dado el peso de esa característica en el modelo y el nivel de bloqueo observado en aplicaciones de terceros (sección VIII.3.7).

XI.3. Validación de la versión publicada

La versión 0.1.0 del paquete contiene `rf_v10` e `if_v9`, mientras que las métricas de modelo de este trabajo corresponden a `rf_v11` e `if_v10`. El 11 de octubre de 2026 se publicó la versión 0.1.1, que contiene esos modelos y el cambio del commit `25c7e64` en la forma en que el middleware entrega la query al extractor (anexo XIII.6). El coautor ejecutó la Configuración 2 sobre la Ronda 5 con esa versión instalada desde el registro (anexo XIII.8). No consta que se haya repetido con ella el experimento de latencia, memoria y CPU.

Se recomienda repetir ese experimento en Linux nativo con la versión 0.1.1 del registro, e identificar el paquete por su SHA-256 en las Configuraciones 1, 3a y 3b sobre la Ronda 5. Se recomienda también medir el desajuste entre la cadena de consulta del entrenamiento y la que recibe el middleware, que este trabajo no midió. Hasta entonces, las mediciones de integración de este trabajo con `rf_v11` e `if_v10` corresponden a tarballs de validación, salvo las dos corridas de la Configuración 2 del coautor, y las cifras de latencia, memoria y CPU corresponden al par B.

XI.4. Costo de memoria y CPU por componente

Los incrementos de memoria (de 331.9 a 375.9 MiB) y de CPU (de +529.6 % a +614.4 % en carga normal y de ataque) superan los criterios del protocolo, y este trabajo no los atribuyó a ningún componente. Se recomienda medirlos en el mismo entorno para tres configuraciones: solo Random Forest, Random Forest con un *worker* de Isolation Forest, y la configuración actual de

tres `worker_threads`. Con esas mediciones debería evaluarse también `IF_POOL_SIZE` igual a 1. El resultado de esas mediciones no se establece aquí.

XI.5. Corpus de evaluación fuera del entrenamiento

La categoría `xss` carece de un corpus sin solapamiento; un caso hallado con ZAP no basta para evaluar su cobertura. Las Configuraciones 1, 3a y 3b se ejecutaron sobre la Ronda 5 de forma provisional, sin paquete identificado en los resultados (anexo XIII.8). Se recomienda construir un corpus limpio de `xss` y repetir la comparación con el cortafuegos de aplicaciones web sobre ambos corpus con el paquete 0.1.1 del registro. Se recomienda también definir para cada payload su forma exacta en el cable, para que el arnés no la altere. Con el tarball del par A, el límite inferior del intervalo de Wilson de `path_traversal` (79.4 %) queda bajo el criterio del 80 %; la medición debe repetirse con un número mayor de payloads.

XI.6. Rigor del protocolo de evaluación

Las particiones de validación y prueba comparten la estructura de casi-duplicados del entrenamiento, y la exclusión por fuente muestra una dependencia marcada de la fuente en varios casos (anexos XIII.2 y XIII.5). Se recomienda construir las particiones con agrupamiento de casi-duplicados y reservar fuentes completas para validación y prueba. El conjunto de prueba se leyó tres veces para la generación vigente, y su lock se re-fijó después del reentrenamiento (anexo XIII.3). Se recomienda registrar cada lectura y escribir el lock en un commit previo al entrenamiento. Se recomienda además calcular las métricas de prueba con la regla de decisión desplegada, revalidar el umbral de 0.35 sobre una partición con menos casi-duplicados, porque la validación actual discrimina poco entre 0.20 y 0.40 (anexo XIII.1) y evaluar Isolation Forest fuera de validación.

CAPÍTULO XII

Bibliografía

[1] OWASP Foundation, *OWASP Top 10: 2021. The most critical web application security risks*, https://owasp.org/Top10/, 2021.

[2] A. L. Buczak y E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, n.º 2, págs. 1153-1176, 2016. DOI: 10.1109/COMST.2015.2494502

[3] I. H. Sarker, A. S. M. Kayes, S. Badsha, H. Alqahtani, P. Watters y A. Ng, "Cybersecurity data science: An overview from machine learning perspective," *Journal of Big Data*, vol. 7, n.º 1, págs. 1-29, 2020. DOI: 10.1186/s40537-020-00318-5

[4] A. Verma y V. Ranga, "Machine learning based intrusion detection systems for IoT paradigms and datasets: A review," *IEEE Access*, vol. 8, págs. 180 664-180 709, 2020. DOI: 10.1109/ACCESS.2020.3027778

[5] D. S. Berman, A. L. Buczak, J. S. Chavis y C. L. Corbett, "A survey of deep learning methods for cyber security," *Information*, vol. 10, n.º 4, pág. 122, 2019. DOI: 10.3390/info10040122

[6] L. Zhen, N. Hafizah Kamarudin, V. Jyn Kok y F. Qamar, "Anomaly detection model in network security situational awareness based on machine learning: Limitation, techniques, and future trends," *IEEE Access*, vol. 13, págs. 126 084-126 129, 2025. DOI: 10.1109/ACCESS.2025.3589620

[7] A. Rahman, S. Sultana, U. Twaha y M. Rowshon, "AI-enhanced web application firewalls for protecting United States critical infrastructure against zero-day exploits," *Scientia. Technology, Science and Society*, vol. 3, n.º 2, págs. 11-32, 2026. DOI: 10.59324/stss.2026.3(2).02

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, n.º 1, págs. 5-32, 2001. DOI: 10.1023/A:1010933404324

[9] F. T. Liu, K. M. Ting y Z.-H. Zhou, "Isolation forest," en *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, págs. 413-422. DOI: 10.1109/ICDM.2008.17

[10] V. Chandola, A. Banerjee y V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, n.º 3, págs. 1-58, 2009. DOI: 10.1145/1541880.1541882

[11] A. Shiravi, H. Shiravi, M. Tavallaee y A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Computers & Security*, vol. 31, n.º 3, págs. 357-374, 2012. DOI: 10.1016/j.cose.2011.12.014

[12] P. Warden y D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[13] Verizon, *2024 Data breach investigations report (DBIR)*, https://www.verizon.com/business/resources/reports/dbir/, 2024.

[14] Snyk, *State of open source security report 2024*, https://snyk.io/lp/state-of-open-source-2024/, 2024.

[15] Mastercard, *Guatemala avanza en su digitalización: Mastercard presenta hallazgos clave sobre ciberseguridad y refuerza su compromiso con la protección del ecosistema digital en el país*, Comunicado de prensa, agosto de 2025. https://www.mastercard.com/car/en/news-and-trends/press/2025/august/guatemala-avanza-en-su-digitalizacion-mastercard-presenta-hallazgos-clave-sobre-ciberseguridad-y-refuerza-su-compromiso-con-la-proteccion-del-ecosistema-digital-en-el-pais.html, Consultado el 11 de octubre de 2026, 2025.

[16] OpenJS Foundation, *Node.js documentation: Event loop and worker threads*, https://nodejs.org/en/docs, 2024.

[17] E. Yuan y S. Malek, "A taxonomy and survey of self-protecting software systems," en *2012 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, IEEE, 2012, págs. 109-118. DOI: 10.1109/SEAMS.2012.6224397

[18] OWASP Foundation, *OWASP Core Rule Set (CRS) project*, https://owasp.org/www-project-modsecurity-core-rule-set/, 2024.

[19] M. Zimmermann, C.-A. Staicu, C. Tenny y M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," en *28th USENIX Security Symposium (USENIX Security 19)*, USENIX Association, 2019, págs. 995-1010.

[20] T. G. Dietterich, "Ensemble methods in machine learning," en *Multiple Classifier Systems. MCS 2000. Lecture Notes in Computer Science*, vol. 1857, Springer, 2000, págs. 1-15. DOI: 10.1007/3-540-45014-9_1

[21] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016.

[22] A. Zheng y A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O'Reilly Media, 2018.

[23] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, n.º 8, págs. 861-874, 2006. DOI: 10.1016/j.patrec.2005.10.010

[24] J. Bai, F. Lu, K. Zhang et al., *ONNX: Open Neural Network Exchange*, https://github.com/onnx/onnx, 2019.

[25] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011.

[26] G. Lucz y B. Forstner, "A Thirty-Day Dataset of Malicious HTTP Requests Blocked by OWASP ModSecurity on a Production Web Server," *Data*, vol. 10, n.º 11, 2025, Art. 186. Conjunto de datos: Zenodo, versión 1, 22 de septiembre de 2025, doi:10.5281/zenodo.17178461. DOI: 10.3390/data10110186

[27] Miessler, Daniel, *SecLists*, https://github.com/danielmiessler/SecLists, Consultado el 9 de octubre de 2026.

[28] Swissky, *PayloadsAllTheThings*, https://github.com/swisskyrepo/PayloadsAllTheThings, Consultado el 9 de octubre de 2026.

[29] P. K. Shinde, Y. R. Ingole, N. P. Tikore, G. S. Gosavi y S. B. Chaudhari, "IntelliWAF: A Six-Layer Intelligent Web Application Firewall with Machine Learning and Anomaly Detection for Real-Time Web Threat Mitigation," *International Journal of Engineering Research & Technology (IJERT)*, vol. 15, n.º 6, 2026, Artículo IJERTV15IS061068. https://www.ijert.org/intelliwaf-a-six-layer-intelligent-web-application-firewall-with-machine-learning-and-anomaly-detection-for-real-time-web-threat-mitigation-ijertv15is061068, ISSN: 2278-0181. DOI: 10.5281/zenodo.20917852

CAPÍTULO XIII

Anexos

XIII.1. Revalidación del umbral de decisión de Random Forest y aporte de Isolation Forest

Este anexo verifica si el umbral de decisión de Random Forest se mantiene estable para la generación `rf_v11`, y mide qué aporta Isolation Forest (`if_v10`) a la detección cuando se combina con Random Forest. Todas las cifras provienen de la evaluación sobre el conjunto de validación, no sobre el conjunto de prueba.

XIII.1.1. Revalidación del umbral de Random Forest

El umbral `RF_THRESHOLD = 0.35` se fijó sobre el conjunto de validación con `rf_v3` y se mantuvo en las generaciones posteriores sin recalibración (`docs/decision-policy.md`, §2.2.1, del repositorio logSguarDian). Para `rf_v11` se repitió el barrido sobre el conjunto de validación de 57 481 filas, sin cambiar el valor desplegado. El autor definió el criterio de estabilidad antes de ejecutar el barrido; no consta registro en el repositorio. El criterio exige que la variación entre los umbrales 0.30 y 0.40 sea menor que 0.5 puntos porcentuales, tanto en la tasa de bloqueo benigno como en el recall global.

Las variaciones observadas fueron de 0.0067 puntos porcentuales en la tasa de bloqueo benigno y de 0.056 puntos porcentuales en el recall global, por lo que el criterio se cumple. Esta lectura acota el resultado a estabilidad y no a optimalidad. Con un recall cercano a 99.9 %, la validación discrimina poco entre los umbrales 0.20 y 0.40. El umbral no se recalibró, y la Configuración 2 y la Ronda 5 se mantienen con 0.35.

El Cuadro 22 presenta el barrido completo.

Umbral T	Benignos bloqueados (%)	Recall global (%)	sqli	xss	path_traversal	cmdi
0.20	0.114	99.930	99.977	99.666	99.842	99.796
0.25	0.114	99.930	99.977	99.666	99.842	99.796
0.30	0.114	99.930	99.977	99.666	99.842	99.796
0.35	**0.107**	**99.908**	**99.962**	**99.644**	**99.723**	**99.796**
0.40	0.107	99.873	99.938	99.577	99.644	99.659
0.45	0.107	99.775	99.877	99.443	99.446	98.978
0.50	0.107	99.634	99.780	99.354	99.051	98.093
0.55	0.067	99.364	99.598	99.043	98.378	96.594
0.60	0.067	99.117	99.417	98.798	97.785	95.436
0.65	0.054	98.742	99.117	98.397	97.191	93.733
0.70	0.047	98.296	98.695	98.063	96.598	92.643
0.75	0.027	97.610	98.053	97.506	95.728	90.872
0.80	0.020	96.774	97.218	96.794	95.055	89.373
0.85	0.020	95.976	96.537	95.569	94.146	87.330
0.90	0.013	94.858	95.576	94.166	92.642	84.128

Cuadro 22: Barrido del umbral de decisión de Random Forest (`rf_v11`) sobre el conjunto de validación de 57 481 filas. Las columnas por clase son recall en porcentaje.

Fuente: `_evidencia/revalidacion-umbral-if/analysis_out.txt`, sección 1. Elaboración propia.

XIII.1.2. Aporte de Isolation Forest

Con $T = 0.35$, Random Forest deja pasar 39 de los 42 594 ataques del conjunto de validación. Isolation Forest marca 19 de esos 39 ataques: 5 de 13 de `sqli`, 10 de 16 de `xss`, 4 de 7 de `path_traversal` y ninguno de los 3 de `cmdi` que Random Forest dejó pasar. El Cuadro 23 resume estas cifras.

Se evaluó una regla hipotética que bloquea una petición si cualquiera de los dos modelos la marca. Esa regla eleva el recall global de 99.908 % a 99.953 %. El costo es un aumento del bloqueo benigno de 0.107 % a 5.925 %, es decir, 866 benignos adicionales bloqueados. El umbral de Isolation Forest (0.004206) se eligió sobre este mismo conjunto de validación, por lo que estas cifras son optimistas. La regla no se desplegó.

Estas cifras respaldan que Isolation Forest solo registre. Su aporte en detección es de 19 ataques frente a 42 594, mientras que bloquear sus marcas agregaría 866 falsos positivos. Según el diseño del sistema, las marcas de Isolation Forest se registran con veredicto `pass_anomaly` y no bloquean (`packages/core/src/middleware.ts`, líneas 243–244 y 333–337 del repositorio logSguarDian).

XIII.1.3. Dependencia de Isolation Forest respecto del User-Agent

De los 874 benignos que Isolation Forest marca en validación, 807 son filas sin User-Agent y 67 son filas con User-Agent. En las 14 818 filas benignas sin User-Agent, la tasa de marca es de 5.45 %. En cambio, entre las 69 filas benignas con User-Agent, Isolation Forest marca 67 (97.1 %). Este grupo es pequeño, por lo que la tasa no permite una estimación precisa. Sin embargo, es compatible con una dependencia de Isolation Forest respecto de las características de User-Agent, que conviene vigilar (`_evidencia/revalidacion-umbral-if/ua_check_regenerado_out.txt`).

Concepto	Casos	Valor
Ataques no bloqueados por Random Forest ($T = 0.35$)	39 de 42 594	—
Ataques de esos 39 marcados por Isolation Forest	19 de 39	—
`sqli`	5 de 13	—
`xss`	10 de 16	—
`path_traversal`	4 de 7	—
`cmdi`	0 de 3	—
Recall global, solo Random Forest	42 555 de 42 594	99.908 %
Recall global, regla "RF o IF"	42 574 de 42 594	99.953 %
Benignos bloqueados por Random Forest	16 de 14 887	0.107 %
Benignos que Random Forest no bloquea y que Isolation Forest marca	866 de 14 871	5.82 %
Benignos marcados por Isolation Forest (tasa global)	874 de 14 887	5.87 %
Benignos bloqueados por la regla "RF o IF"	882 de 14 887	5.925 %

Cuadro 23: Aporte de Isolation Forest sobre el conjunto de validación con $T = 0.35$ y regla hipotética "RF o IF".

Fuente: `_evidencia/revalidacion-umbral-if/analysis_out.txt`, sección 2. El numerador 874 no figura en esa salida; proviene de `_evidencia/revalidacion-umbral-if/ua_check_regenerado_out.txt` (suma de 807 y 67 marcas), y la tasa de 5.871 % coincide con `training/models/if_v10_metadata.json`. Elaboración propia.

XIII.1.4. Limitaciones

Este análisis usa una sola partición de validación de 57 481 filas. No se dispone de una segunda partición que permita medir la variación entre particiones.

El umbral de Random Forest (0.35) se eligió sobre validación con `rf_v3`. El umbral de Isolation Forest de `if_v10` (0.004206) se eligió sobre validación como el máximo recall entre los umbrales con recall mayor o igual a 0.50 y tasa de falsos positivos menor o igual a 0.06 (`training/retrain_v11_contract69.py`, líneas 107–110). Para `rf_v11`, el gate de validación exige al menos tres de las cuatro clases de ataque con F1 mayor o igual a 0.80 (línea 60). Sus hiperparámetros, como `n_estimators=30` y `max_depth=25` (línea 51), están fijados en el script, que no realiza búsqueda de hiperparámetros sobre validación.

El umbral de Isolation Forest pasó de 0.00807 a 0.00421 entre las lecturas del conjunto de prueba. El umbral de 0.00807 corresponde al modelo previo al reentrenamiento, leído el 24 de septiembre de 2026. El de 0.00421 corresponde al modelo reentrenado, leído el 25 y el 26 de septiembre de 2026, y se calculó sobre validación. El historial no atribuye ese cambio al resultado de la primera lectura, pero no puede descartarse su influencia. Este hecho acota la afirmación de que el umbral se fijó solo sobre validación.

Isolation Forest no se evaluó fuera de validación en este anexo. La regla R2 de la metodología impide una nueva lectura del conjunto de prueba, por lo que la regla "RF o IF" no se midió sobre dicho conjunto.

Los scripts `analysis.py` y `part3.py` usan rutas absolutas de la máquina del autor, por lo que requieren ajustarlas para reproducirse.

XIII.1.5. Procedencia

Los modelos evaluados son `rf.onnx` e `if.onnx` del par C, descrito en la sección de pares de modelos (sección VII.3.1). Sus sumas SHA-256 completas son las del Cuadro 24. El commit del repositorio logSguarDian que produjo estos modelos es `03de50f`. El archivo `analysis_out.txt` no registra el commit de ejecución; durante la verificación, el árbol de trabajo estaba limpio en el commit `41553a7`.

Archivo	**SHA-256**
`rf.onnx` (modelo, par C)	25b407e649283606a231665a5a8c8ea1c2c7ad4fd0374a8945b3f119651b0870
`if.onnx` (modelo, par C)	5be28ee8437643b2f13708eee1ac3175348a350ea6fb8f8daeb09bd5717f797e
`analysis_out.txt`	78dbf342acf01aacf112e76cd9fddfa95d3388de551a2c0eb1cd0901344835a4
`part3_out.txt`	a03cb07f2e3433e7dfae4ae35866abe9828c76bffb0aa6c4afe8f049d477322e
`ua_check.txt`	1027dacfbf97cc337994fbac7c989581a3fd7cd26269d1d0e5f4730081c940f7
`analysis.py` (genera `analysis_out.txt`)	7c00ad61b662f899ae5d5f2f0cc73ccc5871db149d33189a6e7f16b53ea4e7d2
`part3.py` (genera `part3_out.txt`)	78ace833702df2078090dad5d6d1c0bb3c83994e6cfd9e221c12b9ce5bd99d9f
`ua_check_regenerado.py` (regenera `ua_check.txt`)	56516f5f6b63d3c3886ef4a6350807c356d4c2fc49d8ea77a0207be4adcffe5f
`ua_check_regenerado_out.txt`	d61660015b8f763af7140987836b6b94559c025d76f2d4ff980766d5ba5fe6a6

Cuadro 24: Sumas SHA-256 de los modelos evaluados, de los archivos de evidencia y de los scripts que los generaron o regeneraron.

Fuente: hashes de los modelos, de `training/models/` en el commit `03de50f` del repositorio logSguarDian; hashes de los archivos de evidencia y de los scripts, calculados con `shasum -a 256` sobre `_evidencia/revalidacion-umbral-if/`. El script original que generó `ua_check.txt` no se conserva; `ua_check_regenerado.py` es una regeneración que reproduce las cifras de validación de ese archivo, y su salida se guarda en `ua_check_regenerado_out.txt`. La línea de `ua_check.txt` sobre el `login` no se reproduce, porque depende de `login_vec.json`. Elaboración propia.

XIII.2. Casi-duplicados entre particiones

Este anexo cuantifica la similitud textual de cada fila de validación, de prueba y de la Ronda 5 con el conjunto de entrenamiento, y la de cada fila de entrenamiento con el resto de ese conjunto. Es una auditoría posterior a la evaluación: no modifica el corpus, los modelos ni las métricas reportadas.

XIII.2.1. Parámetros

La similitud se mide sobre el texto de cada petición. Cada campo (`path`, `query` y `body`) se normaliza con decodificación `unquote_plus`, conversión a minúsculas y colapso de espacios, y los tres campos se unen con el separador ` | `. El texto normalizado se descompone en shingles de tres caracteres, y la similitud entre dos peticiones es el índice de Jaccard exacto de sus conjuntos de shingles.

Para encontrar candidatos se usa MinHash con `num_perm=128` y LSH con bandas de `params =(8,16)`. Cada candidato se verifica con el Jaccard exacto, de modo que todo par informado con $J \geq$ umbral es un par real; los pares que el LSH omite se tratan en las limitaciones.

Se usan los umbrales 0.8 y 0.9. Para cada fila, *cualquiera* es el máximo de J frente a todas las filas de entrenamiento, y *misma clase* es el máximo frente a las filas con la misma etiqueta. En la Ronda 5, el texto de una petición `GET` es su `query` y el de una `POST` es su `body`, en la forma `param=payload`. Quedan fuera las filas sin texto bruto (82 de entrenamiento, 18 de validación y 19 de prueba), y el n de cada cuadro lo refleja.

XIII.2.2. Resultados

Cada celda indica el número de filas con $J \geq$ umbral y su porcentaje sobre n. Los cuadros 25, 26 y 27 cubren las tres particiones con todas sus clases; el cuadro 28 cubre la Ronda 5.

Clase	n	Cualquiera ≥ 0.8	Cualquiera ≥ 0.9	Misma clase ≥ 0.8	Misma clase ≥ 0.9
`benign`	14 887	4 491 (30.17 %)	3 241 (21.77 %)	4 490 (30.16 %)	3 241 (21.77 %)
`sqli`	34 103	25 356 (74.35 %)	18 840 (55.24 %)	25 308 (74.21 %)	18 804 (55.14 %)
`xss`	4 484	2 076 (46.30 %)	1 115 (24.87 %)	2 054 (45.81 %)	1 112 (24.80 %)
`path_traversal`	2 526	1 751 (69.32 %)	1 287 (50.95 %)	1 704 (67.46 %)	1 253 (49.60 %)
`cmdi`	1 463	580 (39.64 %)	323 (22.08 %)	542 (37.05 %)	302 (20.64 %)
Total	57 463	34 254 (59.61 %)	24 806 (43.17 %)	34 098 (59.34 %)	24 712 (43.01 %)

Cuadro 25: Casi-duplicados de la partición de validación frente al entrenamiento (57 463 filas con texto).

Fuente: `neardup_v2_results.json`, sección `val`; `tables.md`. Elaboración propia.

Clase	n	Cualquiera ≥ 0.8	Cualquiera ≥ 0.9	Misma clase ≥ 0.8	Misma clase ≥ 0.9
`benign`	14 886	4 404 (29.58 %)	3 176 (21.34 %)	4 403 (29.58 %)	3 175 (21.33 %)
`sqli`	34 103	25 569 (74.98 %)	18 841 (55.25 %)	25 511 (74.81 %)	18 809 (55.15 %)
`xss`	4 484	2 061 (45.96 %)	1 090 (24.31 %)	2 038 (45.45 %)	1 087 (24.24 %)
`path_traversal`	2 526	1 788 (70.78 %)	1 314 (52.02 %)	1 749 (69.24 %)	1 290 (51.07 %)
`cmdi`	1 463	557 (38.07 %)	326 (22.28 %)	523 (35.75 %)	299 (20.44 %)
Total	57 462	34 379 (59.83 %)	24 747 (43.07 %)	34 224 (59.56 %)	24 660 (42.92 %)

Cuadro 26: Casi-duplicados de la partición de prueba frente al entrenamiento (57 462 filas con texto). Solo se comparó el texto; el conjunto de prueba no se puntuó.

Fuente: `neardup_v2_results.json`, sección `test`; `tables.md`. Elaboración propia.

Clase	n	Cualquiera ≥ 0.8	Cualquiera ≥ 0.9	Misma clase ≥ 0.8	Misma clase ≥ 0.9
`benign`	69 472	20 765 (29.89 %)	14 824 (21.34 %)	20 763 (29.89 %)	14 822 (21.34 %)
`sqli`	159 138	118 649 (74.56 %)	88 043 (55.32 %)	118 375 (74.39 %)	87 878 (55.22 %)
`xss`	20 929	9 534 (45.55 %)	5 217 (24.93 %)	9 475 (45.27 %)	5 201 (24.85 %)
`path_traversal`	11 787	8 257 (70.05 %)	6 035 (51.20 %)	8 044 (68.24 %)	5 930 (50.31 %)
`cmdi`	6 832	2 697 (39.48 %)	1 508 (22.07 %)	2 508 (36.71 %)	1 393 (20.39 %)
Total	268 158	159 902 (59.63 %)	115 627 (43.12 %)	159 165 (59.35 %)	115 224 (42.97 %)

Cuadro 27: Casi-duplicados del entrenamiento frente a sí mismo, excluida la propia fila (268 158 filas con texto).

Fuente: `neardup_v2_results.json`, sección `train_self`; `tables.md`. Elaboración propia.

Con umbral 0.8, el 59.61 % de las filas de validación, el 59.83 % de las de prueba y el 59.63 % de las de entrenamiento tienen al menos un vecino de cualquier clase con $J \geq 0.8$. Con la misma

Clase	n	Cualquiera ≥ 0.8	Cualquiera ≥ 0.9	Misma clase ≥ 0.8	Misma clase ≥ 0.9
`sqli`	200	0 (0.00 %)	0 (0.00 %)	0 (0.00 %)	0 (0.00 %)
`path_traversal`	200	0 (0.00 %)	0 (0.00 %)	0 (0.00 %)	0 (0.00 %)
`cmdi`	200	21 (10.50 %)	7 (3.50 %)	21 (10.50 %)	7 (3.50 %)
Total	600	21 (3.50 %)	7 (1.17 %)	21 (3.50 %)	7 (1.17 %)

Cuadro 28: Casi-duplicados de la Ronda 5 (200 peticiones por clase) frente al entrenamiento.

Fuente: `neardup_v2_results.json`, sección `round5`; `tables.md`. Elaboración propia.

clase, esas cifras son 59.34 %, 59.56 % y 59.35 %. En la Ronda 5, 21 de las 200 peticiones de `cmdi` tienen un casi-duplicado en entrenamiento con $J \geq 0.8$; ninguna de `sqli` ni de `path_traversal` lo tiene.

XIII.2.3. Desempeño en validación según haya o no vecino en entrenamiento

El cuadro separa las filas de validación según tengan o no un vecino en entrenamiento con $J \geq 0.8$. Reporta la tasa de bloqueo de benignos (falsos positivos) y el recall por clase, con su intervalo de Wilson al 95 %. No se reporta F1.

Grupo	Clase	Bloqueados/n	Tasa	IC 95 % (Wilson)
Con vecino	Benignos (tasa de bloqueo, FP)	1/4 491	0.0002	[0.0000, 0.0013]
Con vecino	`cmdi` (recall)	578/580	0.9966	[0.9875, 0.9991]
Con vecino	`path_traversal` (recall)	1 750/1 751	0.9994	[0.9968, 0.9999]
Con vecino	`sqli` (recall)	25 354/25 356	0.9999	[0.9997, 1.0000]
Con vecino	`xss` (recall)	2 074/2 076	0.9990	[0.9965, 0.9997]
Sin vecino	Benignos (tasa de bloqueo, FP)	15/10 396	0.0014	[0.0009, 0.0024]
Sin vecino	`cmdi` (recall)	882/883	0.9989	[0.9936, 0.9998]
Sin vecino	`path_traversal` (recall)	770/775	0.9935	[0.9850, 0.9972]
Sin vecino	`sqli` (recall)	8 736/8 747	0.9987	[0.9977, 0.9993]
Sin vecino	`xss` (recall)	2 394/2 408	0.9942	[0.9903, 0.9965]

Cuadro 29: Bloqueos y recall en validación, separados por la existencia de un vecino en entrenamiento con $J \geq 0.8$. Modelo par C (`rf.onnx` e `if.onnx`; sin F1).

Fuente: `block_gap_counts.json` y `block_gap_counts.md` en `_evidencia/casi-duplicados-v3/`. Elaboración propia.

Lectura acotada. Las diferencias de recall entre ambos grupos son de 0.6 puntos porcentuales o menos en todas las clases; la mayor es de 0.59 en `path_traversal`. De los 16 falsos positivos de validación, 15 están en filas sin vecino.

Con tan pocos casos, la tasa de ese grupo no se estima con precisión. El grupo "sin vecino" contiene además filas que sí tienen un vecino real que el LSH no encontró, cerca del 11 % de los pares con $J \geq 0.8$ según la medición de recall de este anexo. Por eso la brecha real entre grupos puede ser mayor que la observada. Estos datos no establecen una relación causal.

XIII.2.4. Cobertura de `cmdi` en la Ronda 5 sin los payloads con vecino

La cobertura de `cmdi` en la Ronda 5 se recalcula sin los payloads que tienen un vecino en entrenamiento. El denominador es de 200 payloads, y los timeouts, errores y respuestas HTTP 5xx

cuentan como no bloqueados. El par A (tarball e09120…) usa `results_config2.json` del commit `a3aada0`, y el par C usa `results_config2-develop-tarball-20261009.json`, añadido en el commit `4d928b5`.

Par	Subconjunto	Bloqueados/n	Cobertura	IC 95 % (Wilson)
C	Todos los payloads	192/200	0.9600	[0.923, 0.980]
C	Sin $J \geq 0.8$ (excluye 21)	171/179	0.9553	[0.914, 0.977]
C	Sin $J \geq 0.9$ (excluye 7)	185/193	0.9585	[0.920, 0.979]
A	Todos los payloads	183/200	0.9150	[0.868, 0.946]
A	Sin $J \geq 0.8$ (excluye 21)	162/179	0.9050	[0.853, 0.940]
A	Sin $J \geq 0.9$ (excluye 7)	176/193	0.9119	[0.863, 0.944]

Cuadro 30: Cobertura de `cmdi` en la Ronda 5, con y sin los payloads que tienen un vecino en entrenamiento.

Fuente: `cmdi_coverage_v2.json` y `cmdi_coverage_v2.md` en `_evidencia/casi-duplicados-v3/`, sobre los resultados de `results_config2.json` (commit `a3aada0`) y `results_config2-develop-tarball-20261009.json` (commit `4d928b5`). Elaboración propia.

Lectura acotada. Los 21 payloads de `cmdi` con vecino a $J \geq 0.8$ están todos bloqueados en ambos pares. Por eso, al excluirlos, la cobertura baja: de 96.0 % a 95.5 % en el par C, y de 91.5 % a 90.5 % en el par A. Los intervalos de los subconjuntos son amplios por su tamaño (179 y 193 payloads).

XIII.2.5. Limitaciones

Las cifras son cotas inferiores. El LSH puede omitir candidatos. Se midió su recall sobre una muestra aleatoria de 2 000 filas de validación (semilla 42): de 1 351 filas con un par exacto de $J \geq 0.8$, el LSH encontró 1 206 (89.3 %); con $J \geq 0.9$, encontró 889 de 911 (97.6 %). Esa medición cubre solo validación; no se midió el recall sobre prueba ni sobre entrenamiento.

Otras limitaciones:

- Se usa una sola normalización del texto y un solo tamaño de shingle (3 caracteres). Otra normalización cambiaría los valores de J.
- El muestreo de recall se hizo con un solo corte aleatorio.
- Solo se comparó el texto de prueba con el de entrenamiento. No se ejecutó inferencia del modelo sobre las filas casi duplicadas.

XIII.2.6. Lectura

Las particiones comparten la estructura de casi-duplicados del conjunto de entrenamiento; las métricas de prueba miden desempeño en esa distribución. Este anexo describe la estructura y no establece su causa.

Archivo	SHA-256
`neardup_v2.py` (script)	d0879391e85e9a3d2972cab462d6207c8a62584dc40953ea30c9c2e8a17e52b7
`neardup_v2_results.json`	f9d56b77de0e3d8788a8de3e9d179fbd7bb2324801c72e4f08ef3e40b355a8c0
`tables.md`	d3620c8a155f7b9ea3aa6a75762c794509208b6475e4c177f049f4679190d8f5
`run.log`	e544d48f603320008941781cdef2b55a5530e6d4fc4a5f84dfa69fadc826ccb8
`train.parquet` (entrada)	979ad08b629b94e974fd80e1fa9ef8f4f146fc800151ca6cb8515898045b5592
`val.parquet` (entrada)	42b6a546e672d9961e45cf75895520590267bfdeeaaa8c4df95ab6e032970ad1
`test.parquet` (entrada)	3e93b7763732cbd5e23738fba5514d64b826d37635ccdf7f6ee941bce3cfe608
`unified.jsonl` (entrada)	3e74be1b64daf292b44adbd1f5472858967a0079af0cdb8748771d733a0951b2
`large_corpus_requests.json` (entrada)	32a171803259cfc4cde87cfb3cf4e740a4eef26d13b562d885ea8abdf7348b1a
`cmdi_coverage_v2.py` (`casi-duplicados-v3`)	038476a29e4b640036b3c94b9f1be382f7ed8594d46d31ae028d9ba19f260c60
`block_gap_counts.py` (`casi-duplicados-v3`)	bee2b350cad08a02af21de2be562106901069c654c5751cbf3185b5a23de9844
`cmdi_coverage_v2.json` (`casi-duplicados-v3`)	27c18067f10a4263d3037520da1ee1f1bd817d11b25220e6d9509c6c30c8c9bc
`cmdi_coverage_v2.md` (`casi-duplicados-v3`)	0c4709e9d175dbd2111bc462d7ba8be122aca45d4273fd5d5de0418737856db8
`block_gap_counts.json` (`casi-duplicados-v3`)	64e1f288c0354247e1fb6844b16409bda3ada23cae2b7bb61e5b023c738142a8
`block_gap_counts.md` (`casi-duplicados-v3`)	2f07a2393b45ada764a032fc54fe4655e6a3809921db6c5f2ede81cfb0e01017
`provenance.md` (`casi-duplicados-v3`)	ab60e7da6227b5107a40bf950ddb4e0e77c28975120c7c6bfb7683090bad4c76

Cuadro 31: Sumas SHA-256 del script, de los resultados de la auditoría de casi-duplicados y de sus entradas.

Fuente: hashes del script, de `neardup_v2_results.json`, `tables.md` y `run.log` en `_evidencia/casi-duplicados/`, y de las entradas según el registro de la ejecución del 11 de octubre de 2026 en `neardup_v2_results.json`. El hash de `large_corpus_requests.json` coincide con el archivo del commit `58033a0` del repositorio logSguarDian-vulnerable-project. La ejecución se hizo sobre el árbol de trabajo del repositorio logSguarDian en el commit `41553a7`, que estaba limpio. Ese commit es ancestro de `origin/develop` (verificado tras `git fetch origin`) y figura en la rama remota `origin/audit/config2-breakdown`. Los archivos `train`, `val`, `test` y `unified.jsonl` no están versionados en git en ese commit; se identifican por su SHA-256. Elaboración propia.

XIII.2.7. Procedencia

XIII.3. Procedencia de las particiones de prueba

El conjunto de prueba se fija mediante un lock: el hash SHA-256 de un archivo que registra un identificador por fila. Cada lock se escribe en `training/splits/test.lock.sha256`, y la evaluación final lo recalcula antes de inferir. El Cuadro 32 resume la cadena de locks del repositorio, tomada de `git log -all - training/splits/test.lock.sha256`.

Commit	Fecha	Lock antes	Lock después	Propósito
`cc00c3c`	11 jun 2026	(no existía)	`3beeaba4...`	Creación del pipeline
`df62619`	23 jul 2026	`3beeaba4...`	`d1a2a284...`	Antes de `v3`
`6867f28`	24 jul 2026	`d1a2a284...`	`7df5adb8...`	Antes de `v4`
`59990ca`	24 jul 2026	`7df5adb8...`	`ee4b85db...`	Antes de `v6`
`5dc7c06`	25 jul 2026	`ee4b85db...`	`251716bb...`	Antes de `v8`
`fa46bdd`	26 jul 2026	`251716bb...`	`4a5e0b97...`	Antes de `v9`
`b43de9c`	2 ago 2026	`4a5e0b97...`	`6c5f253d...`	Antes de `v10`
`8cbe6c9`	30 ago 2026	`6c5f253d...`	`a3140ab3...`	Antes de `v11`
`6d1fa68`, `72aabd6`, `f15e7cc`	23 sep 2026	`a3140ab3...`	`7486c258...`	Re-fijación de `rf_v11` restaurado

Cuadro 32: Cadena de locks del conjunto de prueba. Los tres commits del 23 de septiembre aplican el mismo cambio de lock.

Fuente: `lock_history.md` (`git log` sobre `training/splits/test.lock.sha256` en el repositorio logSguarDian). El mensaje del commit `6d1fa68` es “chore(training): re-lock test set to match the restored rf_v11 generation”. Elaboración propia.

El lock de cada generación se escribe con el método de `training/split.py` (líneas 150–155): el SHA-256 de la concatenación, separada por saltos de línea, de los identificadores `_row_hash` ordenados. `evaluate_test.py` (líneas 189–192) recalcula el mismo valor a partir de los identificadores que define `row_hash` (líneas 171–181). Según la auditoría, este método reproduce 7 486c258692a3cc66fa0ef74087255b904a9ea4d41c8646bb7c0784b15aa4fd9 sobre `test.parquet` con 57 481 filas. El archivo `test.parquet` no está versionado en el repositorio, por lo que esa reproducción no se repitió en este anexo. El hash del archivo de bytes (3e93b7763732cbd5e23738f ba5514d64b826d37635ccdf7f6ee941bce3cfe608) no es comparable con el lock.

El Cuadro 33 muestra la diferencia de soporte entre la partición de v10 y la de v11.

Clase	Soporte en v10	Diferencia en v11
`benign`	14 866	+20
`cmdi`	1 328	+140
`path_traversal`	2 527	+2
`sqli`	34 103	+4
`xss`	4 484	+7
Total	57 308	+173

Cuadro 33: Soporte por clase de la partición de prueba de v10 y diferencia observada en la de v11.

Fuente: `v10_test_results_dc2aaa6.json` (soporte de v10) y diferencias reportadas por la auditoría de particiones. Elaboración propia.

No consta el motivo del cambio de partición del 23 de septiembre de 2026, ni la fuente de los datos restaurados para la generación `rf_v11`.

XIII.3.1. Procedencia

Archivo	SHA-256
`lock_history.md`	b0ce4ce3a4fc73f6eae0d35974bad9b089fdbe747b339713718773fdd22a90fd
`lock_commits.txt`	1c829f096692e3179e9a96bc829fd6f52b5e57069fb99f7654489bed9281922d
`v10_test_results_dc2aaa6.json`	257f05f7d58b85f597485c4f65e4fcb12745cc5d58c5cf6219427b211457acb3

Cuadro 34: Sumas SHA-256 de los archivos de procedencia de las particiones de prueba.
Fuente: `shasum -a 256` sobre `_evidencia/query-v5/`; verificadas contra `SHA256SUMS.txt`. Elaboración propia.

XIII.4. Longitud del *User-Agent* por clase

Este anexo documenta la distribución de la longitud del *User-Agent* (`ua_length`) por clase. Esta distribución sustenta la afirmación de que solo el 0.4 % de las filas benignas tiene un *User-Agent* largo. Se considera largo un valor mayor que 40 caracteres. Las cifras se calcularon sobre las particiones de entrenamiento y validación; no se abrió la de prueba. El Cuadro 35 reporta los valores por clase.

Clase	n	Filas con `ua_length` $>$ 40	Porcentaje	IC de Wilson al 95 %	Porcentaje con `ua_present` $>$ 0
`benign`	84 359	307	0.36	[0.33, 0.41]	0.48
`sqli`	193 261	157 878	81.69	[81.52, 81.86]	81.79
`xss`	25 453	8 463	33.25	[32.67, 33.83]	33.75
`path_traversal`	14 327	13 998	97.70	[97.45, 97.94]	98.07
`cmdi`	8 321	4 521	54.33	[53.26, 55.40]	58.53

Cuadro 35: Longitud del *User-Agent* por clase, en entrenamiento y validación combinados.
Fuente: `_evidencia/ua-v6/ua_length_audit.json`. Elaboración propia.

Las cifras por separado de entrenamiento y validación difieren de las de la unión en 0.6 puntos porcentuales o menos (la mayor, `cmdi` en validación, con 0.57). El mismo cálculo sobre el corpus completo (383 083 filas de `unified.jsonl`, con el script `corpus_ua_skew.py` del repositorio logSguarDian, commit `8f6a895`) da 0.4 % en benigno, 33.3 % en `xss`, 54.4 % en `cmdi`, 81.7 % en `sqli` y 97.7 % en `path_traversal`. La salida se repitió con resultado idéntico. Esta última cifra incluye filas de la partición de prueba; se trata de una descripción de las peticiones y no de una evaluación del modelo, y no se leyó ningún resultado del modelo sobre esa partición.

El script del corpus mide la longitud en puntos de código de Python, y el extractor la mide en unidades UTF-16 de JavaScript. Ambas medidas coinciden salvo con caracteres fuera del plano multilingüe básico. El umbral de 40 caracteres es fijo. El análisis es descriptivo y no establece causas.

XIII.4.1. Procedencia

Los archivos de `_evidencia/ua-v6/` se copiaron desde la carpeta de auditoría `/tmp/lsg-audit/v6/` y se verificaron contra `SHA256SUMS.txt`. Las particiones `train.parquet` y `val.parquet` no están versionadas en este repositorio; sus sumas SHA-256 se listan en el Cuadro 36.

Archivo	SHA-256
ua_length_audit.py	c9385deb84cbe23b6ed67cb3f35fc673655bdddf5e9a835f3cd3ec1b1eb0b00a
ua_length_audit.json	c3922498a40b7daf9e4246fe25c905c5a09000488ac27236637689f7855d2511
RESULTS.md	44aebc8d0fa4710612ee8925f12649dbd12d67d6b026834c19b5c0f2947adee9
corpus_ua_skew_docs_script.py	521cf43137046e9c408b162539ef623bbadd43e3d8bac21662adc7fa3af1c08c
corpus_ua_skew_docs_output.txt	9e7a29910ee29e4fb73bd4689003d16d166c3eb3e9d29b4b3a99ef7322412b33
SHA256SUMS.txt	813dc901a68873a593e1970c6da45ea81805f371e33f30b362fda8c520689339
train.parquet (entrenamiento, no versionado)	979ad08b629b94e974fd80e1fa9ef8f4f146fc800151ca6cb8515898045b5592
val.parquet (validación, no versionado)	42b6a546e672d9961e45cf75895520590267bfdeeaaa8c4df95ab6e032970ad1

Cuadro 36: Sumas SHA-256 de los archivos de evidencia y de las particiones de entrenamiento y validación. Fuente: `shasum -a 256` sobre `_evidencia/ua-v6/` y sobre `training/splits/` en logSguarDian. Elaboración propia.

XIII.5. Generalización entre fuentes (exclusión por fuente con la configuración de rf_v11)

Este anexo mide cuánto depende el desempeño del modelo de fuentes concretas del corpus. Para cada fuente *S*, se entrena un Random Forest con la configuración de `rf_v11` sobre las filas de entrenamiento que no pertenecen a *S*, y se evalúa sobre las filas de *S*. El modelo de cada fold no es el `rf.onnx` evaluado en el capítulo de Resultados.

XIII.5.1. Protocolo

El Random Forest usa los valores de `retrain_v11_contract69.py` (líneas 50–51 en `origin/develop`):

- `n_estimators=30`;
- `max_depth=25`;
- `class_weight=balanced_subsample`;
- `random_state=42`.

Las características son las 69 del contrato de producción, leídas de los archivos `parquet` ya extraídos (regla R1). El archivo `test.parquet` no se abrió (regla R2).

Se auditan las fuentes con al menos 30 filas en train + val (`MIN_ROWS = 30`, `loso_rf_v11.py`). De las 17 fuentes del corpus, 15 cumplen ese mínimo y son las que aparecen en los cuadros. Las dos restantes son `synthetic_evasion` (20 filas en train + val) y `patt_cmdi_curated` (13 filas), según el recuento de `source_counts.txt` (cuadro de procedencia). Ambas tienen menos de 30 filas, por

lo que el script no las audita. El comentario de `training/label_map.yaml` indica 21 payloads para `patt_cmdi_curated` y el recuento de train y val da 13; no consta el motivo.

XIII.5.2. Resultados por fuente

El cuadro 37 reporta, para las 15 fuentes, el bloqueo de sus ataques y la tasa de falsos positivos sobre sus benignos, con todas sus filas en train + val. Las fuentes con $n < 200$ se marcan y sus intervalos son amplios.

Fuente	n	n<200	Ataques	Bloqueo de ataques [IC 95 %]	Benignos	FP [IC 95 %]
capec	181 486		181 486	0.960 [0.959, 0.961]	0	—
modsec_learn	84 698		25 962	0.996 [0.996, 0.997]	58 736	0.753 [0.750, 0.757]
payload_full	25 262		9 959	0.740 [0.731, 0.748]	15 303	0.003 [0.002, 0.003]
payloads_csv	15 650		10 146	0.908 [0.902, 0.913]	5 504	0.266 [0.255, 0.278]
xss_dataset	9 226		6 224	0.995 [0.993, 0.996]	3 002	0.949 [0.941, 0.957]
owasp_logs	3 280		3 280	0.917 [0.907, 0.926]	0	—
seclists_cmdi	2 087		2 087	0.288 [0.269, 0.308]	0	—
command_injection	1 745		401	0.509 [0.460, 0.557]	1 344	0.000 [0.000, 0.003]
synthetic_cmdi_minimal_context	709		709	0.635 [0.599, 0.669]	0	—
synthetic_cmdi_compound	425		425	1.000 [0.991, 1.000]	0	—
synthetic_nav	357		0	—	357	0.739 [0.692, 0.782]
synthetic_cmdi_windows	340		340	1.000 [0.989, 1.000]	0	—
synthetic_multifield	210		210	0.905 [0.857, 0.938]	0	—
synthetic_nav_ecommerce	113	sí	0	—	113	0.788 [0.703, 0.853]
mlops_telemetry_curated	100	sí	100	0.970 [0.915, 0.990]	0	—

Cuadro 37: Exclusión por fuente: bloqueo de ataques y falsos positivos benignos sobre todas las filas de cada fuente (train + val), con intervalo de Wilson al 95 %.

Fuente: `results.json` y `tables_v5.md` (`_evidencia/loso-rf-v11/`, corrida `run2`). Elaboración propia.

El cuadro 38 compara, para las ocho fuentes grandes y sobre sus filas de validación, el modelo de la exclusión (c) con el `rf.onnx` (a) y con un control (b) que quita de entrenamiento un número aleatorio de filas igual al de la fuente, con cinco semillas.

Fuente	n val	(a) bloqueo	(a) FP	(b) bloqueo	(b) FP	(c) bloqueo	(c) FP
capec	32 028	1.000 [1.000, 1.000]	—	1.000 [1.000, 1.000]	—	0.960 [0.958, 0.962]	—
modsec_learn	14 947	0.998 [0.997, 0.999]	0.001 [0.000, 0.001]	0.998 [0.998, 0.998]	0.001 [0.001, 0.001]	0.997 [0.995, 0.998]	0.760 [0.752, 0.768]
payload_full	4 459	0.998 [0.994, 0.999]	0.000 [0.000, 0.002]	0.997 [0.997, 0.998]	0.000 [0.000, 0.000]	0.737 [0.716, 0.757]	0.001 [0.000, 0.003]
payloads_csv	2 761	0.998 [0.994, 0.999]	0.004 [0.002, 0.011]	0.997 [0.996, 0.998]	0.005 [0.004, 0.006]	0.906 [0.892, 0.919]	0.270 [0.243, 0.299]
xss_dataset	1 628	0.990 [0.982, 0.994]	0.002 [0.000, 0.011]	0.991 [0.991, 0.993]	0.000 [0.000, 0.000]	0.993 [0.986, 0.996]	0.945 [0.923, 0.962]
owasp_logs	578	0.990 [0.978, 0.995]	—	0.994 [0.993, 0.995]	—	0.915 [0.890, 0.935]	—
seclists_cmdi	368	1.000 [0.990, 1.000]	—	1.000 [1.000, 1.000]	—	0.299 [0.254, 0.348]	—
command_injection	308	0.958 [0.883, 0.986]	0.000 [0.000, 0.016]	0.952 [0.944, 0.972]	0.000 [0.000, 0.000]	0.563 [0.448, 0.673]	0.000 [0.000, 0.016]

Cuadro 38: Validación de cada fuente grande: (a) `rf.onnx` par C; (b) control de mismo tamaño, media [mín, máx] de 5 semillas; (c) exclusión de la fuente. Los corchetes de (a) y (c) son intervalos de Wilson al 95 %.

Fuente: `tables_v5.md`, sección 2 (`_evidencia/loso-rf-v11/`). El control se describe en `control_wilson_audit.py`. Elaboración propia.

XIII.5.3. Composición de payload_full

El cuadro 39 muestra la clase que el modelo de exclusión de `payload_full` asigna a sus filas `sqli`. El 74.2 % se asigna a `cmdi`, de las cuales el 99.4 % se bloquea. El 25.8 % se asigna a `benign`, y ninguna se bloquea. Este cuadro describe la distribución de predicciones y no atribuye causa a ella.

Clase predicha (argmax)	Filas	% de sqli [IC]	Bloqueadas / predichas [IC]
benign	2 378	0.258 [0.249, 0.267]	0.000 [0.000, 0.002]
cmdi	6 846	0.742 [0.733, 0.751]	0.994 [0.992, 0.996]

Cuadro 39: payload_full: filas sqli de train + val ($n = 9\,224$) según la clase que predice el modelo de exclusión.

Fuente: tables_v5.md, sección 3. Elaboración propia.

XIII.5.4. Lectura acotada

Sin causas, con las cifras de los cuadros 38 y 37. El bloqueo de ataques de (c) frente a (a), sobre las filas de cada fuente en validación, cae de forma distinta según la fuente. Caída grande, de más de 25 puntos porcentuales: payload_full (0.998 a 0.737), seclists_cmdi (1.000 a 0.299) y command_injection (0.958 a 0.563). Caída moderada, de 7 a 10 puntos: owasp_logs (0.990 a 0.915, 7.5 puntos) y payloads_csv (0.998 a 0.906, 9.2 puntos).

Caída leve, de 4 puntos: capec (1.000 a 0.960). Sin cambio: modsec_learn (0.998 a 0.997) y xss_dataset (0.990 a 0.993). Los falsos positivos benignos suben al excluir modsec_learn (0.001 a 0.760), payloads_csv (0.004 a 0.270) y xss_dataset (0.002 a 0.945). El control de mismo tamaño (b) no reproduce estas caídas: su bloqueo queda entre 0.952 y 1.000 en las ocho fuentes.

XIII.5.5. Notas sobre dos fuentes

La fuente owasp_logs aporta 3 860 filas al corpus. Su archivo de origen (training/data_clean/owasp_logs.jsonl, generado por parsers/parse_owasp.py) tiene 56 399 filas. La deduplicación exacta de la propia fuente deja 3 873 (52 526 duplicados internos, 93 %), y la deduplicación global elimina 13 más que coinciden con fuentes anteriores en el orden de unificación (unify.py, línea 94). No se aplica otro filtro ni muestreo: las 3 860 filas se reparten en 2 702 de entrenamiento, 578 de validación y 580 de prueba, y las 3 280 de entrenamiento y validación son las del cuadro de exclusión por fuente. El número de bloques sin etiqueta que descarta el parser (líneas 97–98) no consta.

La fuente mlops_telemetry_curated aporta 119 filas: 82 de entrenamiento, 18 de validación y 19 de prueba. Proviene de la ejecución del pipeline de MLOps del 29 de agosto de 2026 sobre 244 eventos simulados, de cinco fuentes (sim-host-1 a sim-host-5) y una dedicada (sim-host-novel), que reproducen el corpus de extremo a extremo a través del middleware; no es tráfico real. El agrupamiento (DBSCAN, eps 2.5, min_samples 3) formó 17 grupos, uno de ellos de ruido con 53 eventos. Los grupos se etiquetaron a mano en la interfaz de línea de comandos; los conteos son compatibles con 16 grupos etiquetados y el de ruido sin etiquetar, pero no consta un criterio de curación escrito y el registro de decisiones no se conserva. De 130 filas etiquetadas, la deduplicación por _row_hash dejó 119.

Fuente: _evidencia/paquete-y-fuentes-v7/ y los scripts count_owasp.py, dedup_full.py y telemetry_partitions.py, con SHA-256 en el Cuadro 45. Elaboración propia.

XIII.5.6. Limitaciones

- El modelo de cada fold no es el `rf.onnx` evaluado. Los valores de (c) describen modelos de exclusión, no el sistema desplegado.
- No se midieron casi-duplicados entre fuentes. Una fuente puede tener vecinos casi idénticos en otra, lo que afecta la lectura de (c).
- `capec` concentra la mayor parte de `sqli` y comparte `path_traversal` con `owasp_logs`. Excluirla retira una parte grande de la señal de esas clases, según la composición documentada en `docs/results.md` (sección F3.6).
- Hay fuentes con $n < 200$ (`mlops_telemetry_curated`, `synthetic_nav_ecommerce`), cuyos intervalos son amplios.
- Se usa una sola configuración de hiperparámetros. Las cinco semillas se aplican solo al control (b).
- Solo se evalúa Random Forest. No se reporta macro F1 ni se compara con el LOSO anterior (`loso_results.json`).

XIII.5.7. Procedencia

Archivo	**SHA-256**
`loso_rf_v11.py`	e9e7484715471bf35c83aabcffd5e8cc7f74a123ed34117928ca9c2bd23c4db6
`control_wilson_audit.py`	1dbbe7db7628a11c8cfd6de79ca4f7338df06d6994a68ab0006b2f886e9a9a80
`make_tables_v5.py`	48ac94e93436394d936d89d374e4a916892035332509d8660789c25fe57c5c29
`results.json (salida, run2)`	192811fb4900a030da8ab602f75cb9bdcc14c782469a27a4fc145370fe6be8f4
`tables_v5.md (salida)`	574f4b2b73940f74ef705d7790fa6961b2a2735dce8ad5bd46441278cef1bd4e
`sha256_outputs_v5.txt`	03bd017abcf9236253561e4ddd1c4e8842b440fa7b947e8aa96e41d7f56ebecb
`sha256_inputs.txt`	53dc7a69695c36885bf7846d4843162ab5a2707f75a3179bdc924f5b57e2a2be
`train.parquet (entrada)`	979ad08b629b94e974fd80e1fa9ef8f4f146fc800151ca6cb8515898045b5592
`val.parquet (entrada)`	42b6a546e672d9961e45cf75895520590267bfdeeaaa8c4df95ab6e032970ad1
`rf.onnx (entrada, par C)`	25b407e649283606a231665a5a8c8ea1c2c7ad4fd0374a8945b3f119651b0870
`loso_results.json (referencia: LOSO anterior)`	27672b0c7e5f78fa0fee6a602fc6dd0e9a8a9f8d0a0dc90d8ae00c6acab63646
`source_counts.txt` (recuento por fuente, `python3 count_sources.py`)	b4f419e2ff83e1df15fc73a642b6e7b9cf1c780624cb9f9f7aed593fd9843289
`count_sources.py` (script del recuento)	83789ced7c97921aef09452278e4f80098e7477bd7d67ece227c5000ac8ea29b

Cuadro 40: Sumas SHA-256 de los scripts, de las salidas y de las entradas de la auditoría por exclusión de fuente.

Fuente: hashes de `_evidencia/loso-rf-v11/` (`sha256_inputs.txt`, `sha256_outputs_v5.txt`) y de los archivos de la ejecución. Código de referencia: commit `dc380e3` de logSguarDian (`origin/develop`); la corrida repetida reproduce los valores de la corrida anterior en el commit `35b7a28`, según `tables_v5.md`. Elaboración propia.

XIII.6. Reconstrucción de la query en el middleware

Este anexo documenta un cambio en la forma en que el middleware de logSguarDian reconstruye la cadena de consulta antes de extraer las características. Es una verificación posterior a las evaluaciones de la tesis: no modifica los modelos, las métricas ni el corpus.

XIII.6.1. Historia del cambio

Hasta los commits `c3d591a` y `22550ea`, el middleware reconstruía la cadena de consulta con `URLSearchParams` sobre `req.query` (`packages/core/src/middleware.ts`, líneas 537–539 en `c3d591a` y 571–573 en `22550ea`). Express ya había decodificado una vez la consulta, de modo que esta reconstrucción volvía a codificar los signos `%` que quedaban en los valores. El commit `25c7e64` (10 de octubre de 2026), titulado "fix: stop double-encoding the query string before feature extraction", toma en su lugar el fragmento de `req.originalUrl ?? req.url` posterior al primer `?` (`dc380e3:middleware.ts:581-594`). La derivación anterior solo se conserva cuando no existe ninguno de los dos campos.

XIII.6.2. Versiones medidas

El Cuadro 41 enumera las versiones del middleware que se midieron. Todas las que contienen código del middleware usan `URLSearchParams`; ninguna contiene el fragmento introducido por `25c7e64`. El archivo `middleware_log.txt` no contiene ninguna línea sobre `dist/middleware.js` de `f15a9e8`; la línea de `URLSearchParams` de ese tarball proviene de la extracción del tarball, cuyo SHA-256 coincide con el del commit.

Versión medida	Identificador y SHA-256	Reconstrucción de la query en cada una
Paquete 0.1.0 (`c3d591a`)	c5c1a1da57cfb873350ac0c8ab3e7bad395e8cbf507b18cc3272df447880dcb0 (`dist/middleware.js`:490–492)	`URLSearchParams` (`middleware.ts`, línea 539)
`core_worktree_0.1.0.tgz` (commit `f15a9e8` de logSguarDian-vulnerable-project, `vendor/logsguardian-0.1.0.tgz`; ejecución de latencia en Linux)	c8b975b2577fa8e532ba418fcd5c50dc68155ed25d4a8d23f5dbcc91027ad96e	`URLSearchParams` en `dist/middleware.js`
Commit `4d928b5` del mismo repositorio (`vendor/logsguardian-0.1.0.tgz`; validación; `worktree_vendor.tgz`, Ronda 5)	cf1238deb33383975e12058de71c4423b22d7c5bcba5cf47241c0d0f712b0416	`URLSearchParams` en `dist/middleware.js`
`latency_linux.tgz` (resultados de la ejecución de latencia en Linux)	b96d9014085473331fb4859a5dac4810abdb1caaeb2f15f3378877c2290187cec	Contiene resultados de medición, no código del middleware

Cuadro 41: Versiones del middleware medidas y forma de reconstrucción de la query en cada una.

Fuente: hashes calculados con `shasum -a 256` sobre los tarballes de `_evidencia/query-v5/` (`SHA256SUMS.txt`); los commits `f15a9e8` y `4d928b5` se verificaron con `git -C` en logSguarDian-vulnerable-project, donde `vendor/logsguardian-0.1.0.tgz` tiene los mismos SHA-256; código de `c3d591a` verificado con `git show` en logSguarDian. Elaboración propia.

XIII.6.3. Paquetes publicados en npm

El Cuadro 42 compara las dos versiones publicadas en el registro de npm y la forma en que cada una construye la query.

El par B no aparece en ninguna de las dos versiones. La versión 0.1.1 se publicó tras el cambio de versión `20eeafd` (10 de octubre de 2026, 21:53 hora de Guatemala). No constan tags de git ni, para 0.1.0, el commit de origen.

Versión	Publicación (UTC)	SHA-256 del tgz	Par	Umbral de IF	Construcción de la query
0.1.0	2026-08-30T05:07:31 Z	c5c1a1da57cfb873350ac0c8ab3 e7bad395e8cbf507b18cc3272df 447880dcb0	A	0.002486040118540811, fijo (dist/middleware.js :95)	URLSearchParams (dist/middleware.js:490–492)
0.1.1	2026-10-11T04:02:39 Z	abf8d5c47036d9c3a289bc3615 8f1caaa91b9504689aa97193e51 c6808258d15	C	0.004205941820353609, leído de models/parity_report.json (dist/model-metadata.js:20–40; dist/middleware.js:118)	req.originalUrl con corte en "?" y URLSearchParams solo como respaldo (dist/middleware.js:539–543 y 550–552)

Cuadro 42: Versiones del paquete logsguardian publicadas en npm.

Fuente: package_hashes.txt y SHA256SUMS.txt en _evidencia/paquete-y-fuentes-v7/; líneas de código según el tarball de cada versión. Elaboración propia.

El tarball e09120... del par A y el tgz del registro de 0.1.0 (c5c1a1da57cfb873350ac0c8ab3e 7bad395e8cbf507b18cc3272df447880dcb0) tienen modelos idénticos y el mismo umbral de Isolation Forest. Difieren en el código: el tarball espera al Isolation Forest 5 ms (IF_GRACE_MS), mientras que el registro reemplaza esa espera por un parche asíncrono y añade telemetría y webhooks. Ambos construyen la query con URLSearchParams. El tarball cf1238... del par C y el tgz de 0.1.1 (abf8d5c4...) tienen los mismos modelos y el mismo parity_report.json, y difieren en el código en la construcción de la query; difieren además en la nota de texto de data/class_metrics.json y en el campo de versión de package.json, que en el tarball cf1238... es 0.1.0. Por tanto, las evaluaciones de este trabajo que usaron tarball no emplearon el tgz del registro.

XIII.6.4. Réplica de la Ronda 5

Se reprodujeron las 600 peticiones de la Ronda 5 (200 por clase: sqli, path_traversal y cmdi) con el extractor de dc380e3, el modelo rf.onnx del par C (SHA-256 25b407e64928360 6a231665a5a8c8ea1c2c7ad4fd0374a8945b3f119651b0870), el umbral de decisión $T = 0.35$ y la codificación de run_large_corpus.py. Cada petición se evaluó con dos derivaciones de la query: la anterior a 25c7e64 y la de dc380e3. El Cuadro 43 resume la comparación.

Comparación	Filas con diferencias	Filas con diferencias en las 69 de RF	Cambios de clase	Cambios de decisión	Bloqueadas
Anterior frente a dc380e3	0	0	0	0	574 de 600 en ambas versiones

Cuadro 43: Réplica de la Ronda 5 (600 peticiones): diferencias en las características del vector de Random Forest, cambios de clase y de decisión con $T = 0.35$.

Fuente: round5_wire_report.json y round5_wire_stdout.txt (salida de requery_dc380e3_eval.py). Elaboración propia.

Con las 600 peticiones de la réplica, la derivación de dc380e3 no cambia ninguna característica de Random Forest, ninguna clase predicha ni ninguna decisión de bloqueo. Las cadenas de la réplica se generan con la codificación de run_large_corpus.py, que escapa !'()*, usa + para los espacios y no produce los casos del Cuadro 44.

El Cuadro 44 muestra los casos en que las dos derivaciones difieren. Son cadenas con codificaciones no canónicas.

Caso	Cadena cruda	Anterior	dc380e3	Iguales
%20	q=a%20b	q=a+b	q=a%20b	No
%21%27%28%29%2A	q=%21%27%28%29%2A	q=%21%27%28%29*	q=%21%27%28%29%2A	No
%zz	q=%zz%41	q=%25zz%2541	q=%zz%41	No
%7E	q=a%7Eb	q=a%7Eb	q=a%7Eb	Sí
%252e (doble codificación)	q=%252e%252e%252f	q=%252e%252e%252f	q=%252e%252e%252f	Sí

Cuadro 44: Cadenas de consulta en las que las derivaciones anterior y de dc380e3 difieren, y casos de control en los que coinciden.

Fuente: cases_old_vs_new.jsonl. Elaboración propia.

XIII.6.5. Limitaciones

La corrida real con el tarball bloqueó 581 de 600 peticiones (199, 190 y 192 por clase; columna de rf_v11/if_v10 del Cuadro 17), por lo que la réplica no es exacta. Los scripts leen sus entradas desde rutas de /tmp/lsg-audit/, por lo que no son portables sin ajustes.

Una comparación previa de la query almacenada contra la del middleware se descartó porque su referencia no coincidía con la que llega por la red (GET sin codificar y POST con una query construida). El desajuste entre la cadena de entrenamiento y la del middleware no se midió.

Todas las evaluaciones de este documento son anteriores al commit 25c7e64, por lo que las métricas reportadas corresponden a la derivación anterior.

XIII.6.6. Procedencia

XIII.7. Métricas de rf_v10 e if_v9

Este anexo presenta las métricas de la generación v10 (Random Forest rf_v10 e Isolation Forest if_v9), tomadas de v10_test_results_dc2aaa6.json y del mensaje del commit dc2aaa6 (2 de agosto de 2026). El Cuadro 46 resume las métricas por clase de Random Forest sobre la partición de prueba.

El Cuadro 47 presenta el recall por clase de Isolation Forest, junto con las cifras de validación que registra el mensaje del commit dc2aaa6. El umbral de decisión de if_v9 es 0.002486040118540811 (training/models/if_v9_metadata.json, clave threshold, del repositorio logSguarDian).

En prueba, if_v9 alcanza una precisión global de 0.98064 y una tasa de falsos positivos de 0.05159. En validación, el mismo commit registra una tasa de falsos positivos de 0.0574. Las cifras de validación de rf_v10 registradas en el commit son un F1 de 0.9389 para cmdi.

El objetivo específico 2 exige, al cierre del ciclo de entrenamiento del 2 de agosto de 2026 y sobre un conjunto de prueba bloqueado, un F1-score mínimo de 0.80 en al menos tres de las cuatro categorías de ataque y una tasa de detección de anomalías mínima del 50 % con una tasa de falsos positivos máxima del 10 %. Con las métricas de rf_v10 e if_v9 sobre la partición de prueba de 57 308 filas, los criterios del objetivo se cumplen: el F1 de las cuatro categorías de ataque es de 0.93284 (cmdi, el mínimo), 0.98121 (path_traversal), 0.99597 (sqli) y 0.98507 (xss); el recall de Isolation Forest es de 0.91556 y su tasa de falsos positivos de 0.05159. Las cifras no son

Archivo	SHA-256
`requery_dc380e3.js` (script de re-derivación)	2dddcdb52ba96b0bc53c109de23bd451b675b84b480434da366bbf6048071bad
`requery_dc380e3_eval.py` (script de evaluación)	b4dc88eabb69f08fa15a50023f84a52728f78c866241d9a2721f4cef40315066
`round5_wire_vectors.jsonl` (entrada)	bfade2f053043003162cdd710edd99d85bdf2039a3b12b4dcaa3fea506338214
`round5_wire_report.json` (salida)	ad2ee8e8543727c954a103de7372337a53d0063a7d24fa98455907d0e6871701
`round5_wire_stdout.txt` (salida)	d8fe3c811fed1ff22d4c8847267821065a08e6b0a5a74f8eecd302092d6f1641
`cases_old_vs_new.jsonl` (salida)	4421dcc7ff2f468ae39e58fd9ccbcf11271de58f5852f2e35d045233bcb046ee
`middleware_log.txt` (`git show` de `25c7e64`)	13a3d59d580f644c4aadb6963f85730687fed32ecec36c7ef7ac33625f8cc481
`rf.onnx` (modelo, par C)	25b407e649283606a231665a5a8c8ea1c2c7ad4fd0374a8945b3f119651b0870
`package_hashes.txt` (copia en `_evidencia/paquete-y-fuentes-v7/`)	776f334e73d5209a8049ee9372758d1f72eb3d6feae541d0b67d6350acdd376e
`owasp_counts.txt` (copia en `_evidencia/paquete-y-fuentes-v7/`)	5ed8f7888424cd1bb555f10a196072c5b0218ab08fb71108bab5a31a54e0a475
`dedup_full.txt` (copia en `_evidencia/paquete-y-fuentes-v7/`)	50b5229c30af62a850d2c009ad95ce611d7203a7fc2dc8a62d38d49d4b820686
`telemetry_counts.txt` (copia en `_evidencia/paquete-y-fuentes-v7/`)	f641c77d999b6fc435da16f2cfb8c411684e223100da064fc8e660afeed5314f
`SHA256SUMS.txt` (copia en `_evidencia/paquete-y-fuentes-v7/`)	6503e6fcb08087c5cda29872bca2d0db2c5e8ee54ec9f7a1716260411d3ce22e
`count_owasp.py` (copia en `_evidencia/paquete-y-fuentes-v7/`)	d915f0c19a919f7b40c25080f1122972e40009af4aaa513e01bd351aaa8c144c
`dedup_full.py` (copia en `_evidencia/paquete-y-fuentes-v7/`)	3d2ae964804e83699c451fd314c72f9cc19b780c38de7d6bce3f20e1594338c6
`telemetry_partitions.py` (copia en `_evidencia/paquete-y-fuentes-v7/`)	d86929d641755b9acdb9a69266dc567ac8aa8ebe65855a3faf9a692f6a82eff6

Cuadro 45: Sumas SHA-256 de los scripts, las entradas y las salidas de la réplica de la Ronda 5, y de los archivos de paquete y de fuentes copiados a `_evidencia/paquete-y-fuentes-v7/`.

Fuente: `shasum -a 256` sobre `_evidencia/query-v5/` y `_evidencia/paquete-y-fuentes-v7/`; verificadas contra `SHA256SUMS.txt`. Elaboración propia.

Clase	Precisión	Recall	F1	Soporte
`benign`	0.99805	0.99865	0.99835	14 866
`cmdi`	0.92456	0.94127	0.93284	1 328
`path_traversal`	0.98101	0.98140	0.98121	2 527
`sqli`	0.99560	0.99633	0.99597	34 103
`xss`	0.99164	0.97859	0.98507	4 484
Macro promedio	0.97817	0.97925	0.97869	57 308
Exactitud			0.99361	57 308

Cuadro 46: Métricas de `rf_v10` sobre la partición de prueba (57 308 filas).

Fuente: `v10_test_results_dc2aaa6.json`, sección `rf_v10_test`. Elaboración propia.

Clase	Recall (prueba)	Recall (validación)
`sqli`	0.89948	No registrado
`xss`	0.97993	No registrado
`path_traversal`	0.99763	No registrado
`cmdi`	0.95482	0.9503
Global	0.91556	0.9154

Cuadro 47: Recall de `if_v9` por clase y global, sobre prueba y sobre validación.

Fuente: `v10_test_results_dc2aaa6.json` (prueba) y mensaje del commit `dc2aaa6` (validación). Elaboración propia.

comparables con las de `rf_v11`, por tratarse de otra partición.

Las métricas de v10 y las de v11 no son comparables, por tres razones:

- La partición de prueba de v10 tiene 57 308 filas y su lock es 6c5f253d99f23309ae34f5c4 badcf4635dee06bf3fadc592c6cfb5a2319cfad8 (commit `b43de9c`). La de v11 tiene 57 481 filas y lock 7486c258692a3cc66fa0ef74087255b904a9ea4d41c8646bb7c0784b15aa4fd9. El Cuadro 33 detalla la diferencia de soporte por clase.
- El JSON de v10 no registra el lock ni el hash de la partición con que se evaluó.
- La lectura única por generación que atribuye `docs/model-comparison.md` (líneas 100–102) no consta en los registros revisados. El registro de `rf_v11` documenta tres lecturas de `test.parquet` (24, 25 y 26 de septiembre de 2026) en `docs/decision-policy.md` (línea 51) y en la nota `eval_note` de `packages/core/data/class_metrics.json`.

XIII.8. Evaluaciones posteriores del coautor sobre la Ronda 5

Este anexo reúne mediciones sobre el corpus de la Ronda 5 que ejecutó Sebastián Huertas Gómez en el repositorio de la aplicación vulnerable. El autor de este trabajo no las ejecutó ni las reprodujo, y no forman parte de los criterios de aceptación. Los archivos no registran la fecha de ejecución, solo la de los commits. Los commits `963843e` (18:24), `20cbd88` (20:52) y `00ea3a3` (22:37) tienen fecha del 10 de octubre de 2026 en hora local (UTC−6), y su autor es el usuario de Git `xtsebas`. El commit `00ea3a3` corresponde al 11 de octubre de 2026 en UTC.

Configuración	sqli	path_traversal	cmdi	xss	Total (600)
Configuración 1, sin protección	0/200	0/200	0/200	0/1	0/600
Configuración 2, par A (tarball `e09120...`)	200/200	170/200	183/200	[a]—	553/600
Configuración 2, par C (tarball `cf1238...`)	199/200	190/200	192/200	[a]—	581/600
Configuración 3a, WAF	199/200	197/200	136/200	1/1	532/600
Configuración 3b, WAF y logSguarDian	200/200	200/200	197/200	1/1	597/600
Paquete 0.1.1, arnés original	199/200	190/200	192/200	1/1	581/600
Paquete 0.1.1, arnés corregido	199/200	200/200	185/200[b]	1/1	584/600

Cuadro 48: Bloqueos por categoría en las mediciones del coautor sobre la Ronda 5.

Fuente: conteo propio de los campos `blocked` y `status` de los archivos `results_*.json` de `_evidencia/coautor-ronda5-v10/`. Bloqueado significa HTTP 403; los errores y los timeouts cuentan como no bloqueados. (a) En las filas del par A y del par C no se reporta xss: los archivos de 608 entradas de los commits `a3aada0` y `4d928b5` contienen ocho marcadores xss escritos a mano, que no se evalúan. El resultado de xss de las demás filas es el único caso hallado con ZAP (un payload, una petición por configuración). (b) Incluye un timeout (`cmdi_7`). Elaboración propia.

De los paquetes usados en las Configuraciones 1, 3a y 3b no consta registro en los resultados. El directorio `vendor` del commit `963843e` contiene el tgz con SHA-256 c5c1a1da57cfb873350ac 0c8ab3e7bad395e8cbf507b18cc3272df447880dcb0, que es el de la versión 0.1.0 del registro. La columna del par A usa el tarball e09120006dd17e790838f2ae92bb68ee4c7f3a5e160c2c4c83f5a51 3909fe359 (commit `a3aada0`), y la del par C el tarball cf1238deb33383975e12058de71c4423b22d 7c5bcba5cf47241c0d0f712b0416 (commit `4d928b5`). La corrida de la Configuración 2 de `963843e` tiene como vendor el tgz del registro, pero no consta que se ejecutara con él.

El arnés original codifica cada payload con quote_plus, por lo que los payloads que ya contienen secuencias %xx llegan doblemente codificados. El cambio del commit 00ea3a3 los envía sin codificar, y es lo único que cambia entre las dos corridas de 0.1.1. Con el arnés original, 0.1.1 da exactamente los mismos resultados que el tarball del par C. Por tanto, la mejora de path_traversal de 190 a 200 no se debe al cambio de la query del commit 25c7e64, sino al arnés: los 10 casos que cambian pasan de 404 a 403.

En cmdi cambian 13 veredictos, y 13 payloads con & sin codificar se truncan en el servidor. Por ello, 185/200 depende en parte del arnés. Ninguna de las dos variantes entrega los 200 payloads de cmdi como se pretendía. Los fallos de path del par A son todos respuestas 404 de la aplicación (30 de 200). Sin protección, 184 de 200 payloads de path de la Configuración 1 dan 404, y el arnés los cuenta como no bloqueados.

Las dos corridas de 0.1.1 usaron el paquete instalado desde el registro, según el lock de la rama de prueba: el integrity SHA-512 de package-lock.json coincide con el del tgz de npm pack de 0.1.1 (SHA-256 abf8d5c47036d9c3a289bc36158f1caaa91b9504689aa97193e51c6808258d15). Con el arnés original, path da 190/200 (intervalo de Wilson al 95 %: 91.0–97.3 %) y cmdi da 192/200 (92.3–98.0 %). Con el arnés corregido, path da 200/200 (98.1–100.0 %) y cmdi da 185/200 (88.0–95.4 %), con un timeout. La cifra de 200/200 se midió sobre el corpus que reveló los defectos, por lo que es optimista. El límite inferior de path con el tarball del par A (79.4 %) no se reproduce con 0.1.1 y el arnés original (91.0 %).

Los archivos vendor/*.tgz y models/* del commit 00ea3a3 están vacíos, y no consta el motivo.

XIII.8.1. Escaneo con OWASP ZAP

OWASP ZAP 2.17.0 realizó un escaneo autenticado como el usuario alice sobre localhost:3000, con spider y escaneo activo. El escaneo produjo 538 alertas en total. Una de ellas es una alerta de XSS reflejado en POST /profile, campo bio, con el payload <scrIpt>alert(1);</scRipt> (id xss_placeholder_0 de la Ronda 5).

El número $n = 1$ significa un payload y una petición por configuración, sin repeticiones. Los resultados son 0/1 sin protección y 1/1 en las configuraciones 2, 3a y 3b. El paquete que ejecutaba la aplicación no consta, y tampoco constan la política de escaneo ni el ajax spider. Un solo caso no permite afirmar el cumplimiento del criterio del 80 % para xss.

Archivo o tarball	Origen	SHA-256
results_config1-round5-20261010.json	20cbd88	e4b44ca4b21c023b778c2204c69e58b0a334a128340680aab91a2ebc21a894d3
results_config2-parA-a3aada0.json (results_config2.json)	a3aada0	ed3fbc91b80f68483ffef5ae9dc432dab9ce104836ddbca1a2295dd873031054
results_config2-published-with-real-xss-20261010.json	963843e	e56dc582e2ce57bd3bf4d80e204dd87d0aa17632da4af7f9897c0f574139d534
results_config2-parC-4d928b5.json	4d928b5	dd4ea3c565f9368fabf146c56d1c48de08212e2c0c2e5f677a550532baa36bc9
results_config2-with-real-xss-20261010.json	963843e	80d4ee8251e2758292f23956a59067ea395d3637810e40433a59046856731273
results_config3a-round5-20261010.json	20cbd88	def99facb115e38295243957a1ed297839e0d97cd50fa2bf9eb94d822f9fbd2d
results_config3b-round5-20261010.json	20cbd88	0f854afece9844a270c2aa6fe3883e90faf5786e1a4b085b5be381ba9028144e
results_config2-published-0.1.1-20261010.json	00ea3a3	80d4ee8251e2758292f23956a59067ea395d3637810e40433a59046856731273
results_config2-published-0.1.1-harness-fixed-20261010.json	00ea3a3	66958151c52888a9c629ff440fdc0759a244ba25fb4ba5cb89916fd6b4f03490
zap-xss-report.json	963843e	85389aff3510438d2577ff361da7dc3015557f46bb292b60b9c47bd37ac235b0
zap-alerts-full.json	963843e	bde073136cf0ec61433e466938c1e21a699b84fe7ebc0d8d1fd2b1f1c3291d88
run_large_corpus-origin-main-vs-00ea3a3.diff	origin/main–00ea3a3	a304b458cad3162ecbd02fd42767c8028829b185b47d03c81ccc7e68c03cf117
cmdi_flips.md	Auditoría v10	15d08b706978f224c157a0930cacb40fd7112b38e12abc4ecf03000c7cf7c665
manifest_v10.txt	Auditoría v10	55af9fdc2a4684a287e493bc462b79c485da97020ff5f09a8eff8127111753cd
Tarball de la Ronda 5 original (par A)	a3aada0	e09120006dd17e790838f2ae92bb68ee4c7f3a5e160c2c4c83f5a513909fe359
Tarball de validación (par C)	4d928b5	cf1238deb33383975e12058de71c4423b22d7c5bcba5cf47241c0d0f712b0416
Tgz de 0.1.0 del registro (vendor de 963843e)	Registro npm	c5c1a1da57cfb873350ac0c8ab3e7bad395e8cbf507b18cc3272df447880dcb0
Tgz de 0.1.1 del registro (npm pack)	Registro npm	abf8d5c47036d9c3a289bc36158f1caaa91b9504689aa97193e51c6808258d15

Cuadro 49: Procedencia de los archivos copiados a _evidencia/coautor-ronda5-v10/ y de los tarballes citados.

Fuente: SHA256SUMS.txt de _evidencia/coautor-ronda5-v10/ (shasum -a 256); los commits se verificaron con git show sobre logSguarDian-vulnerable-project. Elaboración propia.